from zero to production patterns

Event-Driven Agentic AI with Spark Structured Streaming & Kafka

Events, Kafka, Spark streaming and LLM agents explained from the ground up, then combined into real-time agent systems.

L1 · Easy: e-commerce order support | L2 · Mid: airline disruption | L3 · Advanced: payments fraud triage

Akash Sharma

1

Event-driven architecture from zero

What an event is, how Kafka stores events, and the patterns teams use to build systems out of them.

What is an event?

past tense:
it already
happened

immutable:
never edited,
only followed by
new events

event: OrderShipped

key: order-981
time: 2026-10-15 14:02:11
payload: { carrier: "BlueDart",
eta: "2026-10-18" }

has a KEY
(what it is about)

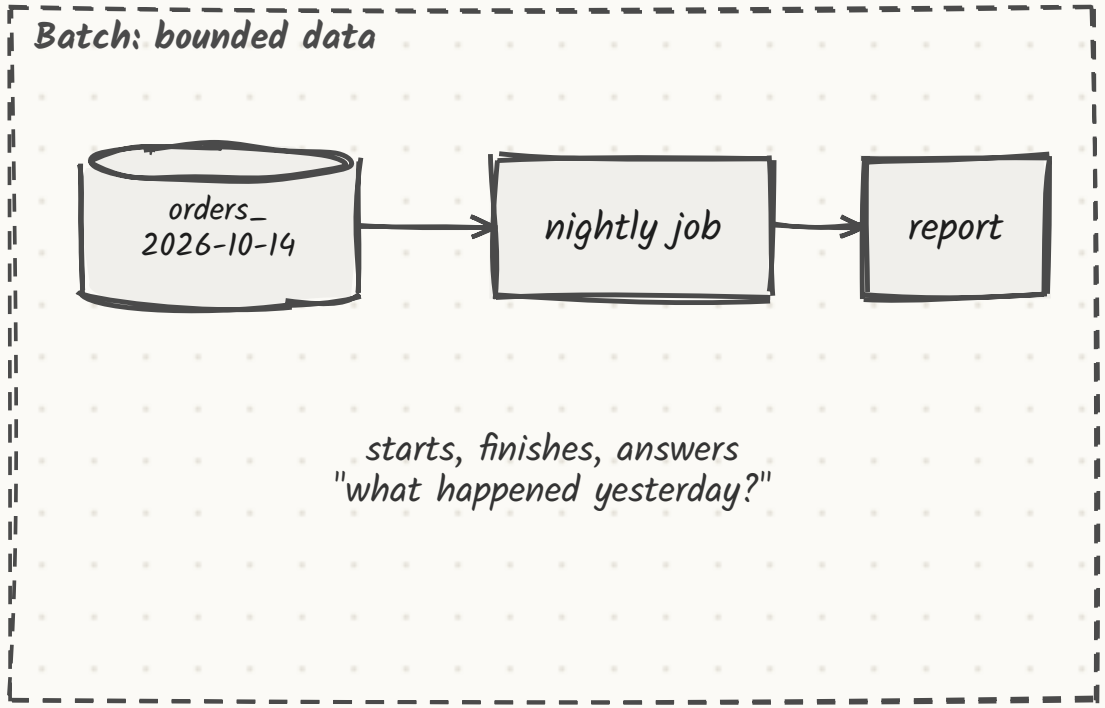
has a TIME
(when it happened)

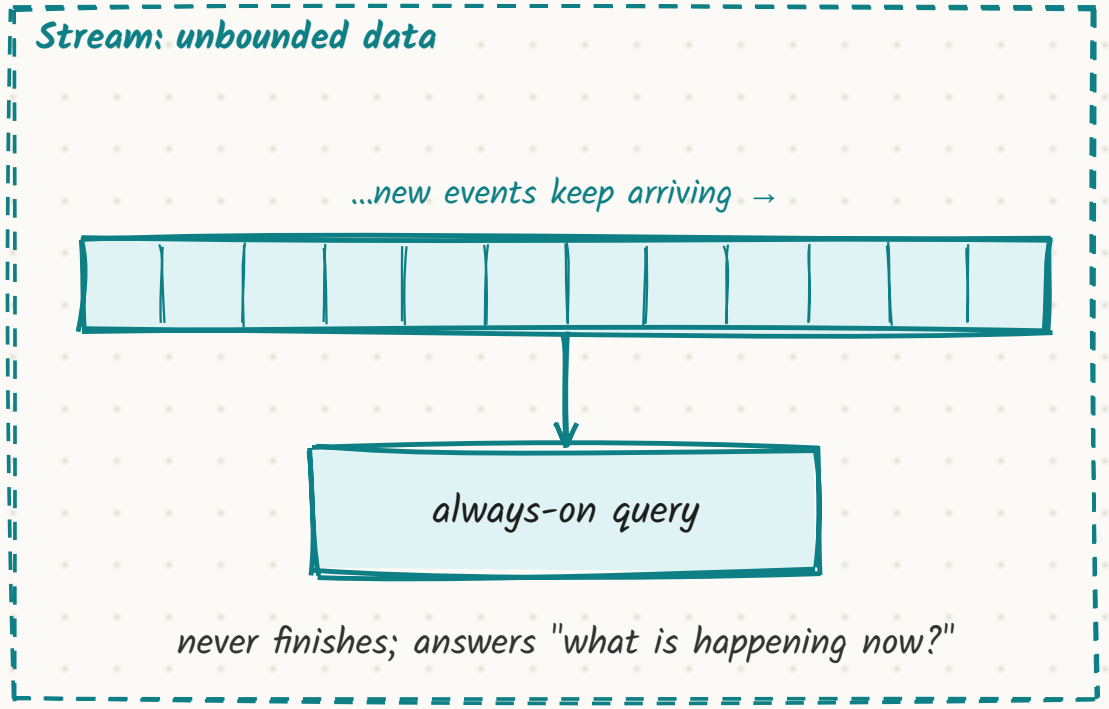
Events, commands and queries

Three kinds of messages; only one of them is an event

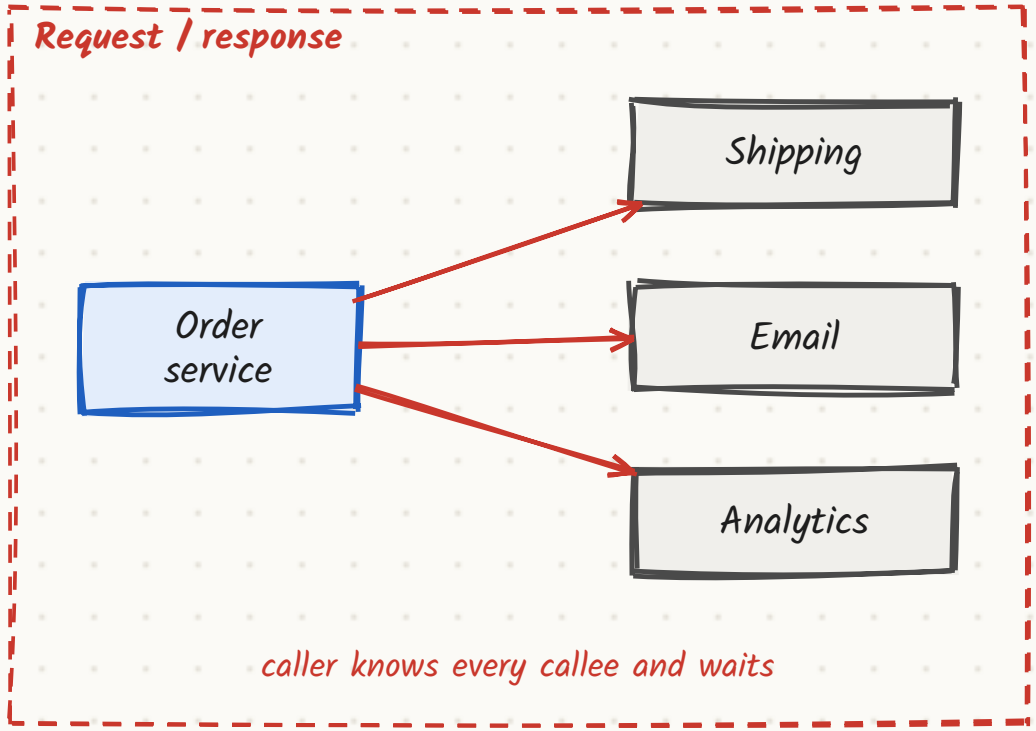
EVENT	COMMAND	QUERY
"OrderShipped"	"ShipOrder"	"GetOrderStatus"
fact · past tense	request · imperative	question · no side effect
many listeners	one handler, may fail	needs an answer now
broadcast it	send it	ask it

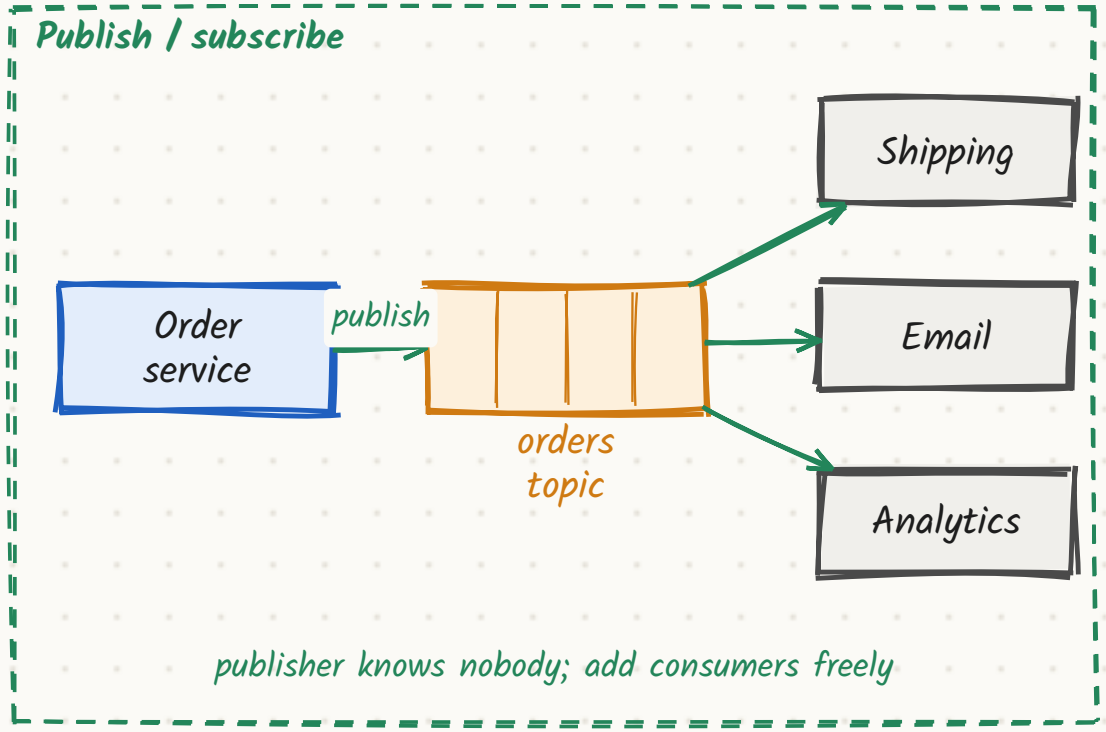
Batch vs. stream



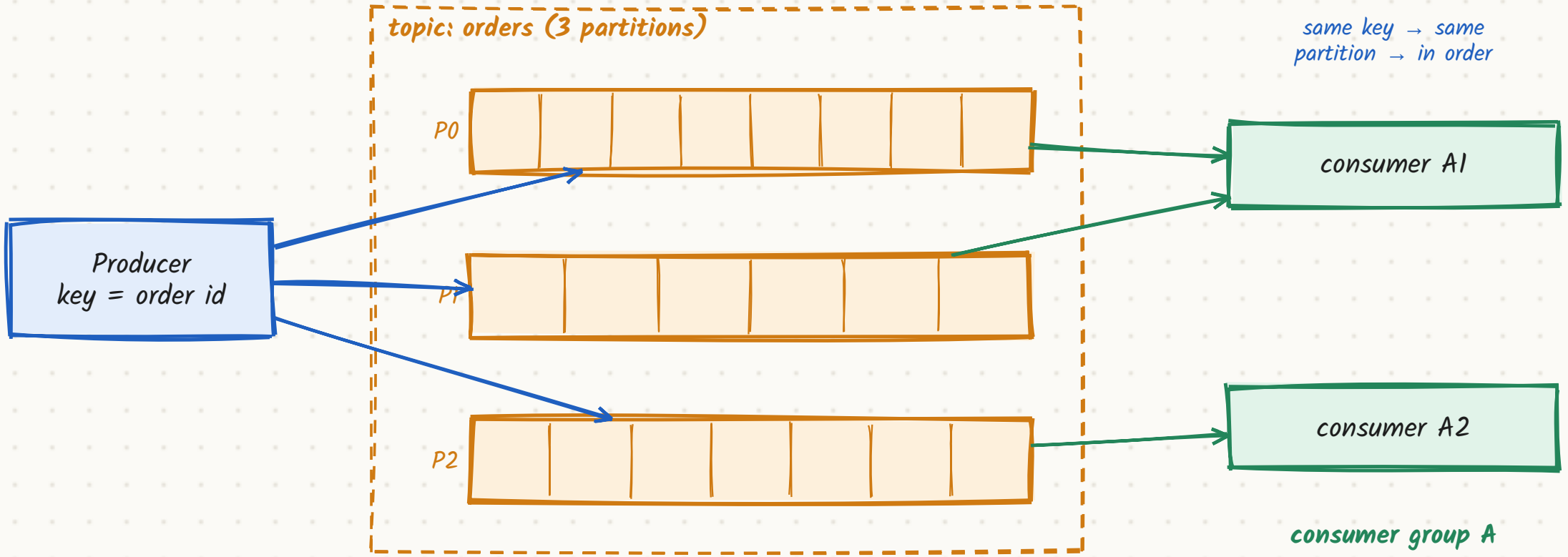


Request/response vs. publish/subscribe





Kafka in one picture



each box = one record at an OFFSET (0, 1, 2 …); new records append on the right

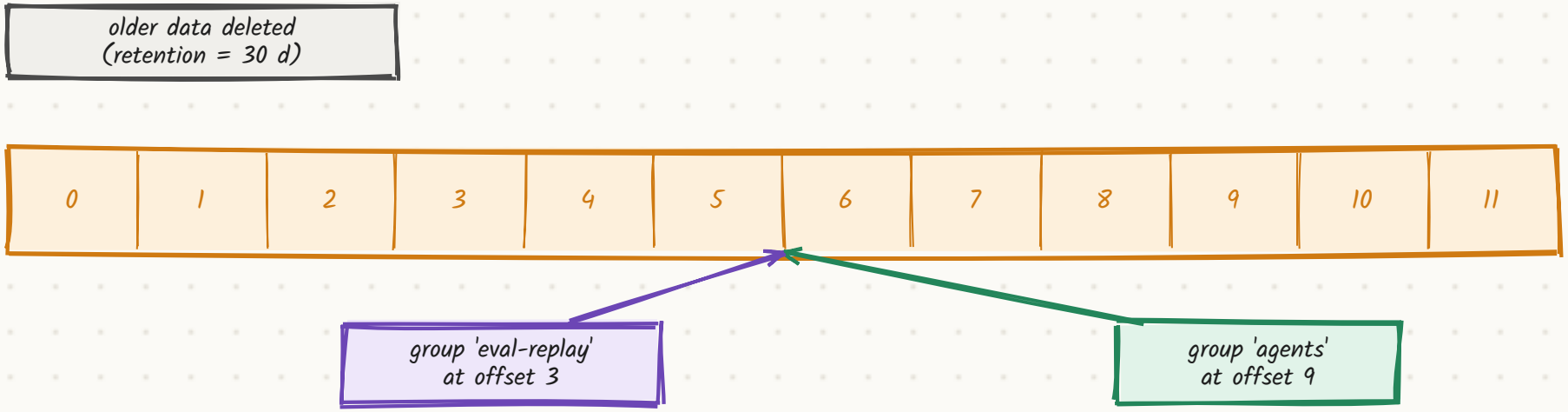
Keys, partitions and ordering

- The producer picks a **key** (order id, conversation id).
- Kafka hashes the key → the record always lands in the **same partition**.
- Order is guaranteed **only within a partition**.
- So: all events of one order are in order; events of different orders are not.

Choose the key = the thing whose events must stay in order.

For agents: ***conversation id*** *or* ***entity id*** *(order, flight, card).*

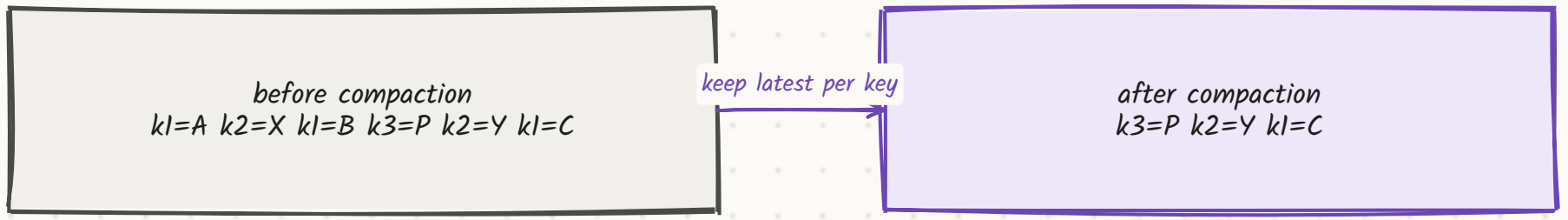
Consumer groups, offsets and replay



Reading does not delete. Each group remembers its own position, so a new group can replay history.

- A **consumer group** = a team of consumers sharing a topic.
- Each partition is read by **one** member of the group.
- The group **commits** the offset it has finished.
- Another group has its own offsets → can **replay** from the start.

Retention and compaction



a compacted topic is a changelog of a table: perfect for "latest state per key"

- **Delete** policy: keep records for N days (e.g. 30), then drop the oldest.
- **Compact** policy: keep the latest record per key.
- A **null value** (tombstone) deletes the key.

Delivery guarantees

Guarantee	What can happen	Typical cause	What you do
At-most-once	messages can be lost	commit offset before processing	only for metrics you can lose
At-least-once	messages can be duplicated	process, crash before commit, re-read	make consumers idempotent (default choice)
Exactly-once	each message affects the result once	replayable source + checkpoint + idempotent/transactional sink	Spark micro-batch + idempotent sink; Kafka transactions inside Kafka

Idempotency: making "twice" safe

- **Idempotent** = doing it twice has the same effect as doing it once.
- Give every event a unique **eventId**.
- Consumers remember processed ids (or use upserts by id).
- Side effects carry an **idempotency key** to the external API.

Refund API called twice with the same **Idempotency-Key: e-7f3a**
→ one refund.

Without it → two refunds.

Four ways to use events

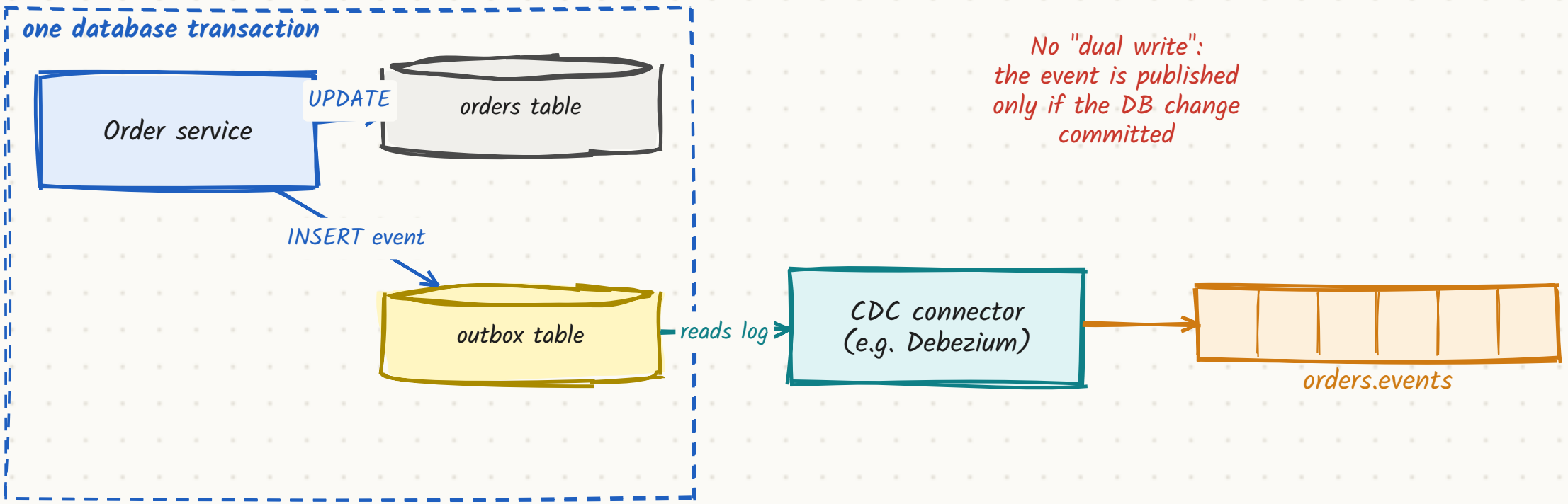
1 · Event notification
"OrderShipped(order-981)"
thin event; consumers call back for details

2 · Event-carried state transfer
"OrderShipped(order-981, carrier, eta, items…)"
fat event; consumers keep their own copy

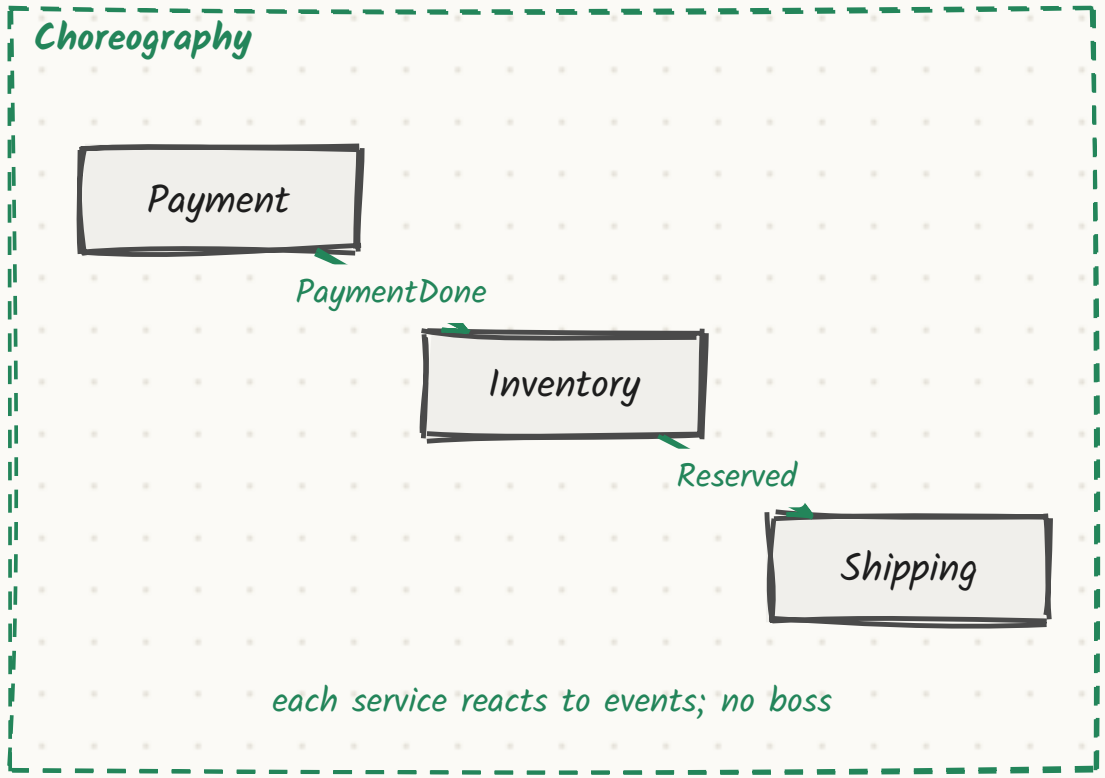
3 · Event sourcing
the log of events IS the source of truth;
current state = fold(events)

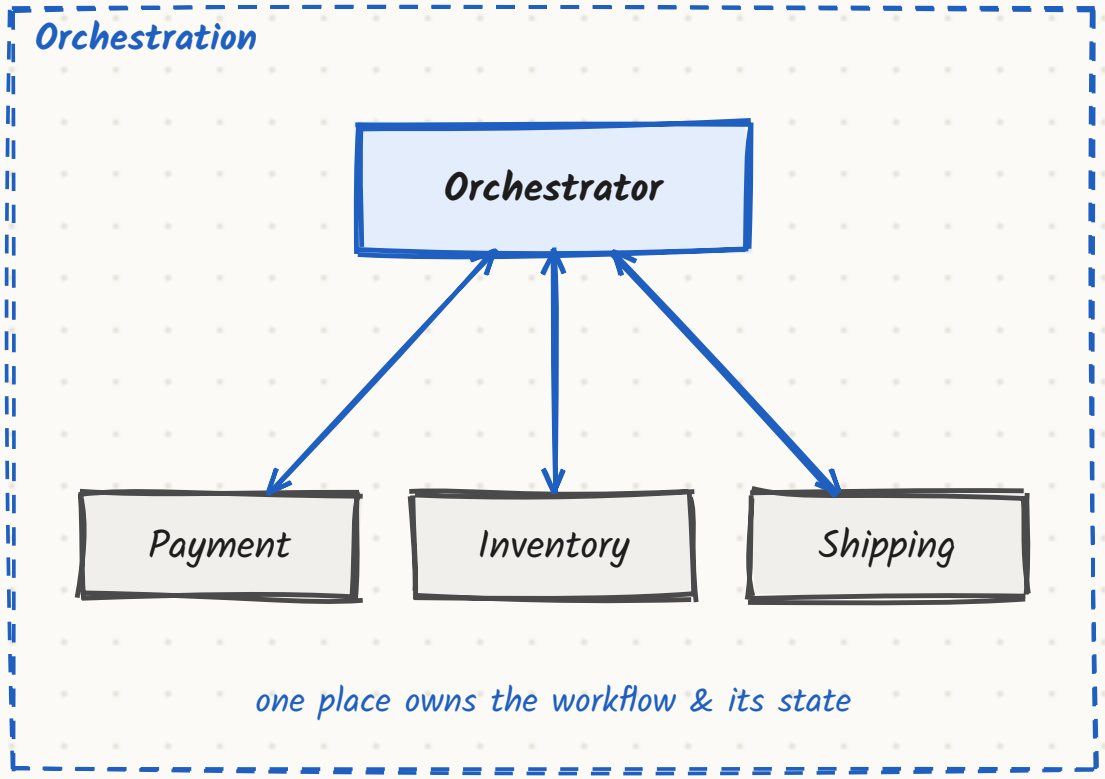
4 · CQRS
writes go one way (commands → events),
reads come from views built from events

Getting events out of databases: CDC and the outbox



Choreography vs. orchestration

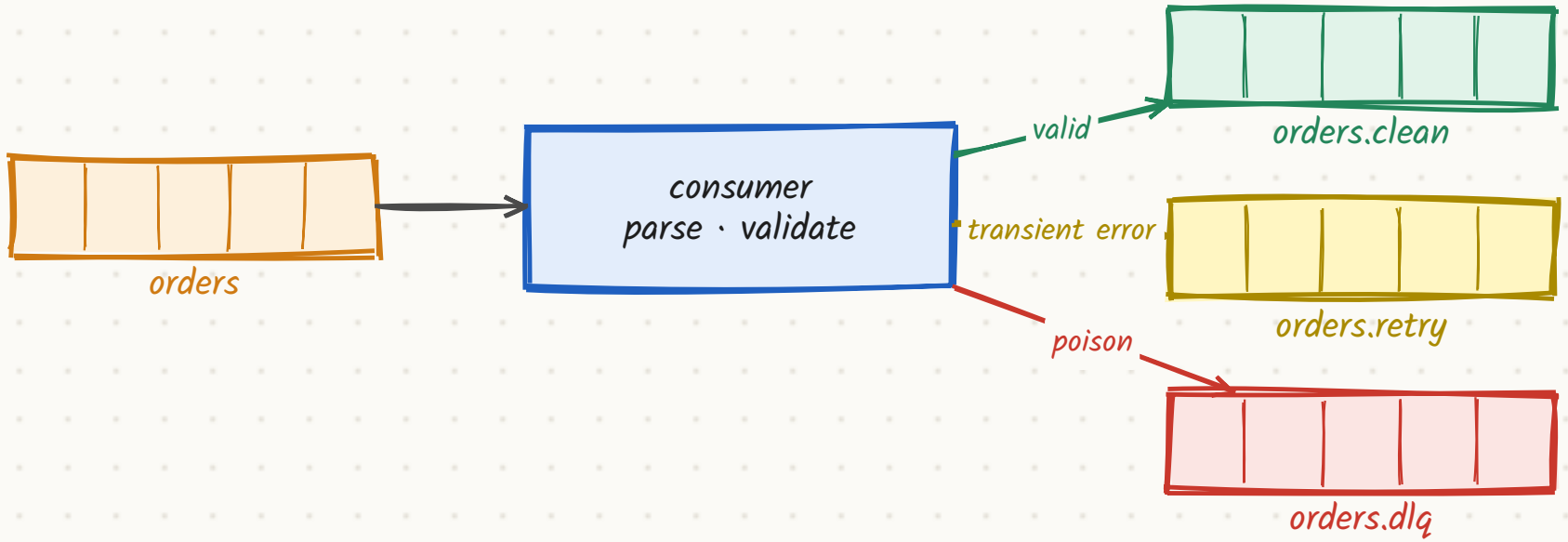




Schemas and contracts

Change	Old consumers OK?	New consumers on old data OK?	Rule of thumb
Add optional field (with default)	yes	yes	always safe
Remove optional field	yes	yes (default used)	safe if it had a default
Rename a field	no	no	add new + deprecate old
Change a field's type	no	no	new field or new topic version
Change meaning, same name	silently wrong	silently wrong	never; bump the event type

When things go wrong: retries, DLQs and backpressure



DLQ record keeps:
- original bytes
- error message
- source offset
- attempt count

Part 1 recap: the vocabulary

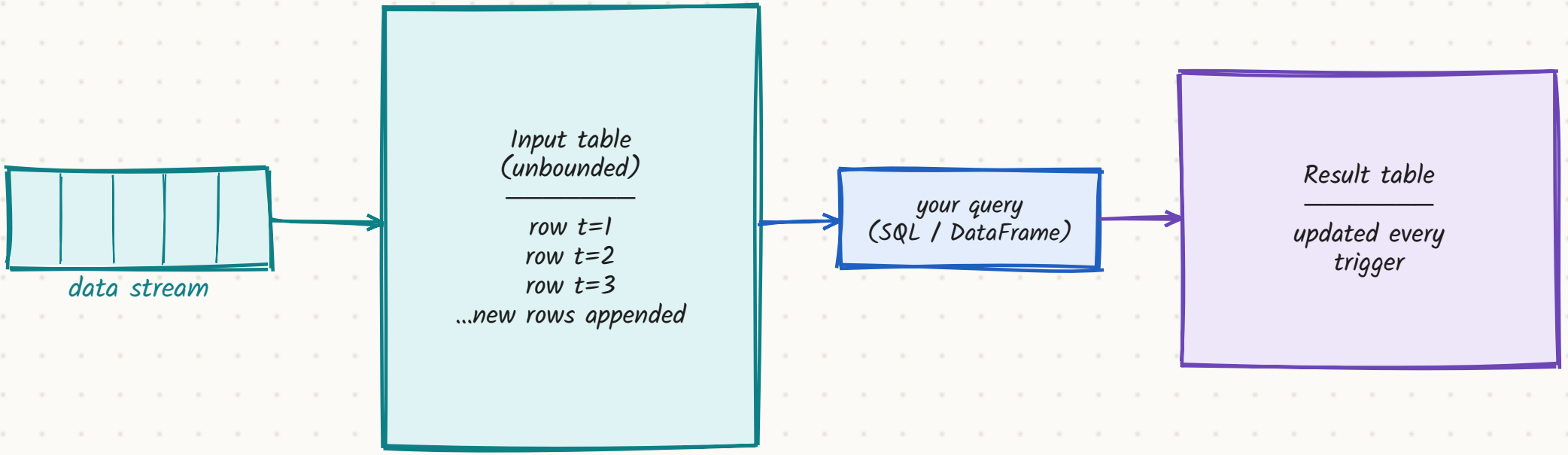
Term	Meaning
Event	An immutable fact that something happened, with a key and a time
Topic	A named, append-only log of events in Kafka
Partition	A slice of a topic; order is guaranteed only inside one partition
Offset	The position of a record inside a partition
Key	Decides the partition: same key → same partition → ordered
Producer / consumer	Writes to / reads from topics
Consumer group	Consumers sharing the work; each partition goes to one member
Retention / compaction	Keep for N days / keep only the latest value per key
Idempotent	Doing it twice has the same effect as doing it once
DLQ	Dead-letter queue: where records that cannot be processed are parked

2

Spark Structured Streaming from zero

How Spark turns an endless stream into small, safe, repeatable steps, and how it remembers things between them.

The big idea: a stream is a table that keeps growing



You write the query as if the table were static. Spark runs it incrementally on new rows.

Your first streaming query

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._

val spark = SparkSession.builder.appName("first-stream").getOrCreate()

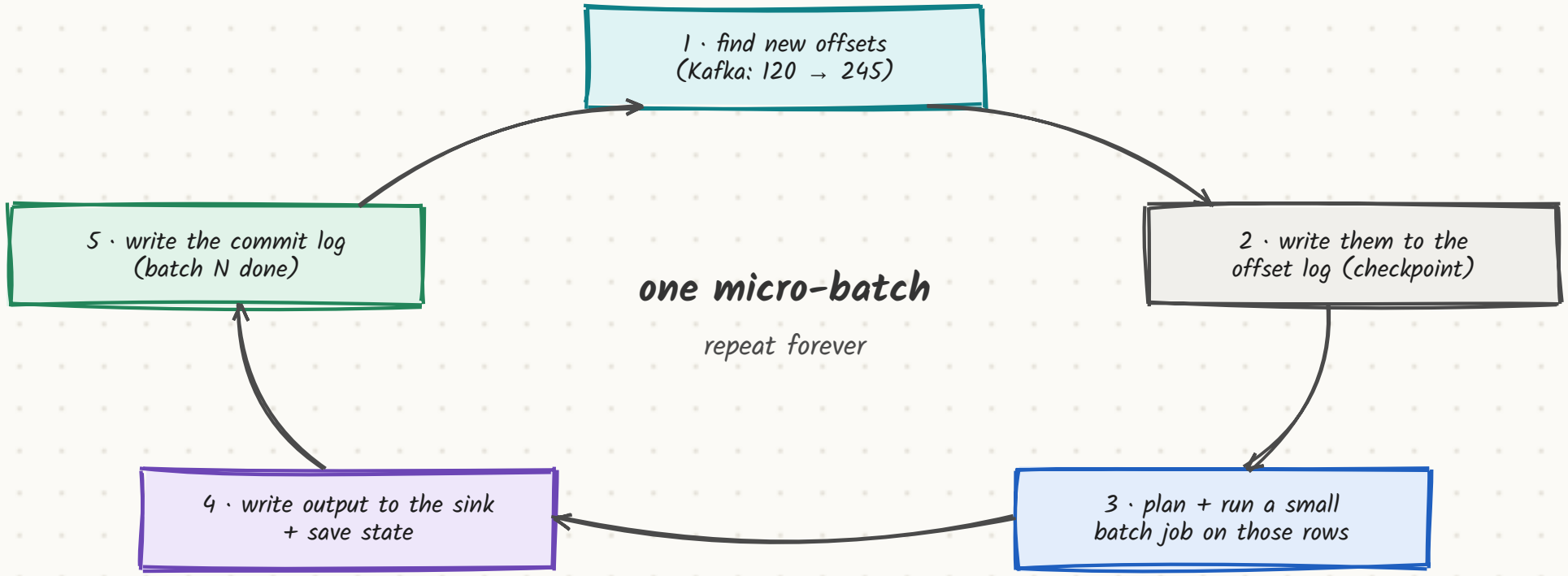
val orderSchema = new StructType()
  .add("orderId", StringType).add("status", StringType)
  .add("amount", DoubleType).add("eventTime", TimestampType)

val orders = spark.readStream                          // 1. SOURCE: an unbounded table
  .format("kafka")
  .option("kafka.bootstrap.servers", "localhost:9092")
  .option("subscribe", "orders")
  .option("startingOffsets", "latest")
  .load()
  .select(from_json(col("value").cast("string"), orderSchema).as("o"))
  .select("o.*")

val shipped = orders.where(col("status") === "SHIPPED")    // 2. QUERY: ordinary DataFrame code

shipped.writeStream                                    // 3. SINK + checkpoint + trigger
  .format("console")
  .option("checkpointLocation", "/tmp/chk/first-stream")
  .outputMode("append")
```

Under the hood: the micro-batch loop



Sources and sinks

	Name	Notes
Source	Kafka	replayable, fault-tolerant; the default for event-driven systems
Source	Files (json, parquet, csv...)	new files in a folder become rows
Source	Rate / socket	testing only
Sink	Kafka	at-least-once; write key, value, headers, topic
Sink	Files / Delta / Iceberg	exactly-once via the checkpoint and commit protocol
Sink	foreachBatch	any batch writer (JDBC, Redis, vector DB); micro-batch only
Sink	foreach (ForeachWriter)	row-by-row custom writer
Sink	console / memory	debugging and tests

Checkpoints: where Spark remembers its progress

```
/chk/first-stream/
├── metadata            # query id (stays the same across restarts)
├── offsets/            # write-ahead log: "batch 41 will read Kafka offsets 120..245"
│   ├── 40
│   └── 41
├── commits/            # "batch 40 finished and its output is written"
│   └── 40
├── sources/0/          # source-specific info (Kafka: initial offsets)
└── state/              # only for stateful queries: one folder per operator/partition
    └── 0/0/ 1.delta 2.delta 10.snapshot …
```

Triggers: when does Spark run?

Trigger	Behaviour	Use it for
default (unspecified)	next micro-batch starts as soon as the previous one ends	lowest micro-batch latency
ProcessingTime("30 seconds")	a micro-batch every 30 s (skipped if no data)	steady pipelines, fewer small files
Once (deprecated)	one batch with everything available, then stop	replace with AvailableNow
AvailableNow (3.3+)	process all available data in several batches, then stop	scheduled jobs, backfills, replays
Continuous("1 second") (experimental)	long-running tasks; checkpoint every 1 s	~ms, stateless map-like pipelines

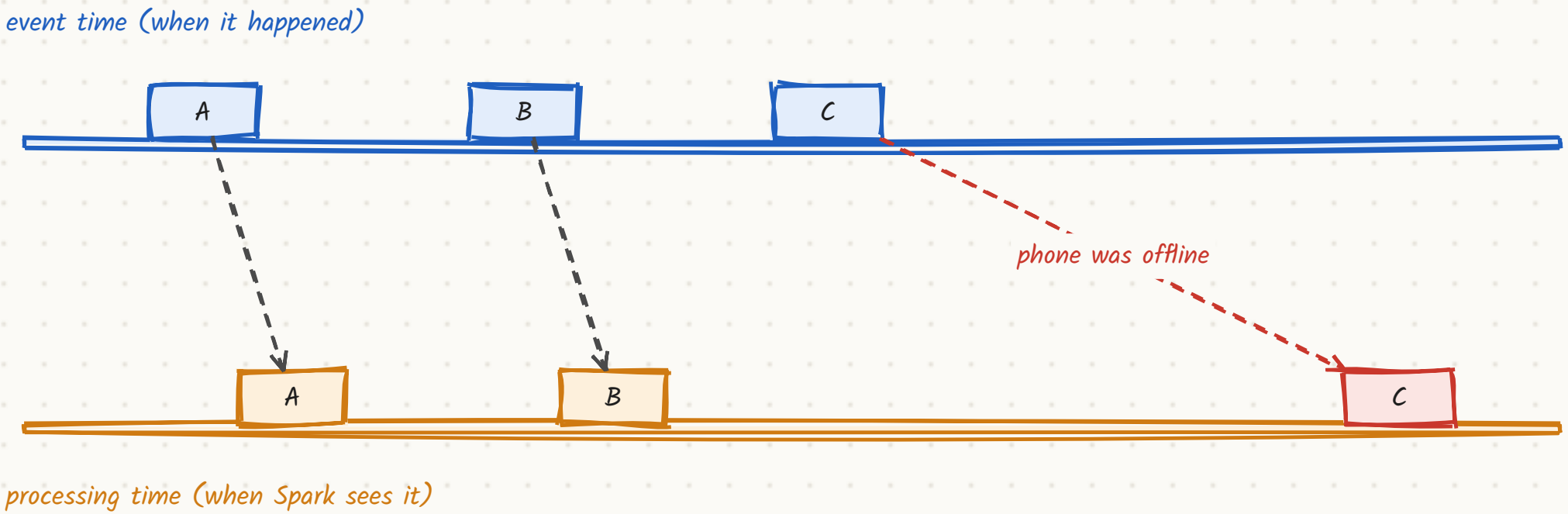
Output modes: what gets written each time?

Mode	What is written each trigger	Works with
append	only rows that will never change again	stateless queries; windowed aggregations with watermark; joins
update	only rows that changed since the last trigger	aggregations; mapGroupsWithState
complete	the whole result table	aggregations only; result must stay small

Stateless and stateful operations

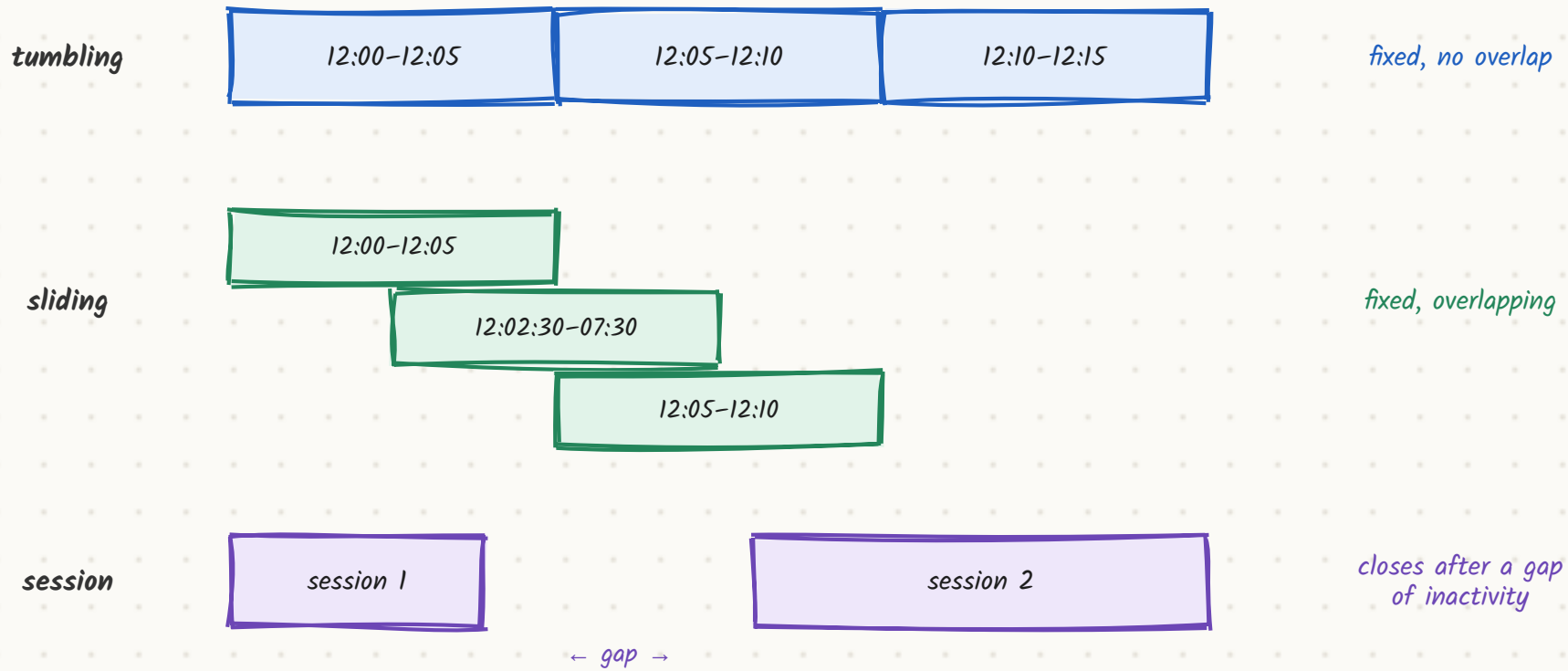
Operation	Stateful?	Needs watermark?	Notes
select, filter, map, UDF	no	no	also allowed in continuous mode
stream-static join	no	no	static side re-evaluated each batch
groupBy + agg	yes	for append mode	state grows with keys / windows
dropDuplicates	yes	to bound state	dropDuplicatesWithinWatermark in 3.5
stream-stream join	yes	yes for outer joins	plus a time-range condition
(flat)mapGroupsWithState	yes	for event-time timeouts	your own state and logic
sort, limit, distinct	—	—	not supported on streams (use foreachBatch)

Event time vs. processing time

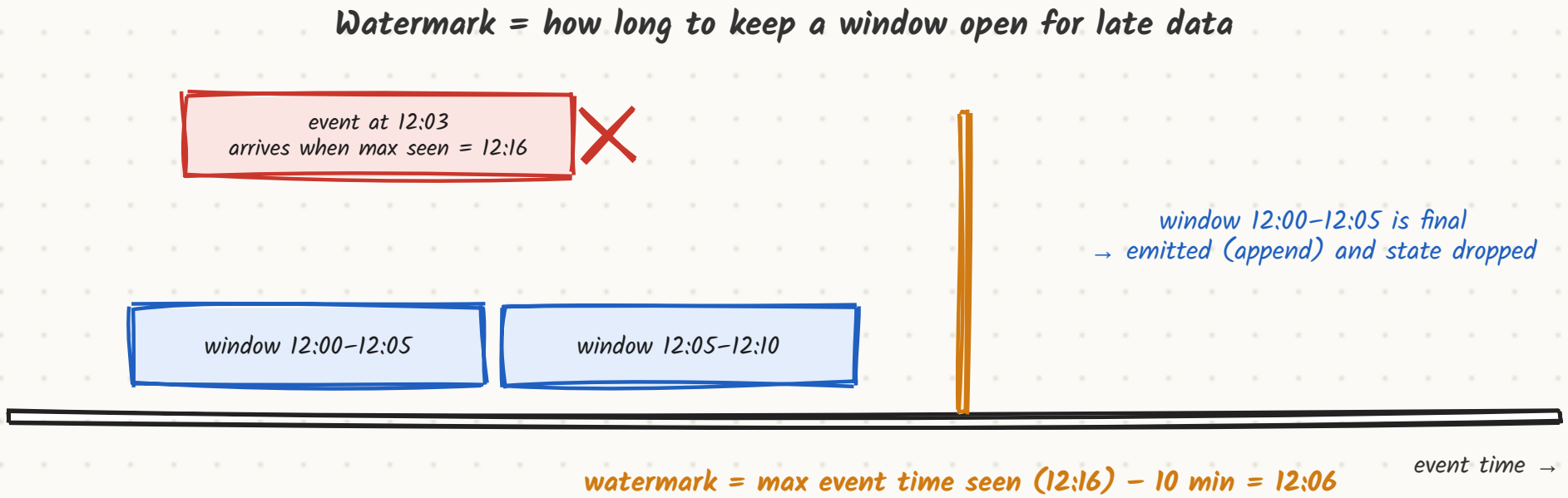


Use EVENT time for business logic. Expect events to arrive late and out of order.

Windows: tumbling, sliding, session



Watermarks: how long to wait for late data



Streaming aggregations in practice

```
// orders per 5-minute window, tolerating events up to 10 minutes late
val perWindow = orders
  .withWatermark("eventTime", "10 minutes")
  .groupBy(window(col("eventTime"), "5 minutes"), col("status"))
  .agg(count("*").as("orders"), sum("amount").as("revenue"))

perWindow.writeStream
  .outputMode("append")      // each window is written ONCE, after the watermark passes it
  .format("parquet")
  .option("path", "/data/orders_per_5m")
  .option("checkpointLocation", "/chk/orders_per_5m")
  .trigger(Trigger.ProcessingTime("30 seconds"))
  .start()

// sliding:  window(col("eventTime"), "10 minutes", "5 minutes")
// session:  session_window(col("eventTime"), "15 minutes")   (Spark 3.2+, append mode)
```

Deduplication

```scala
// Spark 3.5+: dedupe on an id; state is kept only while it can still matter
val unique = events
  .withWatermark("eventTime", "1 hour")
  .dropDuplicatesWithinWatermark("eventId")

// Spark 3.0–3.4: include the event-time column so the watermark can clean state
val uniqueOld = events
  .withWatermark("eventTime", "1 hour")
  .dropDuplicates("eventId", "eventTime")
```

At-least-once delivery means duplicates **will** arrive.

Dedupe on a stable **eventId**, and give the state a way to expire.

Joins: stream-static and stream-stream

```
// stream-static: enrich every order with the customer's tier (no state kept)
val tiers   = spark.read.parquet("/ref/customer_tier")       // re-read each micro-batch*
val enriched = orders.join(broadcast(tiers), Seq("customerId"), "left")

// stream-stream: match each order with its payment within 15 minutes (state kept)
val o = orders.withWatermark("eventTime", "20 minutes").as("o")
val p = payments.withWatermark("payTime", "20 minutes").as("p")
val paid = o.join(p, expr("""
    o.orderId = p.orderId AND
    p.payTime BETWEEN o.eventTime AND o.eventTime + interval 15 minutes"""),
  "leftOuter")                  // unpaid orders appear (with nulls) once the watermark passes
```

Your own state: mapGroupsWithState / flatMapGroupsWithState

```scala
case class Click(userId: String, page: String, ts: java.sql.Timestamp)
case class Visit(userId: String, pages: Int, lastPage: String)

// running state per user, forgotten after 30 minutes without clicks
def track(user: String, clicks: Iterator[Click], state: GroupState[Visit]): Visit = {
  if (state.hasTimedOut) {                       // called with NO rows when the timeout fires
    val last = state.get; state.remove(); last
  } else {
    val seen = clicks.toSeq
    val prev = state.getOption.getOrElse(Visit(user, 0, ""))
    val next = prev.copy(pages = prev.pages + seen.size, lastPage = seen.maxBy(_.ts.getTime).page)
    state.update(next)
    state.setTimeoutDuration("30 minutes")       // MUST be set again on every call
    next
  }
}

val visits = clicks.as[Click]
  .groupByKey(_.userId)
  .mapGroupsWithState(GroupStateTimeout.ProcessingTimeTimeout)(track)
// mapGroupsWithState → update mode;  flatMapGroupsWithState → append or update
```

Spark calls your function with: **key** · **new rows** for that key · **GroupState**

You read, update, remove state and set a **timeout**.

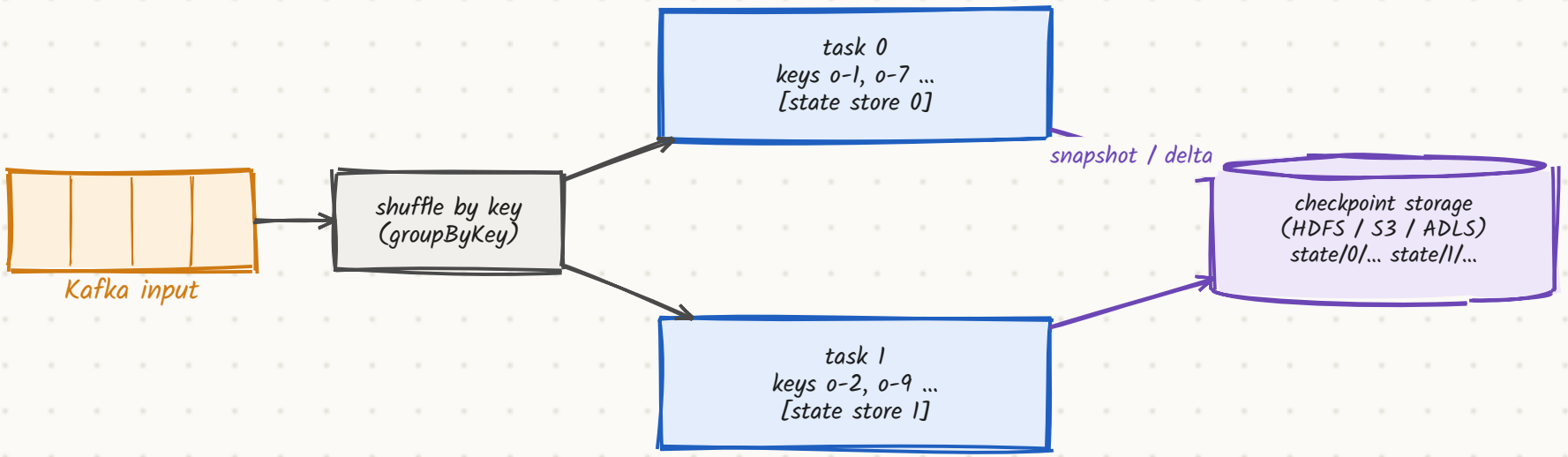
GroupState timeouts explained

- Choose the timeout type once: `NoTimeout`, `ProcessingTimeTimeout`, `EventTimeTimeout`.
- Set it **per key**: `setTimeoutDuration` or `setTimeoutTimestamp`.
- It is **reset every time** your function runs for that key → set it again on every call.
- When it fires, your function runs with **no rows** and `hasTimedOut = true`.

Processing-time timeout never fires early, but can fire late (it is checked when a batch runs).

Event-time timeout fires when the watermark passes the timestamp.

The state store: in memory, on disk, in the checkpoint



Each key's state lives with the task that owns its shuffle partition (spark.sql.shuffle.partitions).

- Default store keeps state in executor **memory** → GC pauses at scale.
- **RocksDB** store (3.2+): state on local disk, off the heap.
- **Changelog checkpointing** (3.4+) saves only changes.
- `shuffle.partitions` = state partitions: **fixed** once started.

Writing anywhere: foreach and foreachBatch

```
orders.writeStream
  .foreachBatch { (batch: DataFrame, batchId: Long) ⇒
    batch.persist()
    batch.write.format("jdbc").option("dbtable", "orders_latest").mode("append").save()
    batch.where(col("amount") > 10000).write.format("kafka")
      .option("kafka.bootstrap.servers", brokers).option("topic", "big.orders").save()
    batch.unpersist()
    // batchId is stable across retries → store it to make writes idempotent
  }
  .option("checkpointLocation", "/chk/fanout")
  .start()
```

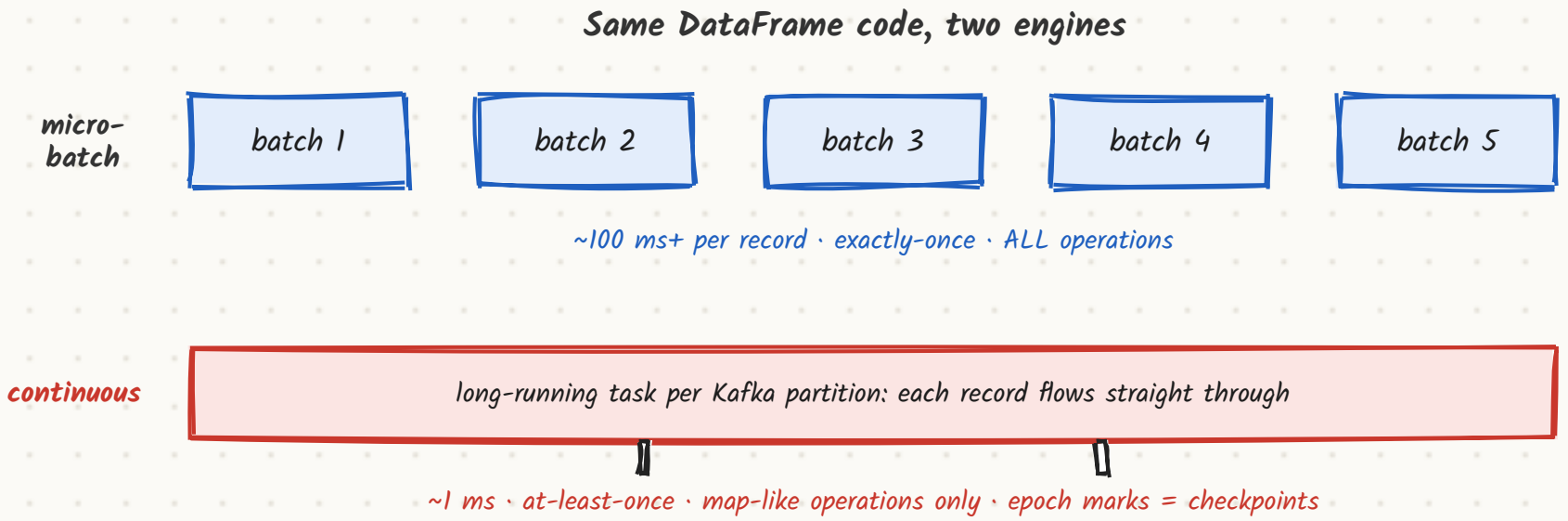
Exactly-once, end to end

End-to-end exactly-once needs all three links



Spark's Kafka sink is at-least-once → downstream consumers must deduplicate

Continuous Processing: the low-latency mode



- `Trigger.Continuous("1 second")`: checkpoint every second.
- **Map-like only**; Kafka in and out.
- **At-least-once**; cores ≥ partitions.
- **No task retry**: a failure stops the query.

Monitoring a streaming query

```
spark.streams.addListener(new StreamingQueryListener {
  override def onQueryStarted(e: QueryStartedEvent): Unit = ()
  override def onQueryTerminated(e: QueryTerminatedEvent): Unit =
    alert(s"query ${e.id} stopped: ${e.exception.getOrElse("clean stop")}")
  override def onQueryProgress(e: QueryProgressEvent): Unit = {
    val p = e.progress
    metrics.gauge("input_rows_per_s",     p.inputRowsPerSecond)
    metrics.gauge("processed_rows_per_s", p.processedRowsPerSecond)  // < input ⇒ falling behind
    metrics.gauge("batch_ms",             p.durationMs.get("triggerExecution"))
    metrics.gauge("state_rows",           p.stateOperators.map(_.numRowsTotal).sum)
    metrics.gauge("watermark",            p.eventTime.get("watermark"))
  }
})
```

Signal	Means
processed/s < input/s	falling behind; lag grows
batch duration > trigger	batches overrun; latency grows
state rows keep rising	state never expires
watermark stuck	an input stopped or event times are bad
Kafka consumer lag	how far behind the newest data

Tuning knobs that matter

Knob	What it does	Starting point
maxOffsetsPerTrigger	caps records per micro-batch	set it; protects you after downtime
minPartitions	splits big Kafka partitions into more Spark tasks	only if partitions are few and fat
spark.sql.shuffle.partitions	number of state partitions	≈ 2–3 × cores; fixed after first start
Trigger interval	latency vs. batch overhead	start with default; widen if batches are tiny
RocksDB state store	keeps large state off the JVM heap	use for any agent / card state
Changelog checkpointing	smaller, faster state commits	enable on 3.4+
Executor cores	parallel tasks	≥ Kafka partitions (and ≥ partitions for continuous)

Changing a running query safely

Change between restarts (same checkpoint)	Allowed?
Add / remove a filter, change a projection	yes
Change maxOffsetsPerTrigger	yes
Change the output topic of a Kafka sink	yes
Change the code inside a ForeachWriter / state function	yes (you own the semantics)
Subscribe to a different topic	no
Add or remove a source	no
Change grouping keys / aggregates / dedupe columns	no
Change the state class or the timeout type of (flat)mapGroupsWithState	no
Change spark.sql.shuffle.partitions or the state store provider	no

3

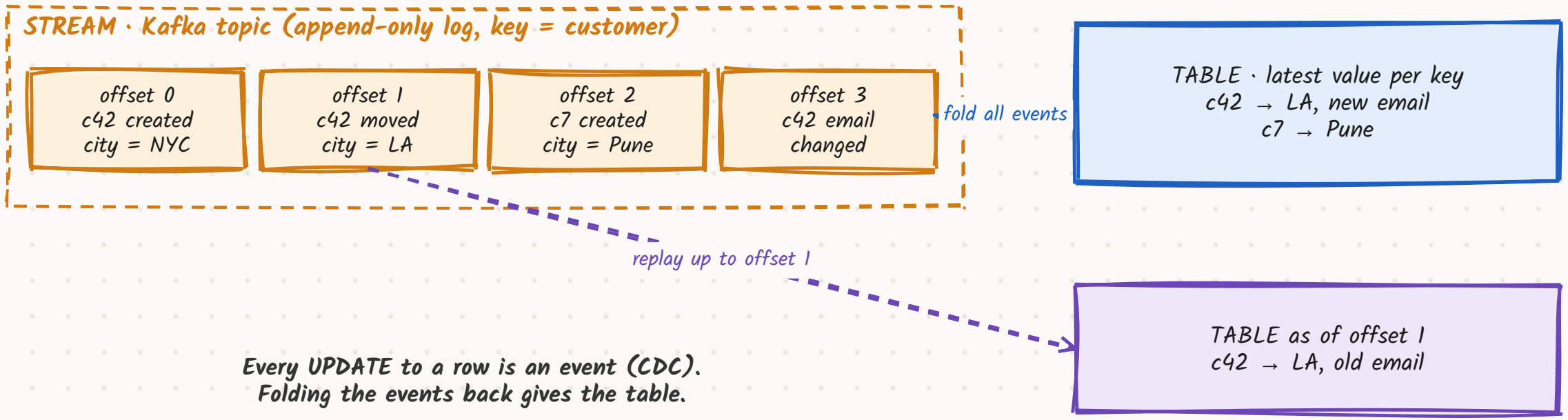
Modelling & serving real-time data

How to shape data that never stops arriving, and where to keep it so people and agents can query it in milliseconds.

Batch thinking vs. stream thinking

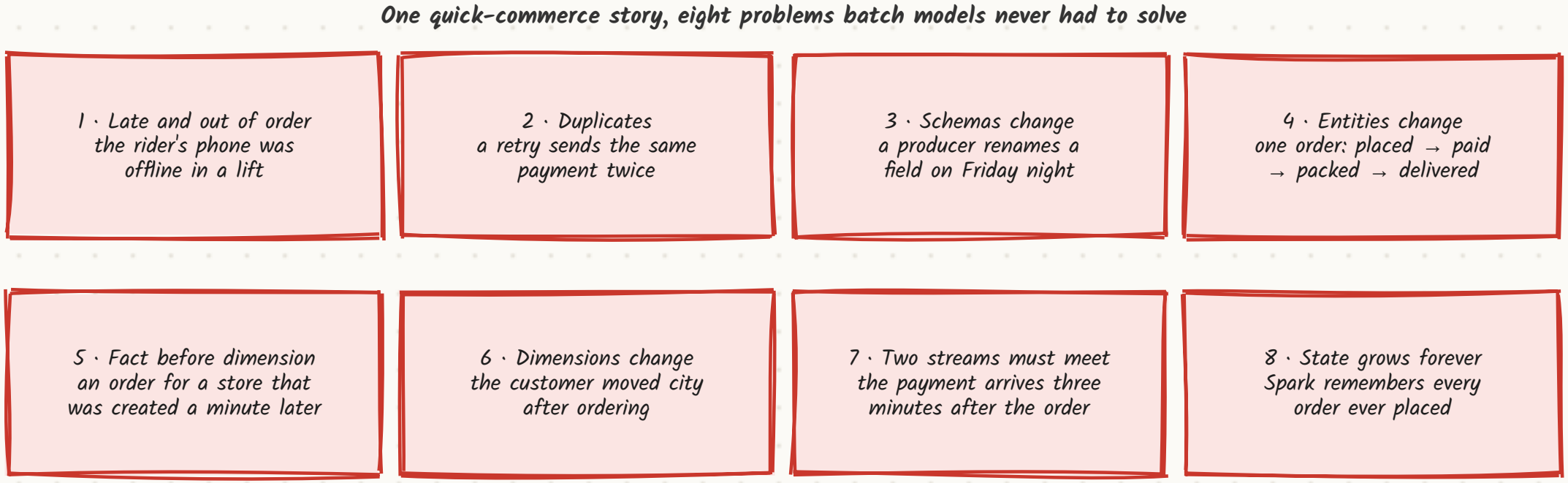
	Batch world	Streaming world
Data	complete before you start	unbounded, never finished
Order	sort it whenever you like	only per key/partition; late and out of order
Correctness	true at load time	true “as of now”, may be revised
Dimensions	frozen per load	changing while facts arrive
Aggregates	computed once per run	maintained continuously (state + watermarks)
Recovery	rerun the job	replay the log from an offset
Main question	what is the answer?	when is the answer good enough to publish?

Streams and tables are two views of the same data

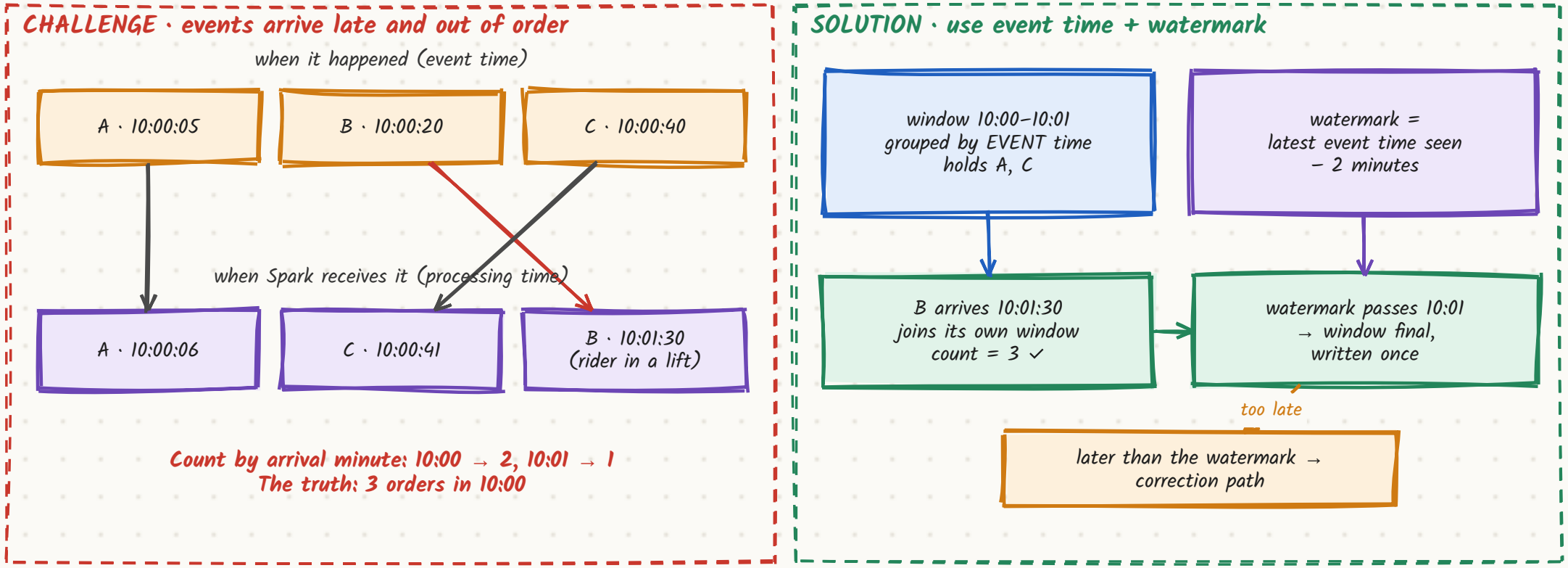


Every UPDATE to a row is an event (CDC). Folding the events back gives the table.

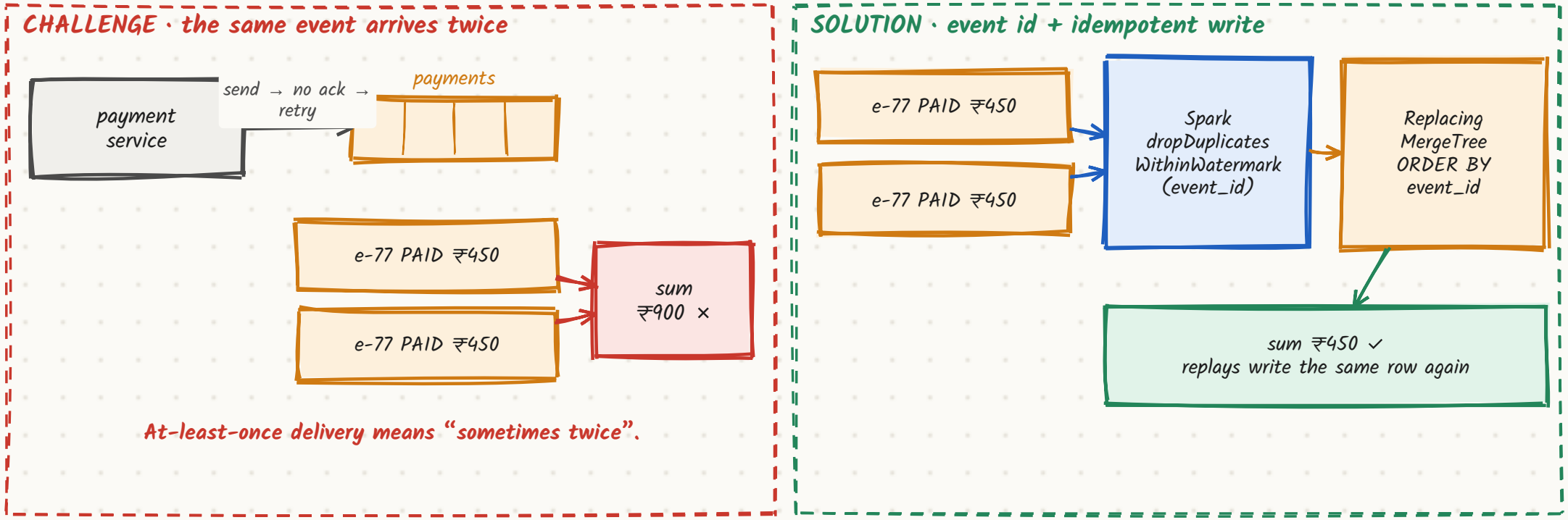
Eight modelling challenges, one story



Challenge 1 · Late and out-of-order events



Challenge 2 · The same event arrives twice



Why classic models need adapting

- **Kimball star schema**: facts arrive before or after the dimension row they need; SCD Type 2 needs an "as of event time" lookup.
- **Foreign keys** cannot be enforced; an order can arrive before its customer exists.
- **Data Vault** fits better: hubs = business keys, satellites = change records, inserts only.

> The models still apply. What changes is **when** they are computed and **which version** of a dimension a fact sees.

Designing the event itself

- **Immutable**, past tense, one business fact.
- **Self-describing**: id, type, version, event time, key.
- Name: `{domain}.{entity}.{action}` → `orders.order.paid`.
- Granularity: one event per **state transition**; split only when consumers need it.

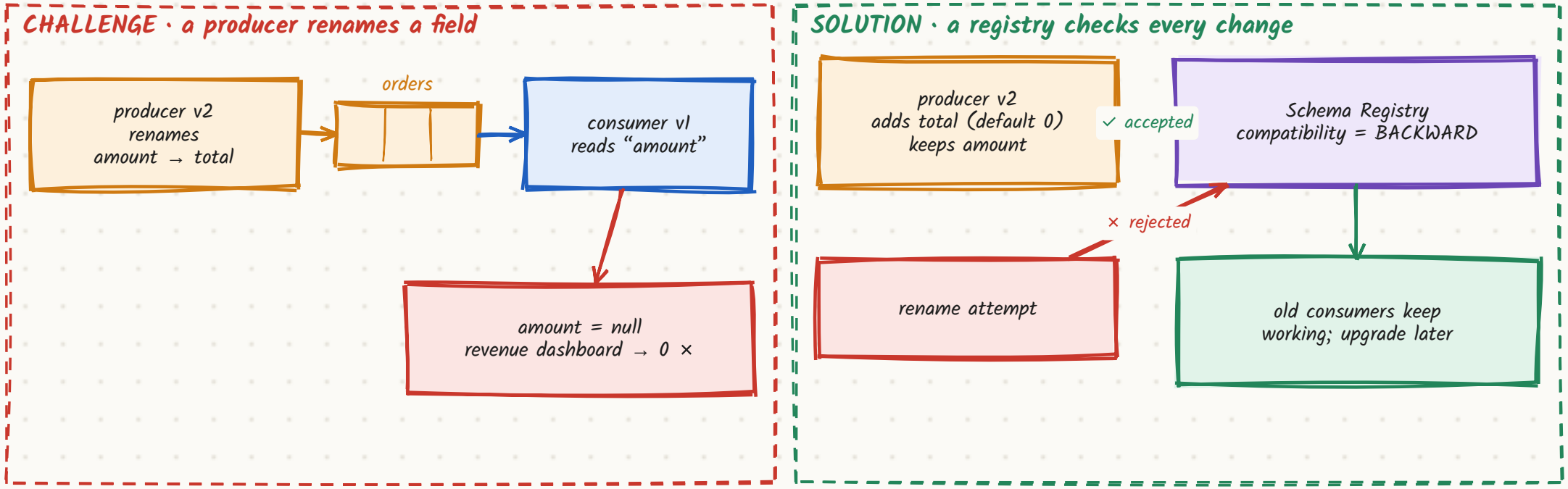
Payload style	Carries	Consumer must
Notification	ids only	call back for details
State transfer	full new state	nothing; can rebuild
Delta / CDC	before + after	apply changes in order

Schemas are contracts

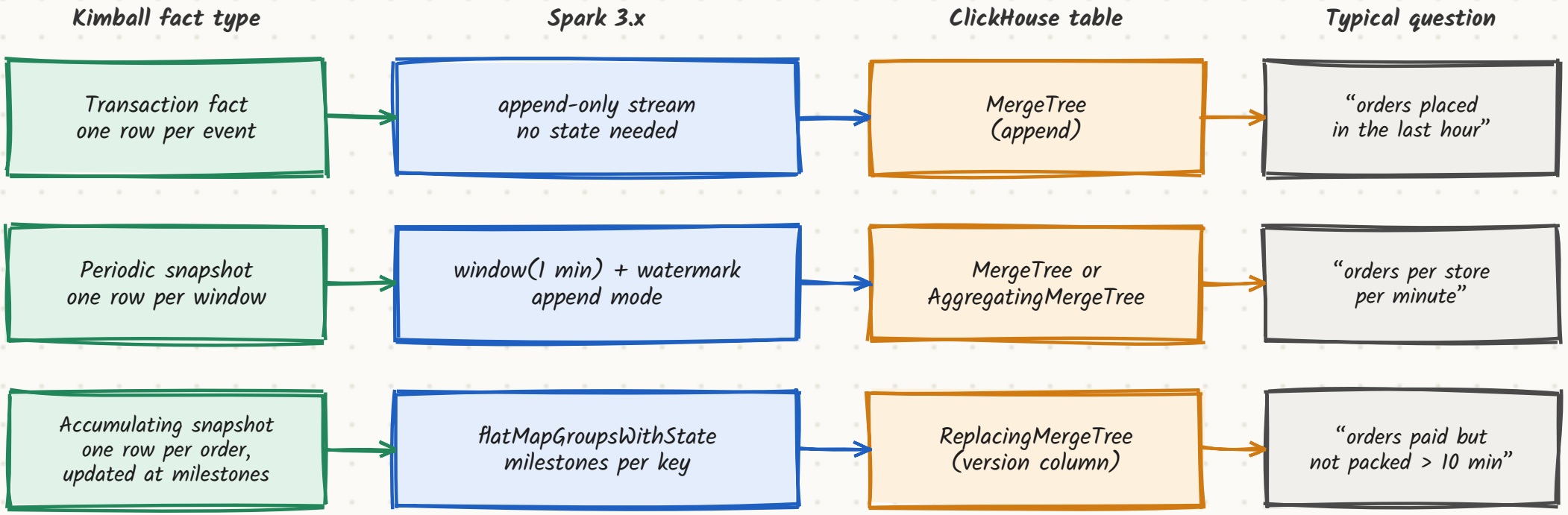
Mode	Guarantee	Upgrade first
BACKWARD (default)	new schema reads old data	consumers
FORWARD	old schema reads new data	producers
FULL	both directions	either
***_TRANSITIVE**	against all versions, not just the last	same

- Add fields **with defaults**; never rename or change a type in place.
- Protobuf: never reuse a field number.
- Wire format: 1 magic byte + 4-byte schema id, then the payload.
- Breaking change → new topic or new major version.

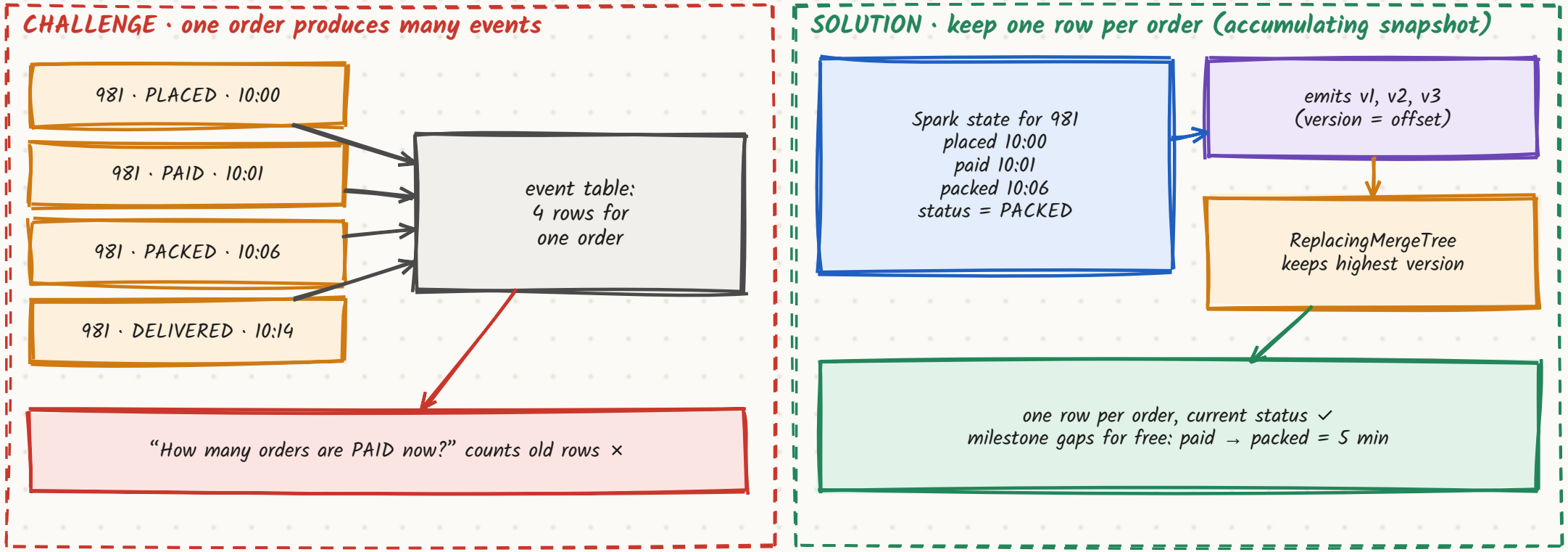
Challenge 3 · A schema change breaks consumers



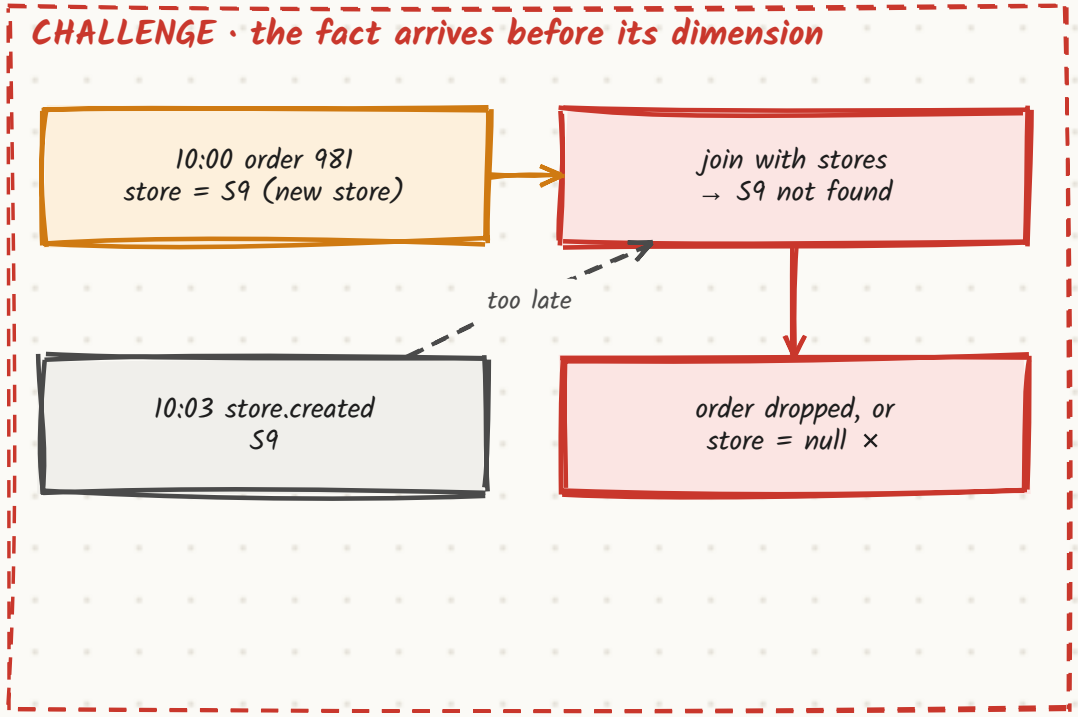
Facts in a stream: Kimball's three fact types

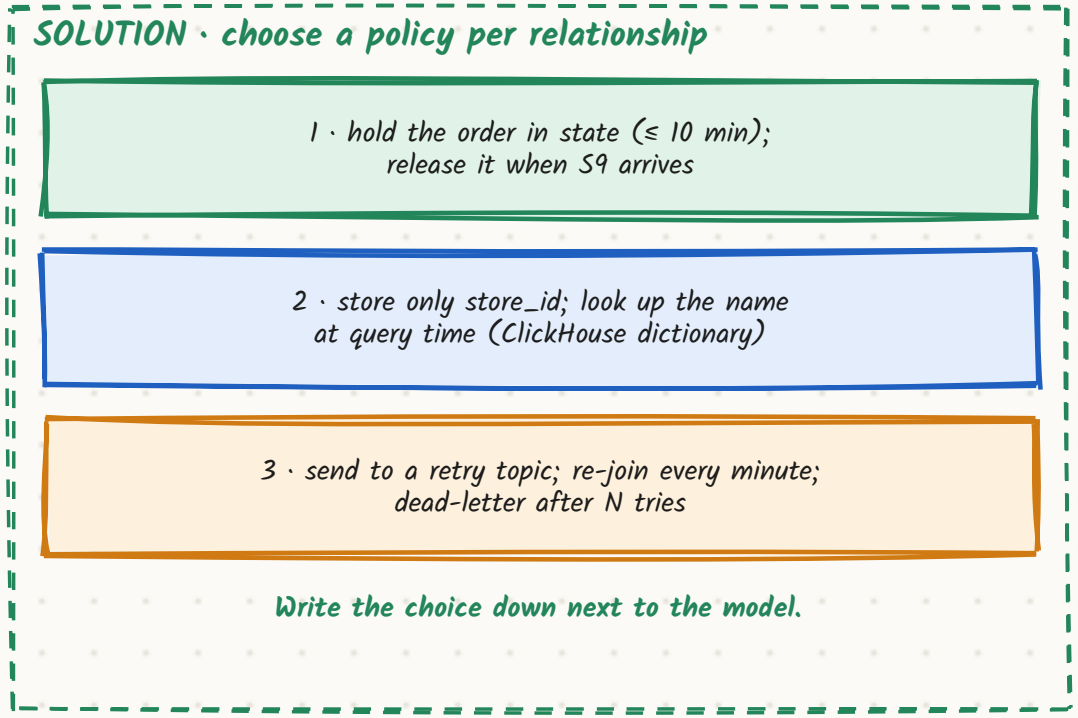


Challenge 4 · One entity, many events

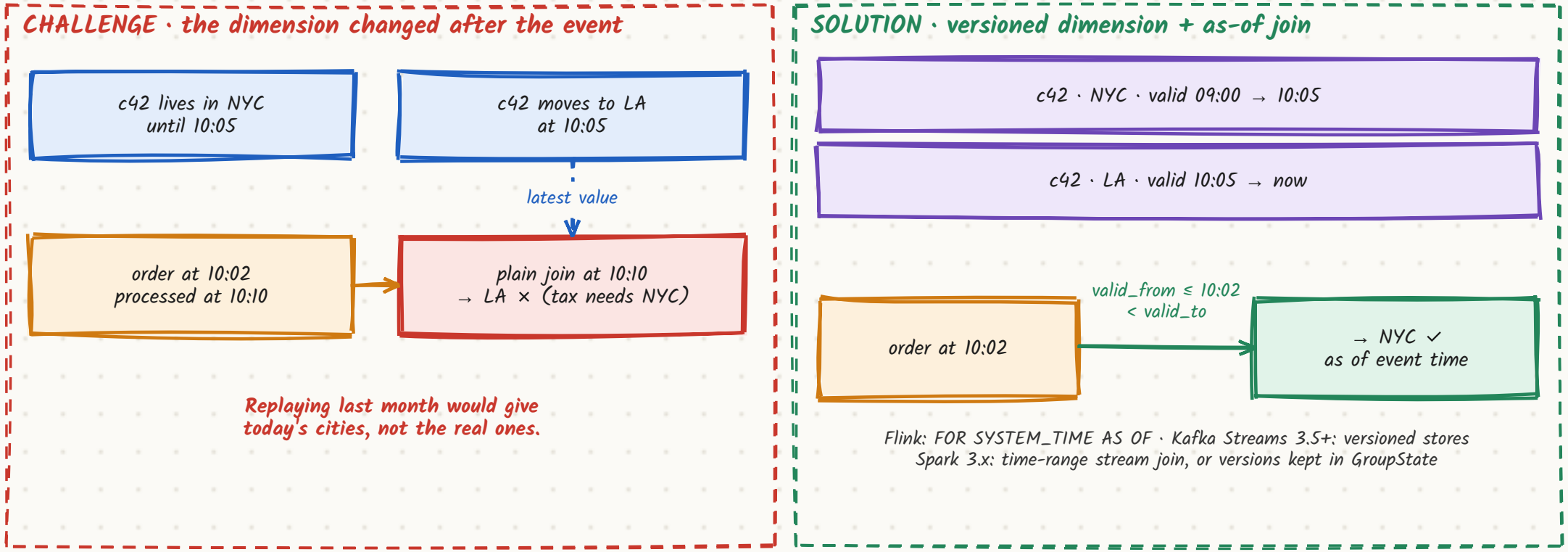


Challenge 5 · The fact arrives before its dimension





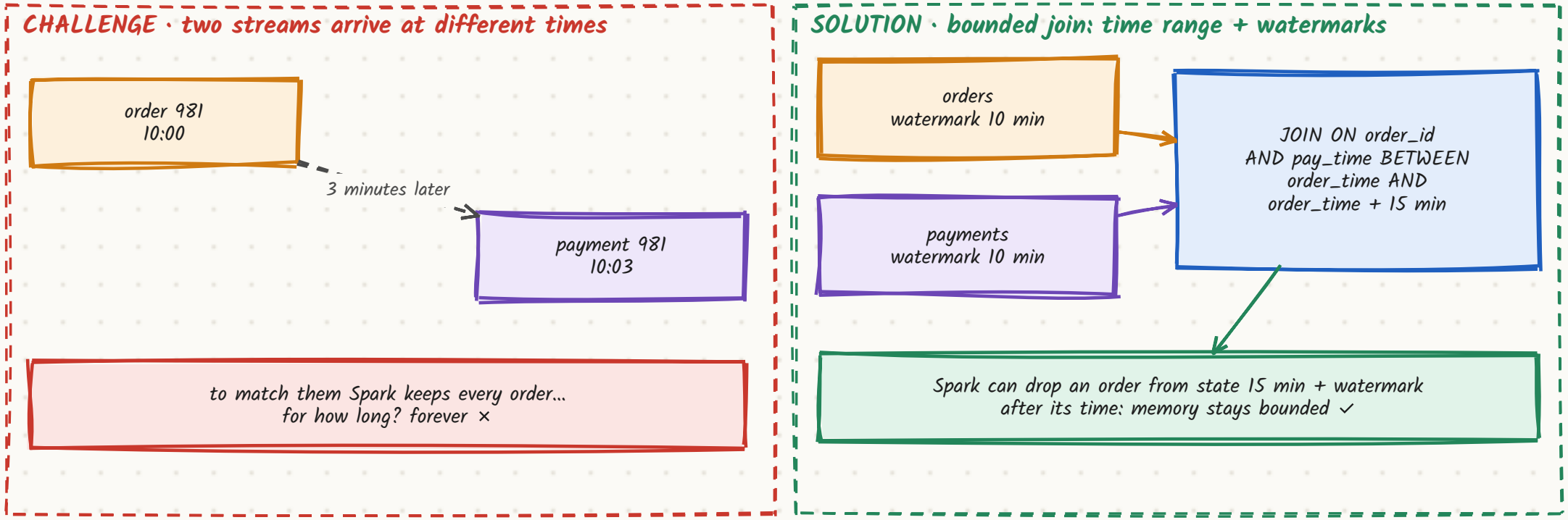
Challenge 6 · The dimension changes after the event



Dimensions that change while facts arrive

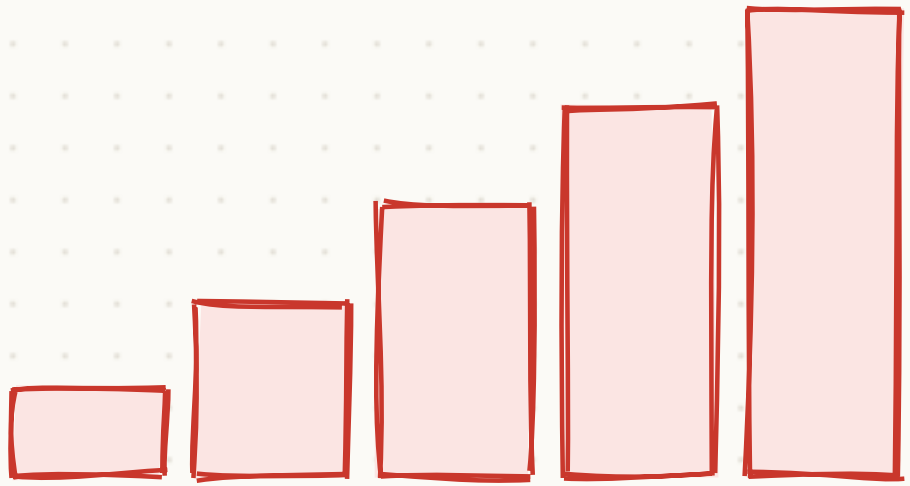
Approach	What the fact sees	Spark 3.x	Elsewhere
Latest value (SCD 1)	dimension as it is now	stream-static join (static side re-read each batch)	KTable join; ClickHouse dictionary
As of event time (SCD 2)	version valid at event time	no built-in; stream–stream join with time range, or keep versions in GroupState	Flink FOR SYSTEM_TIME AS OF; Kafka Streams versioned stores (3.5+)
Denormalise at source	value copied into the event	nothing to join	event-carried state
Join at query time	whatever the query asks	write facts with keys only	ClickHouse dictGet / JOIN

Challenge 7 · Two streams must meet



Challenge 8 · State that grows forever

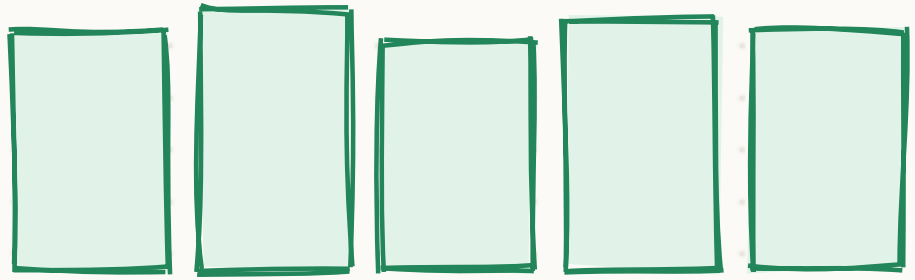
CHALLENGE · state keeps every key forever



day 1 → day 5: state size = every order ever ×
slower batches, bigger checkpoints, then out of memory

SOLUTION · expire state when the entity is done

GroupState timeout: remove 2 h after the last event
watermark: drop windows once closed



state size ≈ orders active right now ✓

Updates, deletes and late data in the sink

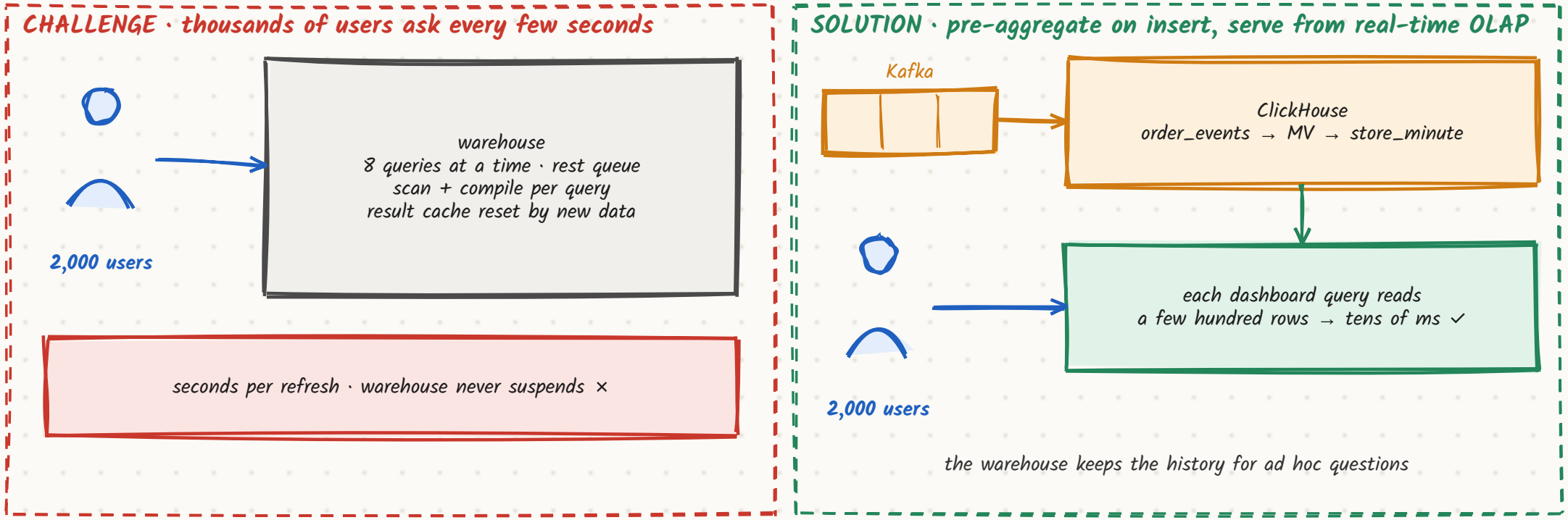
Situation	Model it as	Sink behaviour
Facts never change	append-only	plain insert; dedupe on event id
Entity changes over time	versioned upsert	keep highest version per key
Entity deleted	tombstone / is_deleted flag	hide or purge on merge
Late event changes an old window	correction row or re-emitted aggregate	sum corrections, or replace by key
Spark retries a batch	same rows again	idempotent by key + version

Event sourcing, projections and snapshots

- The **log is the source of truth**; current state is a fold of events.
- **Projections**: many read models from one log, each shaped for a query.
- **Snapshots** bound replay time (every N events or T minutes).
- Overkill for simple CRUD; essential for audit and "as of" questions.

A context card (Part 5) is a projection. A ClickHouse accumulating snapshot is another projection of the same events.

Serving: thousands of users, every few seconds



Why warehouses struggle with real-time serving

Requirement	Cloud warehouse (Snowflake, BigQuery)	Real-time OLAP (ClickHouse, Pinot, Druid)
Freshness	seconds to minutes (Snowpipe Streaming: “as low as 5 s”)	sub-second to seconds
Query latency	typically seconds; caches reset when data changes	tens of milliseconds
Concurrency	Snowflake: 8 concurrent queries per warehouse by default	hundreds to thousands per node
Cost when always on	billed while running; auto-suspend ≥ 60 s	sized for steady load
Best at	ad hoc SQL, large joins, history	fixed query shapes on fresh data

Pick the serving store by query shape

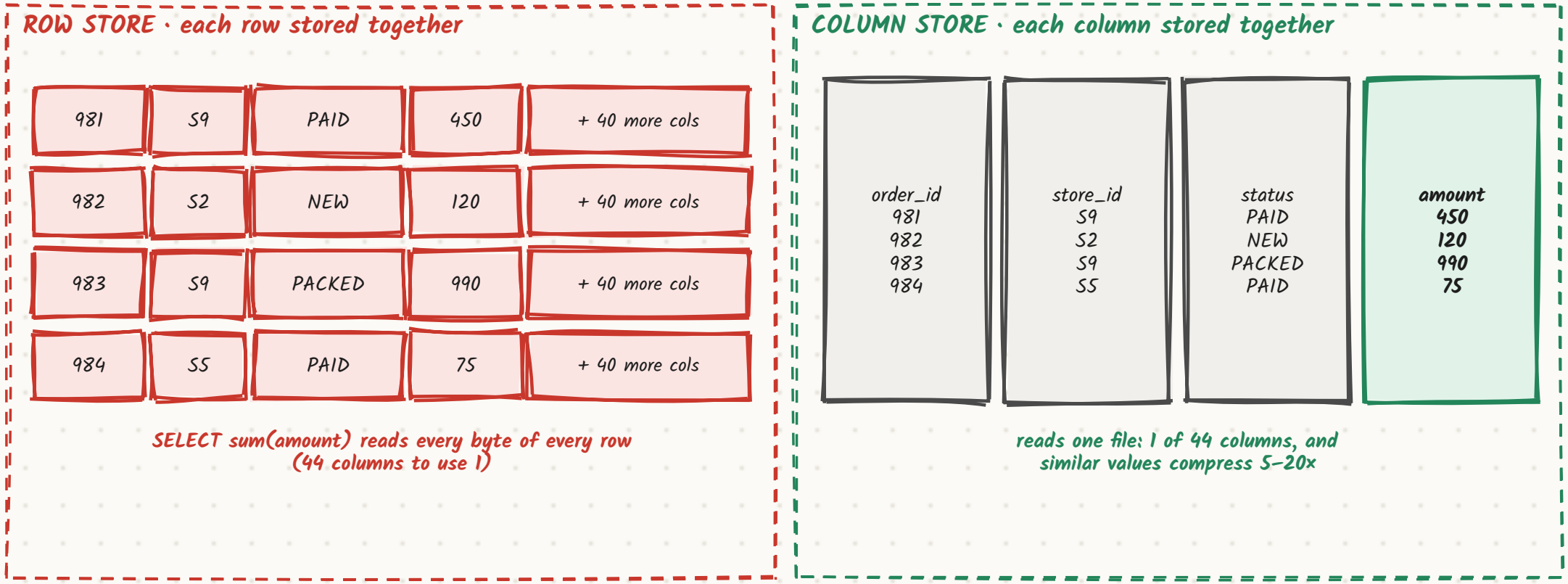
Query shape	Example	Store	Typical latency
Get one record by key	card for order 981	Redis, DynamoDB, Cassandra	~1 ms
Aggregate fresh events, fixed shapes	orders per store per minute, p90 pack time	ClickHouse, Pinot, Druid	10–100 ms
Search text or similar items	policy paragraph, similar cases	OpenSearch, vector store	10–100 ms
Ad hoc SQL over history	last year's cohort analysis	Snowflake, BigQuery, Delta/Iceberg + engine	seconds+
Time series metrics	consumer lag, CPU	Prometheus, time-series DB	~10 ms

What ClickHouse is

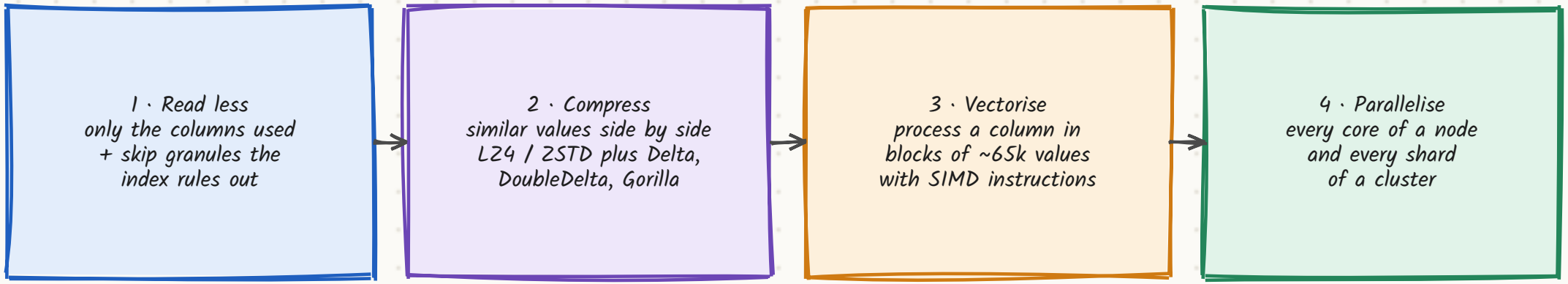


Same SQL language, very different physical design and job.

Row store vs. column store

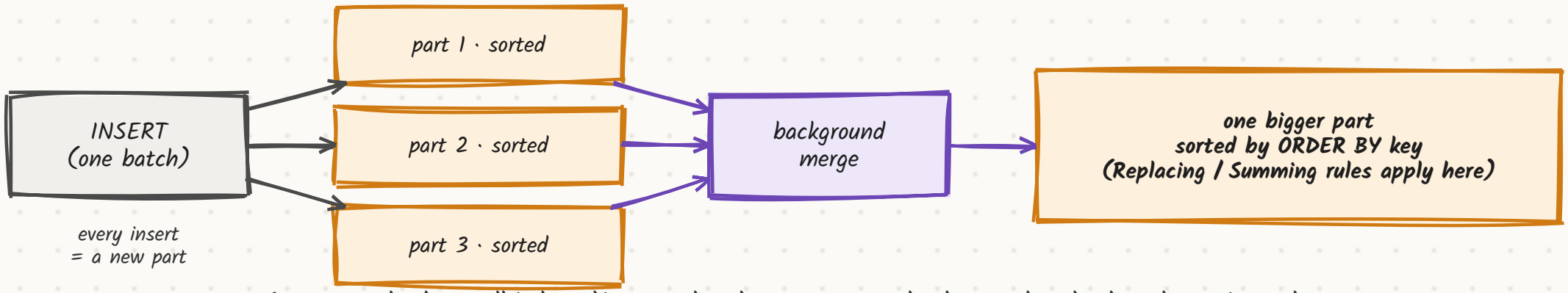


Why ClickHouse is fast: four ideas

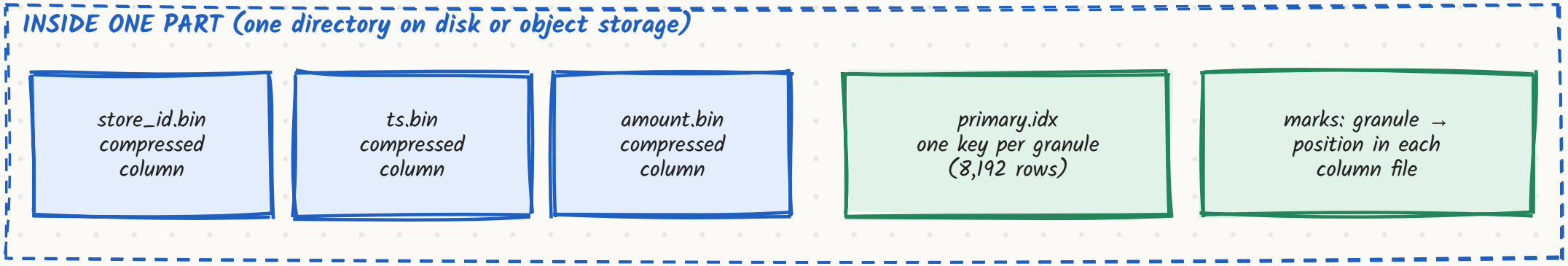


Less data from disk × less work per value × more workers = milliseconds

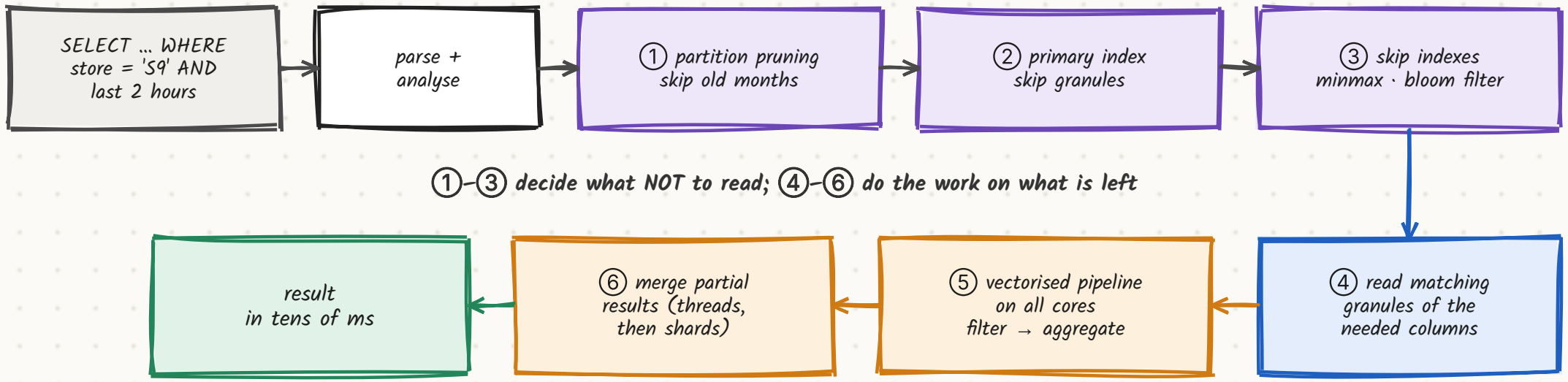
Inside a table: parts, merges and the sparse index



A query reads the small index, skips granules that cannot match, then reads only the columns it needs.



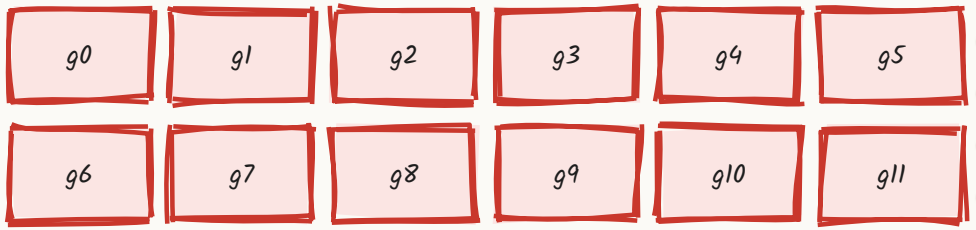
How a query runs, step by step



When the filter is not in ORDER BY

PROBLEM · filter on a column not in ORDER BY

table ORDER BY (store_id, ts) · WHERE customer_id = 'c42'



primary index cannot help:
every granule is read ×

FIX · skip index, or a projection

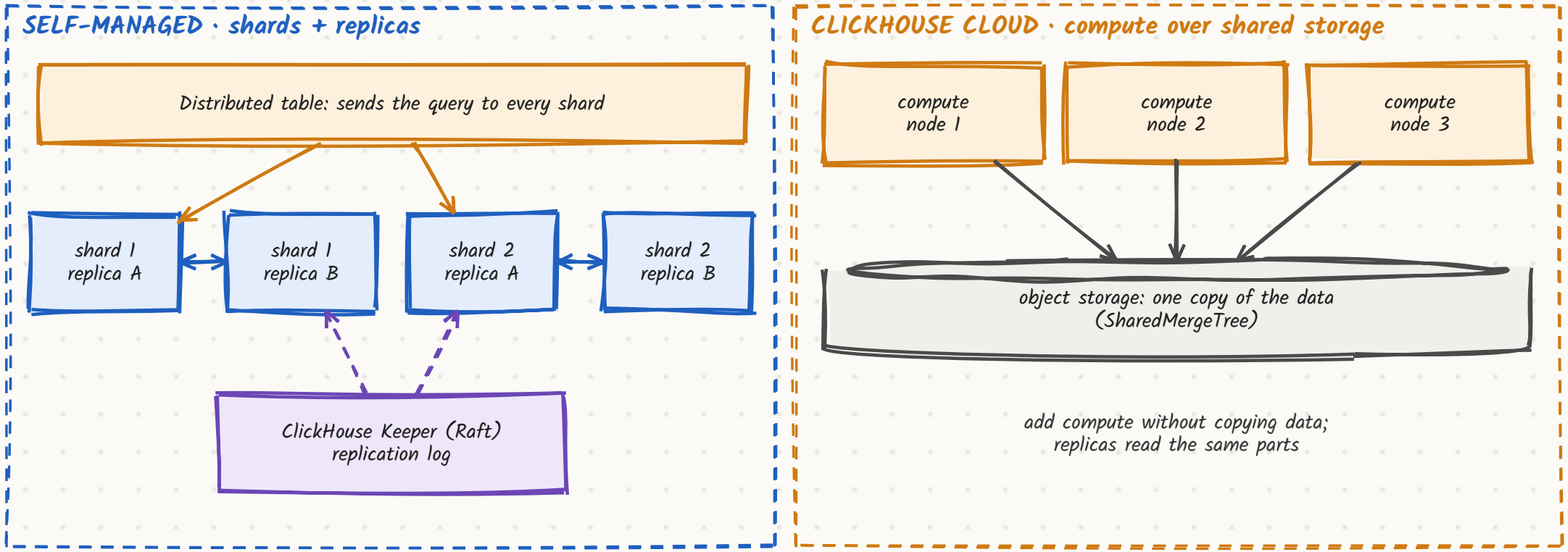


bloom_filter index: only g3 and g8 might contain c42

projection: a hidden copy of the table sorted by customer_id, kept up to date on every insert

skip index = cheap, probabilistic · projection = fast, costs storage

Scaling ClickHouse: shards, replicas and shared storage



The MergeTree family: modelling change at merge time

Engine	Merge rule	Use it for
MergeTree	none (keep all rows)	transaction facts, logs
ReplacingMergeTree(ver, is_deleted)	keep highest version per ORDER BY key	CDC upserts, accumulating snapshots, cards
SummingMergeTree	add numeric columns per key	simple counters and sums
AggregatingMergeTree	combine partial aggregate states	uniq, quantiles, averages via MVs
Replicated* variants	same, with replication	high availability

Merges are asynchronous. Until they finish, read with `FINAL` or aggregate again (`argMax`, `-Merge`).

Incremental materialized views: aggregate on insert

ClickHouse: transaction facts + per-minute rollup on insert

```
-- Transaction facts: one row per (deduplicated) event, appended by Spark
CREATE TABLE shop.order_events (
  event_id String, order_id String, store_id LowCardinality(String),
  kind LowCardinality(String), event_time DateTime64(3), amount Decimal(12,2)
) ENGINE = ReplacingMergeTree
PARTITION BY toYYYYMM(event_time)
ORDER BY (store_id, event_time, event_id);

-- Periodic snapshot maintained on insert: store x minute
CREATE TABLE shop.store_minute (
  store_id LowCardinality(String), minute DateTime,
  orders   AggregateFunction(uniq, String),          -- immune to duplicates
  revenue  AggregateFunction(sum, Decimal(12,2))
) ENGINE = AggregatingMergeTree
ORDER BY (store_id, minute);

CREATE MATERIALIZED VIEW shop.store_minute_mv TO shop.store_minute AS
SELECT store_id, toStartOfMinute(event_time) AS minute,
       uniqState(order_id) AS orders, sumState(amount) AS revenue
FROM shop.order_events            -- fires on every insert into order_events
WHERE kind = 'PLACED'
GROUP BY store_id, minute;
```

Getting Kafka data into ClickHouse

Path	How it works	When to use it
Kafka table engine + MV	ClickHouse consumes the topic; a materialized view moves rows into a MergeTree	simple, no extra service
ClickPipes (ClickHouse Cloud)	managed ingestion from Kafka-compatible sources	on ClickHouse Cloud
Kafka Connect sink	connector writes batches to ClickHouse	Connect already in use
Spark foreachBatch	stream processor writes one large insert per micro-batch	after stateful logic (dedupe, snapshots)

Insert **large, infrequent** batches: thousands to hundreds of thousands of rows, about once a second or less. Many tiny inserts mean many tiny parts and the "too many parts" error.

Modelling a ClickHouse table for streaming data

- **ORDER BY**: columns you filter on, low to high cardinality (store, then order id).
- **PARTITION BY** month or day, never by id; partitions are for data lifecycle.
- **One wide table** beats many joins; denormalise stable attributes.
- Dimensions as **dictionaries** (`dictGet`), refreshed on a schedule.
- **LowCardinality(String)** for repeated strings; **TTL** for retention.

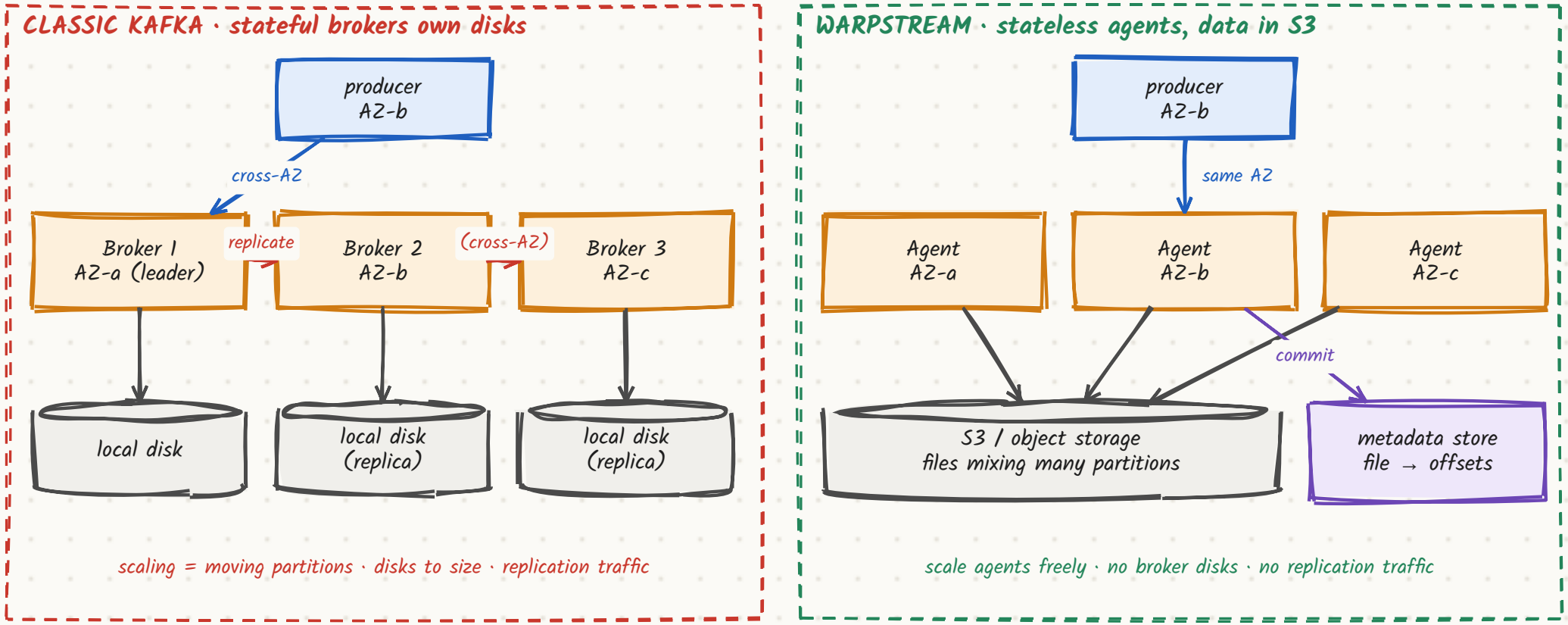
orders_fact: the accumulating snapshot

```
-- Accumulating snapshot: one row per order, latest version wins
CREATE TABLE shop.orders_fact (
  order_id    String,
  store_id    LowCardinality(String),
  status      LowCardinality(String),
  placed_at   DateTime64(3),
  paid_at     Nullable(DateTime64(3)),
  packed_at   Nullable(DateTime64(3)),
  delivered_at Nullable(DateTime64(3)),
  amount      Decimal(12,2),
  version     UInt64,               -- Kafka offset or source updated_at
  is_deleted  UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(version, is_deleted)
PARTITION BY toYYYYMM(placed_at)
ORDER BY (store_id, order_id);
```

When not to use ClickHouse

Need	Why ClickHouse struggles	Use instead
Many single-row updates and transactions	column files and immutable parts make point updates expensive	Postgres / MySQL
One record by key at very high QPS	works, but a key-value store is simpler and faster	Redis, DynamoDB
Thousands of tiny inserts per second	each insert is a part; merges fall behind	batch upstream or use async_insert
Ad hoc joins of several huge tables	joins are improving but not its strength	warehouse / lakehouse
Strict uniqueness at read time	deduplication happens at merge time	FINAL / argMax, or an OLTP store

Kafka's storage problem, and what WarpStream changes

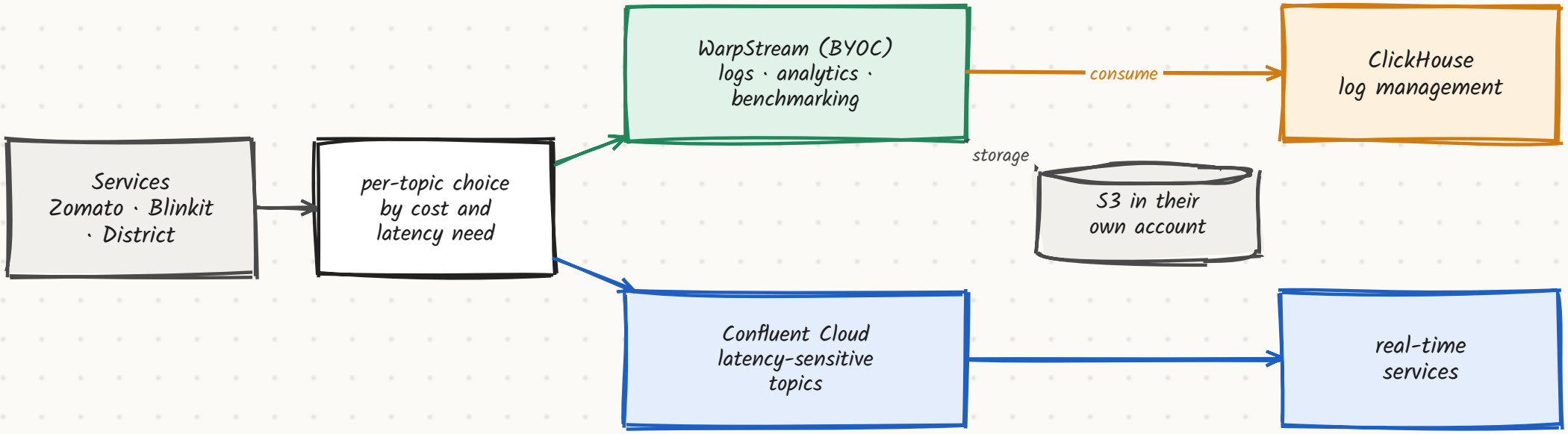


WarpStream trade-offs: latency for cost and simplicity

Configuration	Produce p99	End-to-end p99
Default (S3 Standard, 250 ms batches)	≈ 500 ms	≈ 900 ms
Low-latency (S3 Express One Zone, 25 ms batches)	< 50 ms	< 400 ms
Classic Kafka on local disks (typical)	single-digit to tens of ms	tens of ms

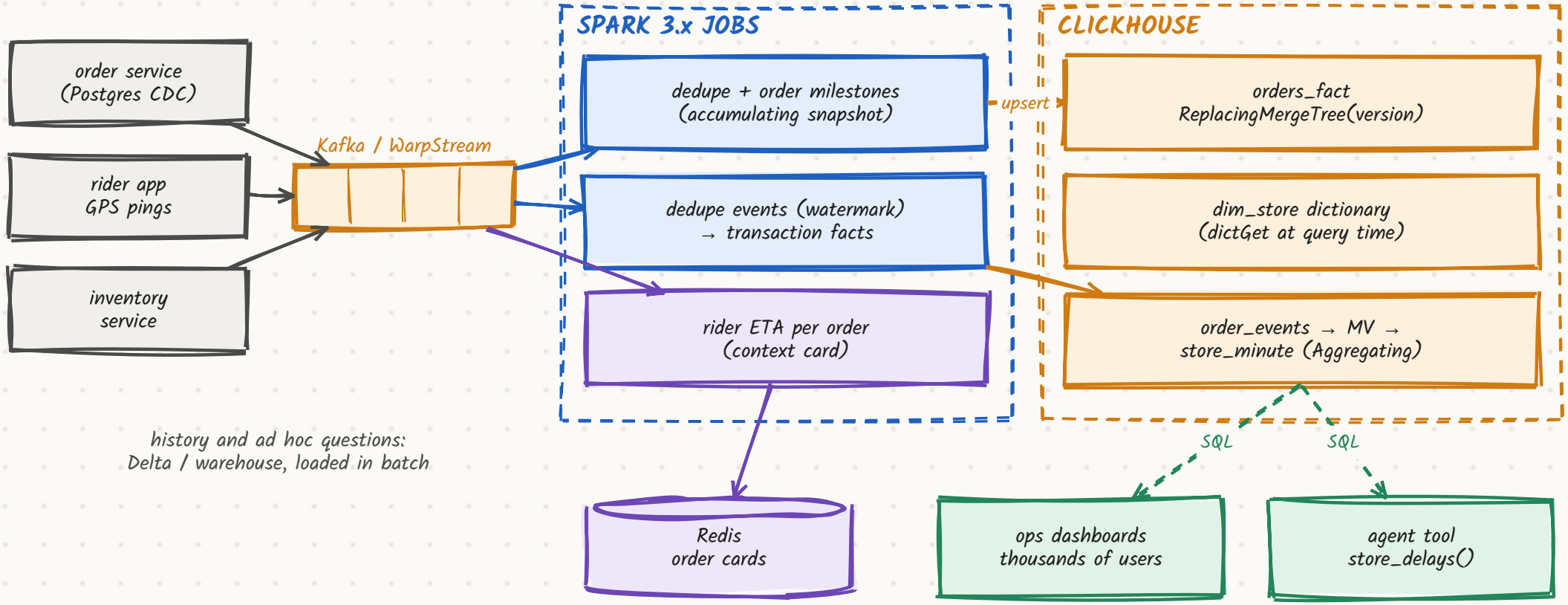
Upstream Kafka is heading the same way: **KIP-1150 (diskless topics)** was accepted in March 2026; implementation work continues in follow-up KIPs.

Case study: Eternal (Zomato, Blinkit, District)



Logs: 2.3 to 25+ GiB/s across clusters · about 60% cheaper than their single-AZ self-run Kafka (WarpStream case study)

Worked example: quick-commerce operations, end to end



Spark side: the accumulating snapshot in code

Order milestones with flatMapGroupsWithState (Spark 3.x)

```
def fold(row: OrderRow, e: OrderEvt): OrderRow =
  if (e.offset ≤ row.last_offset) row                    // duplicate or replay: ignore
  else {
    val r = row.copy(status = e.kind, last_offset = e.offset)
    e.kind match {
      case "PLACED"    ⇒ r.copy(placed_at = e.eventTime, amount = e.amount)
      case "PAID"      ⇒ r.copy(paid_at = e.eventTime)
      case "PACKED"    ⇒ r.copy(packed_at = e.eventTime)
      case "DELIVERED" ⇒ r.copy(delivered_at = e.eventTime)
      case _           ⇒ row
    }
  }

val milestones = events                                  // Dataset[OrderEvt]
  .groupByKey(_.orderId)
  .flatMapGroupsWithState(OutputMode.Update, GroupStateTimeout.ProcessingTimeTimeout) {
    (id: String, evts: Iterator[OrderEvt], st: GroupState[OrderRow]) ⇒
      if (st.hasTimedOut) { st.remove(); Iterator.empty }  // order finished long ago
      else {
        val sorted = evts.toSeq.sortBy(_.offset)
        val start  = st.getOption.getOrElse(
          OrderRow(id, sorted.head.storeId, "NEW", null, null, null, null, BigDecimal(0), -1L))
        val next   = sorted.foldLeft(start)(fold)
        st.update(next); st.setTimeoutDuration("2 hours")
        Iterator(next)                                   // one updated row per order
      }
  }
```

The streaming modelling playbook

If you need...	Model	Spark 3.x	Serve from
simple enrichment	stream–static join	join with Delta/static DF	wherever the facts go
live operational state per entity	accumulating snapshot / projection	flatMapGroupsWithState	Redis (lookups), ClickHouse (filters)
live metrics and dashboards	transaction + periodic snapshot facts	append stream + windows	ClickHouse / Pinot / Druid
historically correct dimensions	versioned dimension + temporal join	stream–stream join with time range	warehouse / lakehouse
audit and "as of" questions	event sourcing	replay from Kafka / lake	lakehouse + projections
many sources, full lineage	Data Vault (insert-only)	hubs / sats with hash keys	warehouse / lakehouse

Checklist: every event has an id, key, version and event time · schemas registered with a compatibility mode · every sink idempotent by key + version · late-data policy written down · state has a key and an expiry.

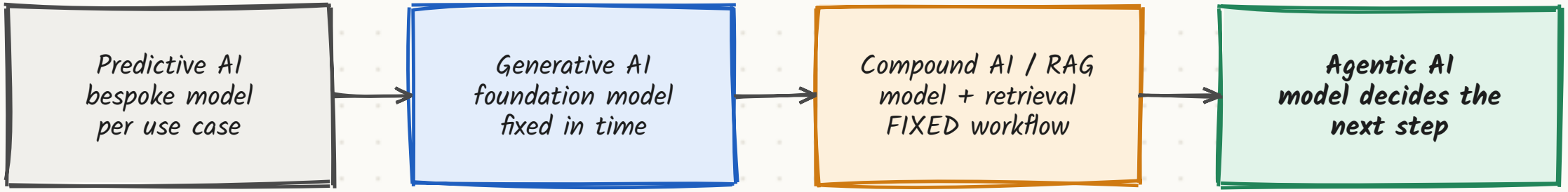
4

LLMs and agents from zero

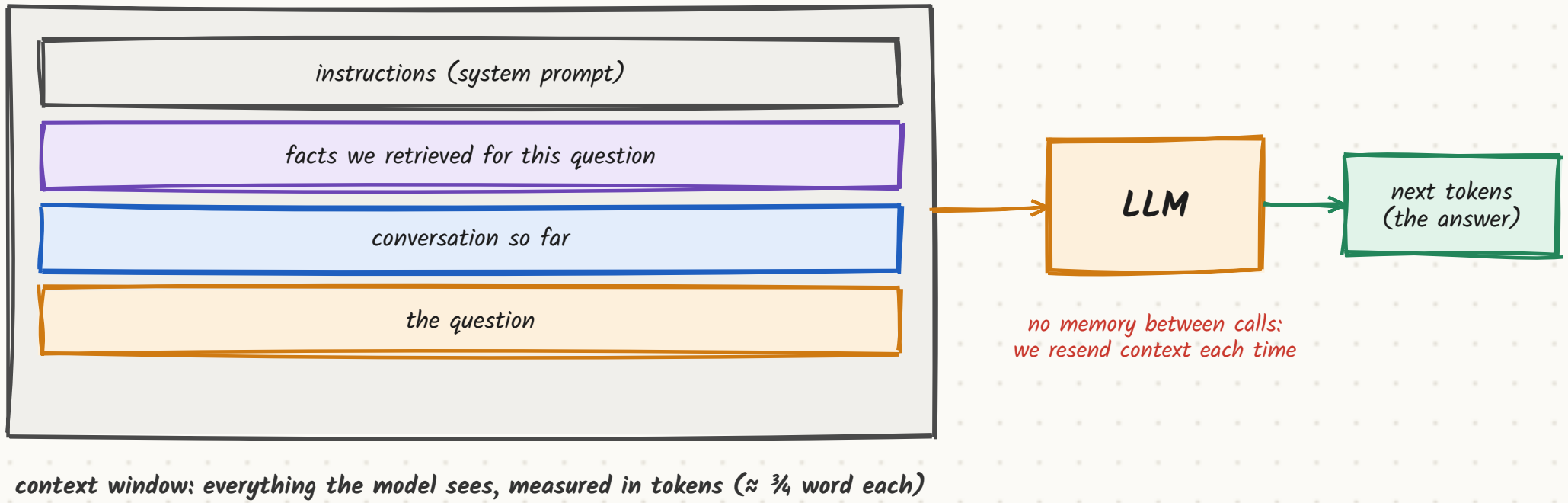
Tokens, context windows, embeddings, retrieval, tools and the agent loop, without the hype.

Three waves of AI

Each wave fixes the last wave's limit



LLMs in five minutes



context window: everything the model sees, measured in tokens (≈ ¾ word each)

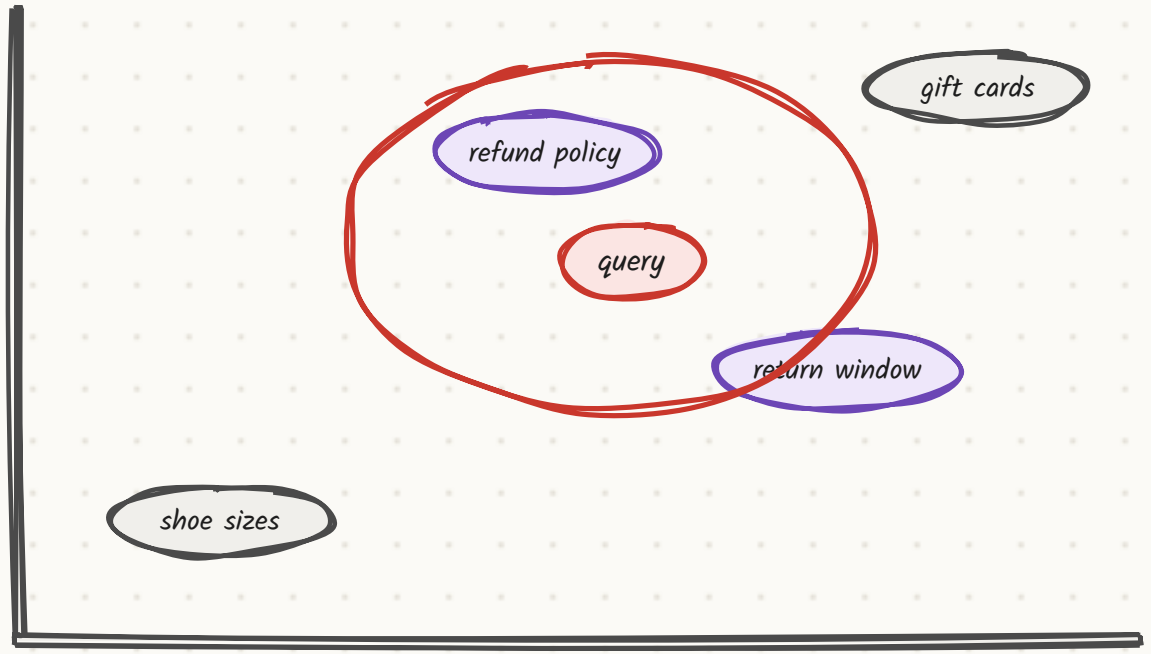
Prompts: programs made of data

- **System prompt**: role, rules, tone, output format.
- **Context**: facts we retrieved for this request.
- **History**: the relevant previous turns.
- **Question**: the user's message or the triggering event.
- **Tool definitions**: what the model may ask us to do.

Most of a prompt is **data we assemble**, not text we write once.

Fresh, correct, small context → good answers.

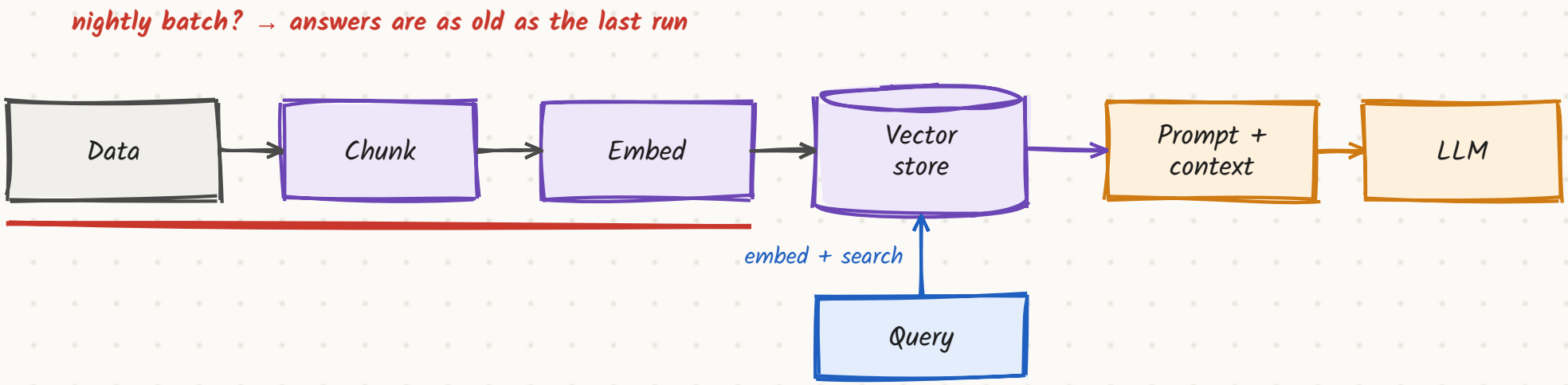
Embeddings and vector search



"Can I send it back?"
→ [0.12, -0.83, 0.40 …]

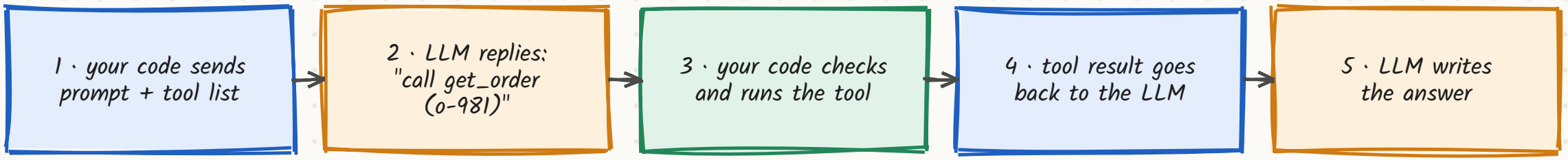
similar meaning ⇒ nearby vectors;
search = find nearest neighbours

Retrieval-augmented generation (RAG)



- **Augment**: build the knowledge base.
- **Infer**: retrieve context, call the model.
- **Workflow**: steps and tools for complex requests.
- **Post-process**: validate output before it reaches anyone.

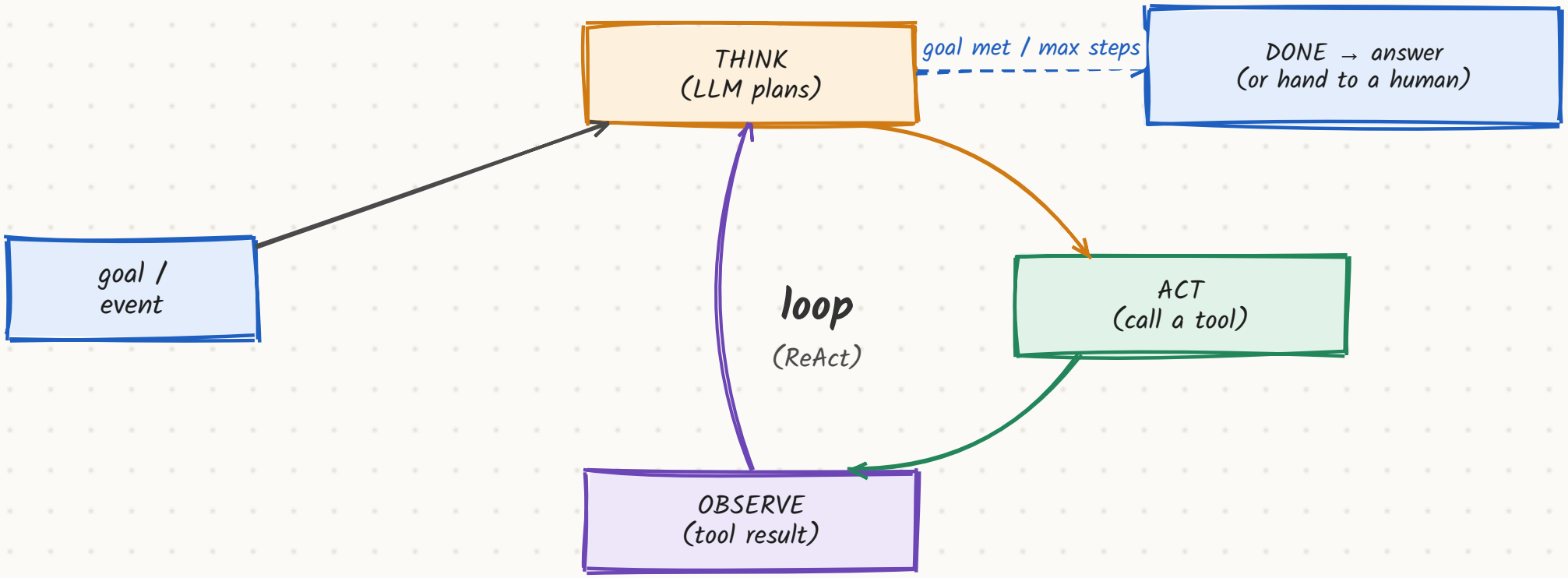
Tool calling



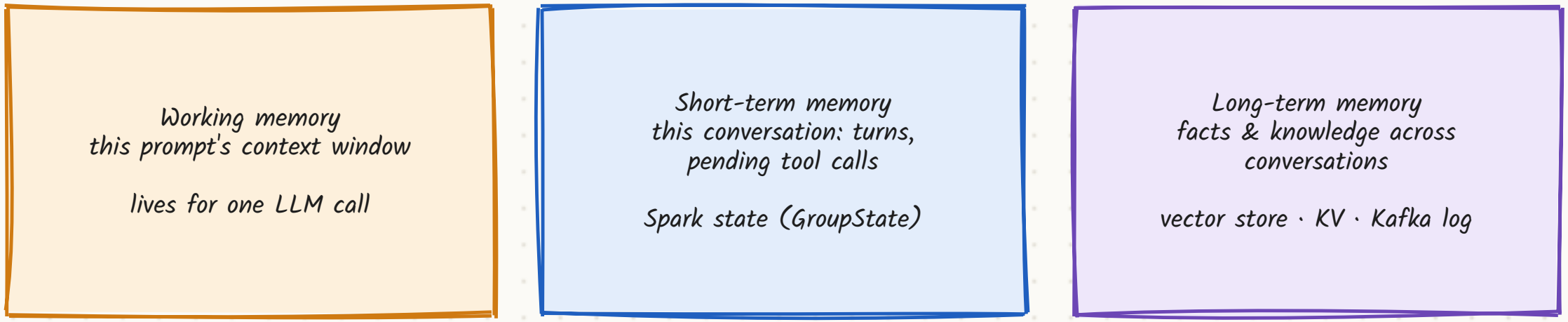
The model never runs a tool. It only ASKS for one, as structured JSON.

Your code decides whether the request is allowed. That decision point is where safety lives.

The agent loop

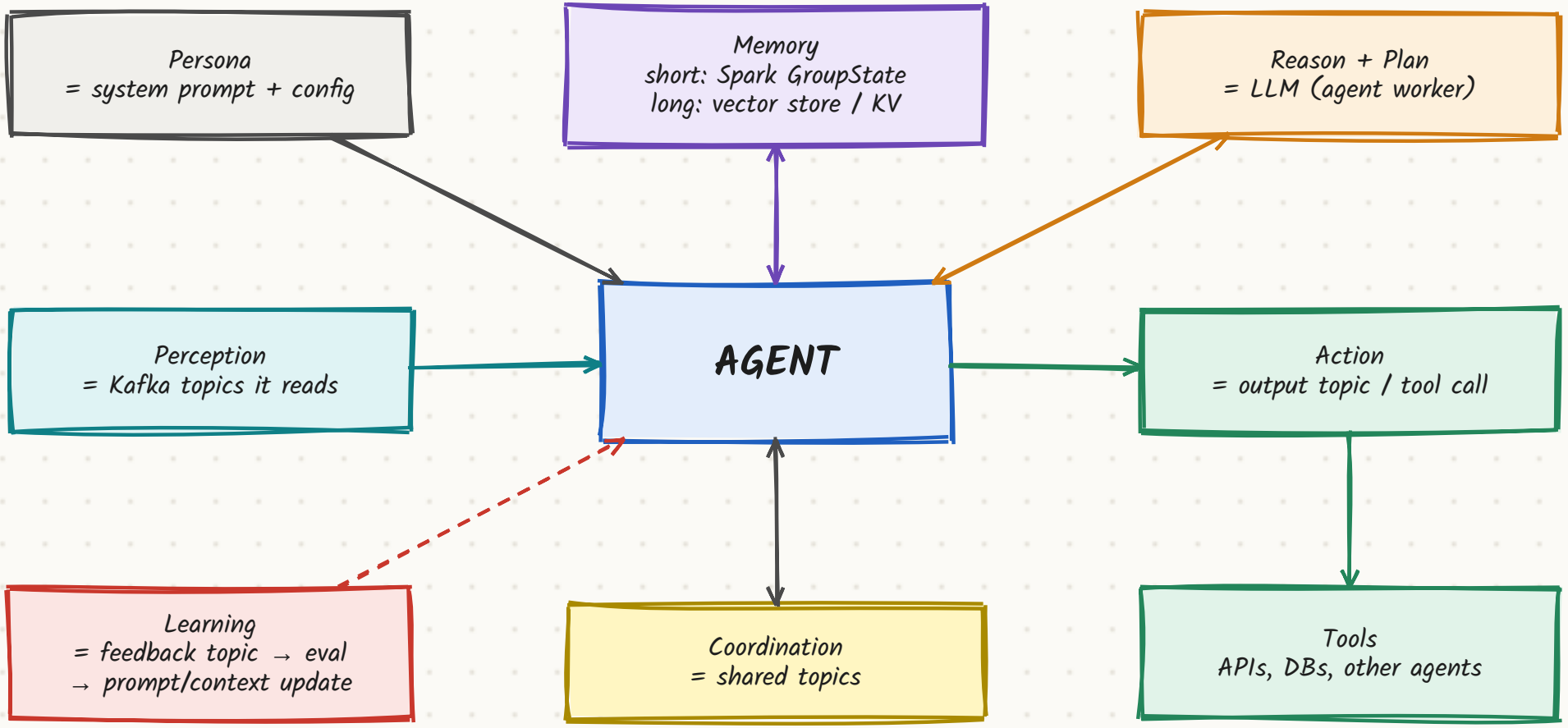


Agent memory



Agents forget by default. Every kind of memory is something we build.

Anatomy of an agent, mapped to Spark and Kafka



From one agent to many

Pattern	Idea	Example in this guide
Orchestrator–worker	one agent splits work, workers do it, results are combined	investigation tasks in L3
Hierarchical	orchestrators of orchestrators	trip → rebooking / baggage / compensation (L2)
Blackboard	agents share findings on a common board	fraud evidence board (L3)
Market-based	agents bid for work, a matcher assigns it	assigning cases to analysts (L3)

What can go wrong with agents

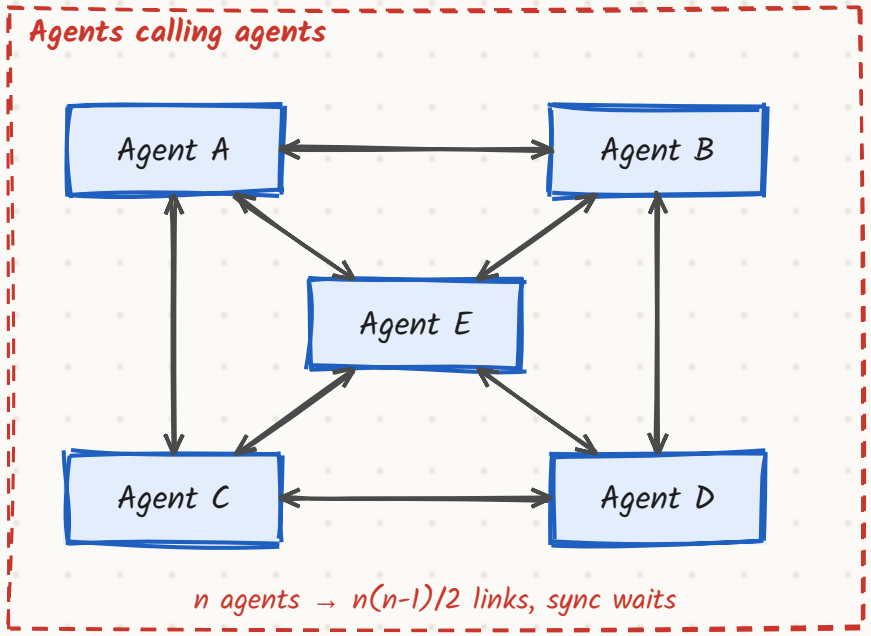
Risk	What it looks like	Main defence
Hallucination	confident answer not supported by facts	fresh context + grounding checks
Stale context	correct reasoning on old facts	streaming context + freshness metrics
Prompt injection	user text tricks the agent into misusing tools	untrusted-data handling + guarded tools
Runaway loops	many steps, many tool calls, high cost	step / time / cost limits
Duplicate actions	a retry refunds twice	idempotency keys
Silent drift	quality slowly drops after a change	continuous evaluation + replay

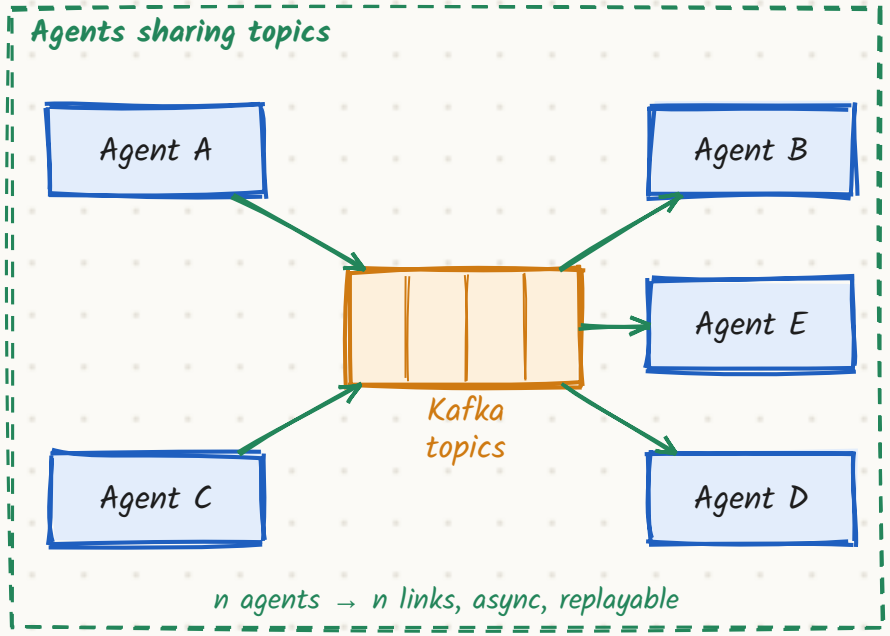
5

Putting it together

Why agents need events, how the Confluent (Flink) material maps to Spark 3.x, and the architecture we will build.

Why agents need events



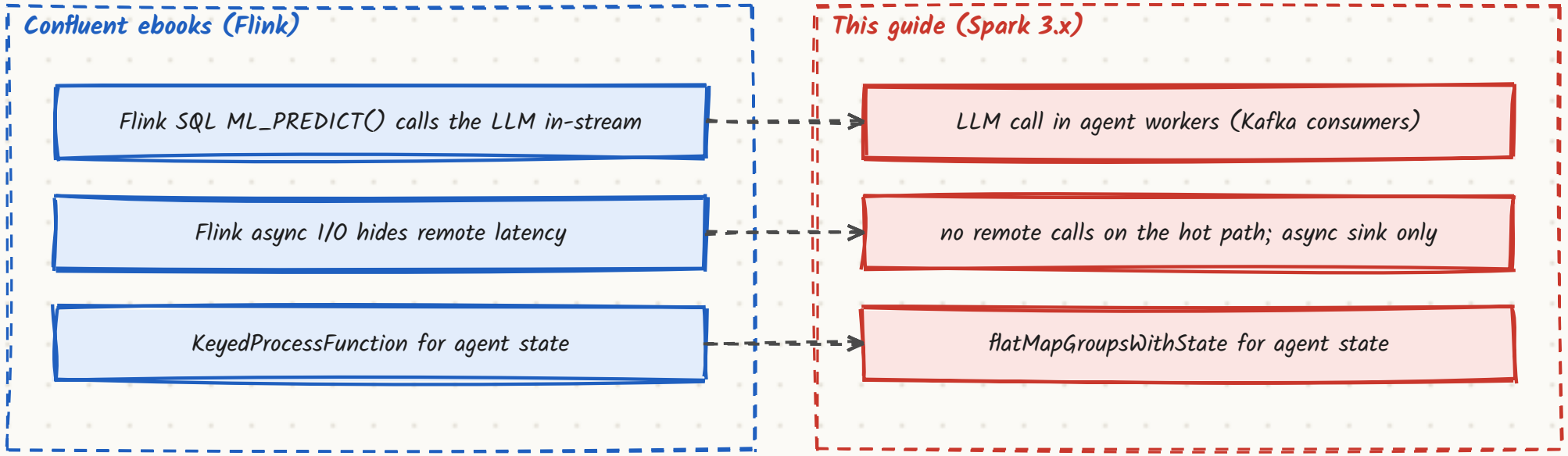


- **Async**: a slow LLM call delays one conversation.
- **Loose coupling**: a new agent is a new consumer.
- **Replay**: the log is memory, audit trail and test set.
- **Fan-out**: one event feeds many agents.

From Confluent's Flink examples to Spark 3.x

Confluent / Flink concept	Spark 3.x (OSS, Scala)	What to watch
Kafka topics, connectors	Same Kafka; Spark Kafka source/sink; Kafka Connect still feeds topics	Kafka sink = at-least-once
Flink SQL / Table API	Spark SQL / typed Dataset API	same ideas, different operators
CREATE MODEL + ML_PREDICT	no SQL-native remote model call; mapPartitions in foreachBatch, or agent workers	never block a low-latency task on an LLM
Async I/O operator	no equivalent; async ForeachWriter or worker service	bound in-flight calls
KeyedProcessFunction + timers	mapGroupsWithState / flatMapGroupsWithState + GroupState timeout	one state object, one timeout per key
Watermarks	withWatermark(col, delay)	global watermark = min across inputs
Exactly-once Kafka sink (2PC)	exactly-once processing; Kafka sink at-least-once	idempotency keys everywhere
Flink Actions: dedup, masking	dropDuplicates(WithinWatermark), UDFs	WithinWatermark needs 3.5
Temporal / interval joins	stream-stream joins with watermarks + time range	append mode only
Stream Governance	Schema Registry + from_avro, contract checks in code	Confluent wire format has a 5-byte header
ms-latency native engine	Continuous Processing (experimental, map-like only)	cores ≥ partitions, no task retry

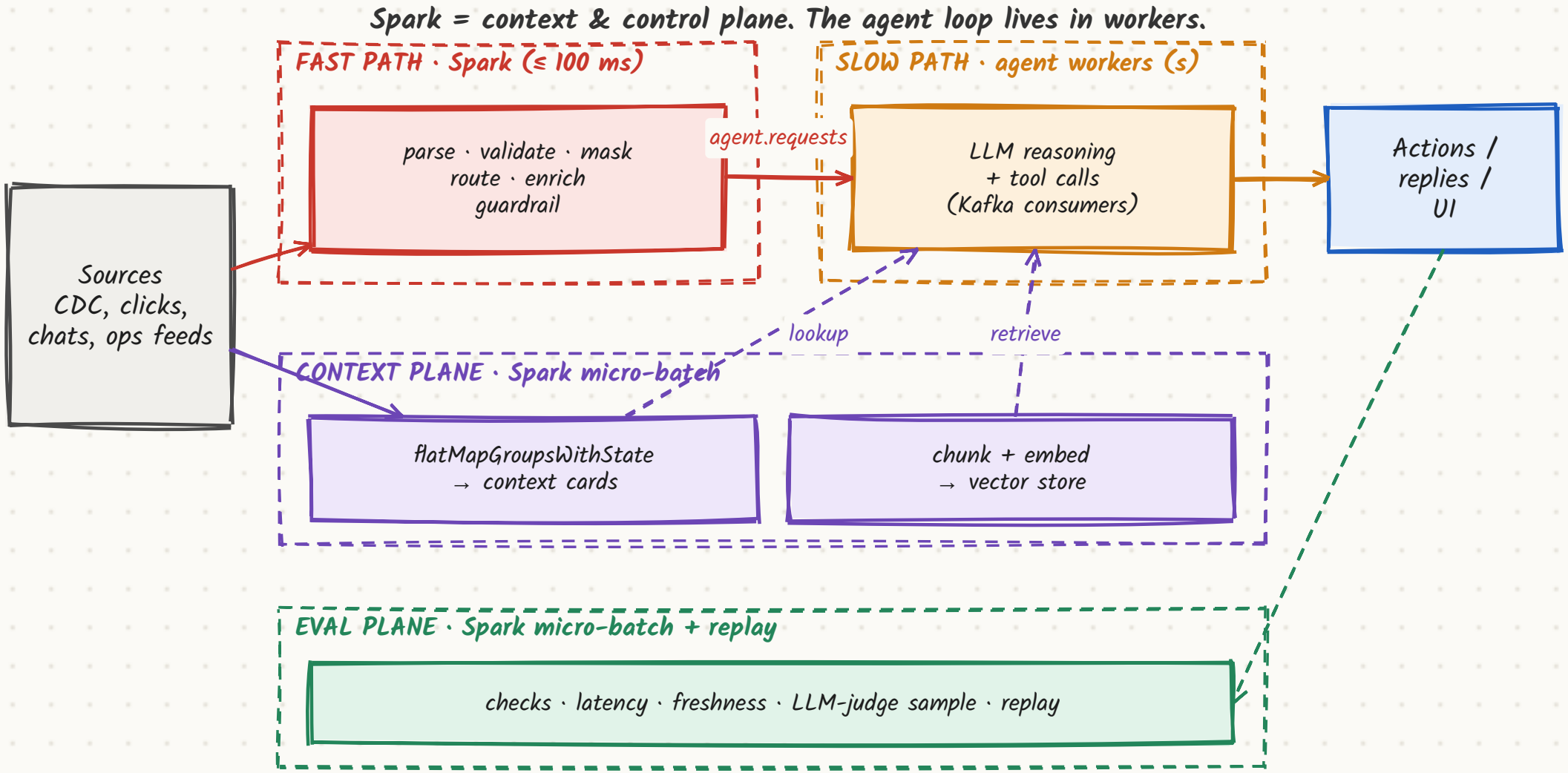
The three design changes that matter most



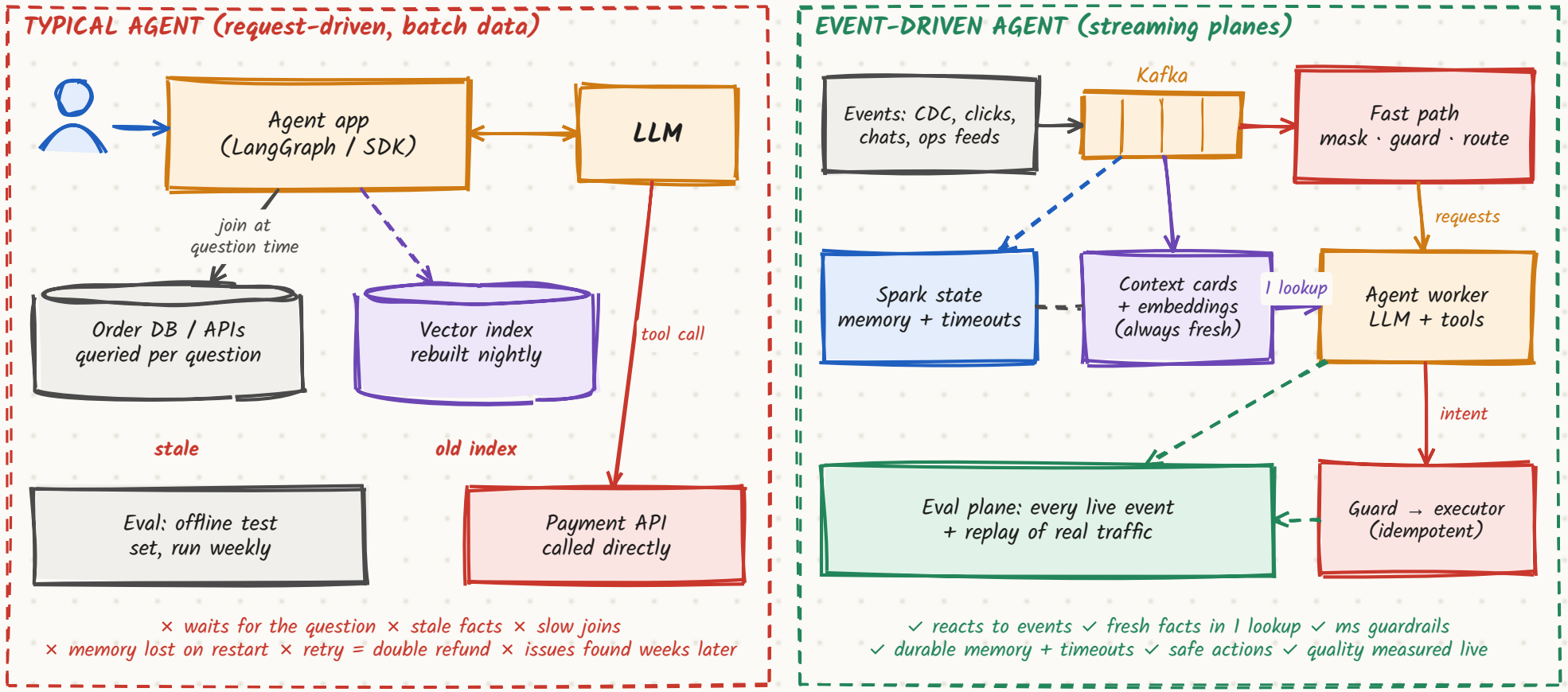
Spark 3.x: what we have to work with

Feature	Since	Why an agent builder cares
Kafka headers in source/sink	3.0	carry trace ids and hop timestamps
Stream-stream full outer & left semi joins	3.1	match requests with results
RocksDB state store	3.2	large agent / card state without GC pauses
Session windows	3.2	group a user's activity into sessions
Trigger.AvailableNow	3.3	replays, backfills, scheduled evals
Changelog checkpointing; async progress tracking	3.4	faster commits, lower latency
dropDuplicatesWithinWatermark	3.5	dedupe by event id with bounded state
Continuous Processing	2.3 (experimental)	~ms stateless fast path

The reference architecture: four planes



Typical agents vs. event-driven agents



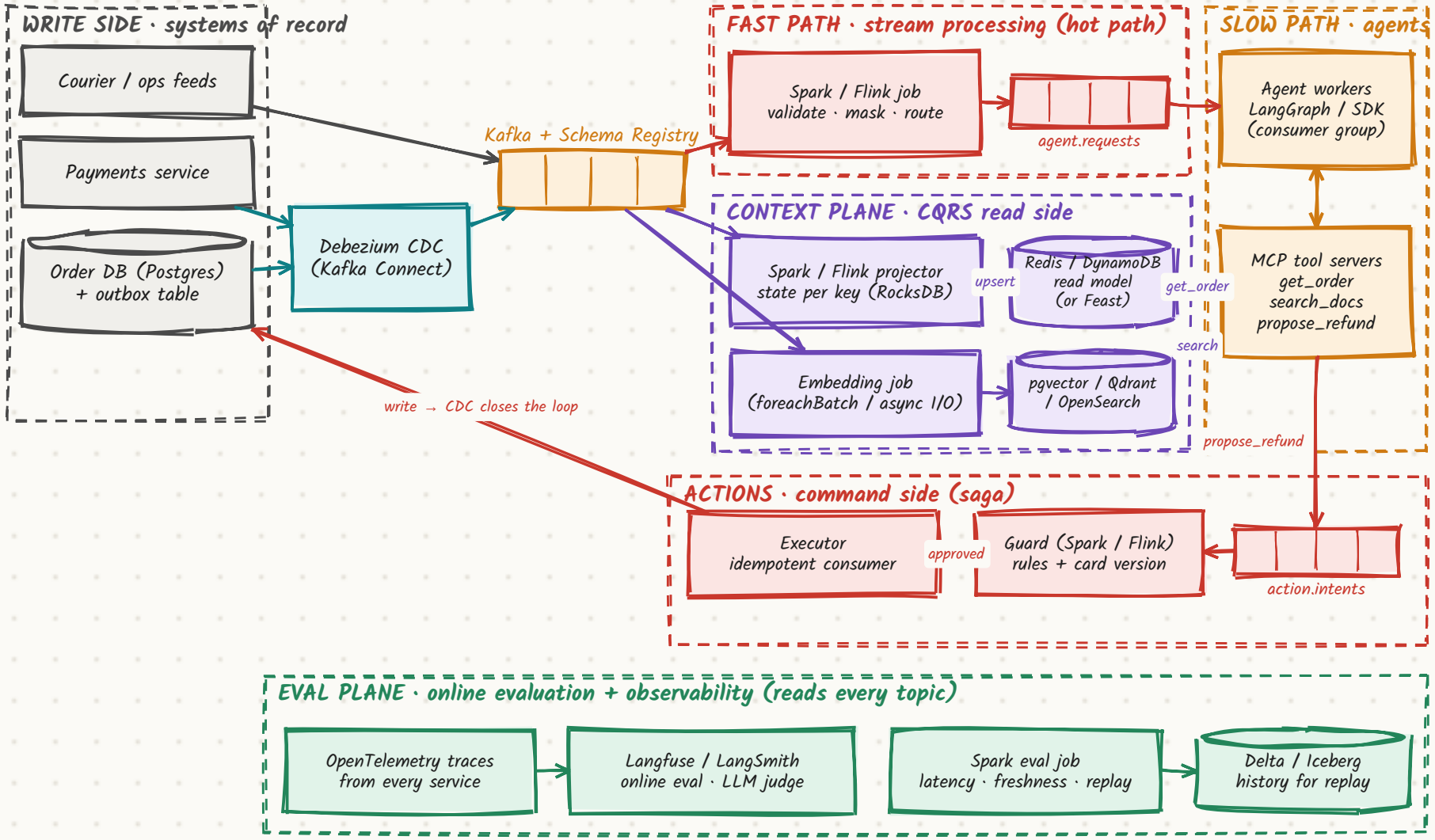
Same LLM, same tools. Streaming changes when the agent runs, what it knows, and how safely it acts.

What each streaming plane fixes

Pain in a typical agent	Streaming plane that fixes it	What you get
Agent only runs when a user asks	Event backbone (Kafka)	proactive agents: a delayed flight starts the conversation
Facts fetched at question time: slow DB/API joins	Context plane: context cards	one key lookup, ~1 ms, no load on production DBs
Vector index rebuilt nightly → stale answers	Context plane: streaming embeddings	knowledge fresh within about a minute
Chat memory in the app process, lost on restart	Spark state (GroupState)	checkpointed memory per conversation, with expiry
User waits forever when the LLM is slow	Timeouts in Spark state	fallback reply within the deadline
PII and abuse reach the LLM unfiltered	Fast path	masking, validation, routing in milliseconds
Tool calls hit payment APIs directly	Intents + deterministic guard	no double refunds, no injected €5,000 refunds
Quality checked offline, weeks later	Eval plane	every live event checked; replay real traffic before release

Not every agent needs this. A low-volume internal assistant is fine as a plain request–response app. The planes pay off when agents react to events, act on money or bookings, need fresh facts, or run at volume.

Four planes, real stack



The established names behind each plane

Plane	Established pattern	Commonly used tools
Write side	System of record; Transactional Outbox	Postgres / MySQL + outbox table
Getting changes out	Change Data Capture	Debezium on Kafka Connect
Event backbone	Event log; Kappa architecture	Kafka + Schema Registry (Avro / Protobuf)
Fast path	Hot path; Content-Based Router; Message Filter	Spark Structured Streaming or Flink
Context plane	CQRS read side; Materialized View; online feature store	Spark / Flink projector → Redis, DynamoDB, OpenSearch; Feast
Context (unstructured)	RAG ingestion pipeline	embedding job → pgvector, Qdrant, OpenSearch
Slow path	Competing Consumers; ReAct agent loop; tool protocol	LangGraph or an agent SDK; MCP tool servers
Actions	Command side; Saga; Idempotent Consumer; Dead Letter Channel	intents topic, guard job, executor service
Eval plane	Online evaluation; LLM-as-judge; observability	OpenTelemetry, Langfuse / LangSmith, Delta / Iceberg, Grafana

The context plane is CQRS

CQRS part	In this architecture
Write model	source databases (order DB, payments); unchanged
Change capture	Debezium CDC or a transactional outbox → Kafka
Event log	Kafka topics
Projector	Spark `flatMapGroupsWithState` (or Flink keyed state): one record per key
Read model	compacted topic (durable copy) + Redis / DynamoDB (served to agents)
Query	agent tool `get_order`: one key lookup
Command	agent publishes an intent → guard → executor writes the DB

- ➜ **State store = kitchen**: Spark's working memory, not queryable from outside.
- ➜ **Redis = serving counter**: what agents and the guard read.
- ➜ **Many read models**: Redis for lookups, vector store for search, Delta for analytics and replay.
- ➜ **Loop closes**: executor write → CDC → card updated in seconds.

Why a materialized view, not a normal view?

	Normal view	Database materialized view	Streaming materialized view (card)
Work happens	on every read	on a refresh schedule	on every change
Freshness	always current	stale between refreshes	seconds behind
Cost per read	full query with joins	a lookup	a lookup
Load on production DB	every agent call	at refresh time	none
Joins across systems	no (one database)	no (one database)	yes (any topic)
History-derived facts	no	limited	yes (“ETA changed 3 times”)

A normal view does the work on every read. A card does the work on every change. Agents read far more often than data changes.

One request, end to end

- **1.** Chat message lands on `customer.messages` (Kafka).
- **2.** Fast path masks the card number, routes to `agent.requests`.
- **3.** A LangGraph worker in the consumer group picks it up.
- **4.** Tools: `get_order` → Redis (~1 ms); `search_docs` → vector store (refund policy).
- **5.** LLM decides a late-delivery refund applies → `propose_refund` publishes an intent.
- **6.** Guard checks rules and card version → executor refunds once (idempotency key).
- **7.** Debezium sees the DB change → projector updates the card → next turn sees “refunded”.
- **8.** Every step emits OpenTelemetry spans; Langfuse scores a sample; the eval job tracks latency and freshness.

Choosing the engine per plane

Plane	Spark Structured Streaming 3.x	Apache Flink	Kafka Streams
Fast path	micro-batch ~100 ms+; continuous mode experimental	per record, ms; best fit	per record, ms; no cluster to run
Per-key state + timers	GroupState; one timeout per key	keyed state; many timers per key	state stores; punctuators
Context cards	flatMapGroupsWithState → topic / Redis	keyed state → upsert-kafka / Redis	KTable; Interactive Queries can serve cards
Embeddings / remote calls	foreachBatch (batched calls)	Async I/O (non-blocking)	blocking calls in the processor
Lake, backfill, replay	strongest: same API for batch	possible (batch mode)	Kafka in, Kafka out only
Scala	first-class	Scala API removed in Flink 2.0 (use Java API)	Scala wrapper available

Fresh start and the fast path needs milliseconds → Flink. Team and data already on Spark → Spark can run all four planes. Kafka-only and small → Kafka Streams.

Q1

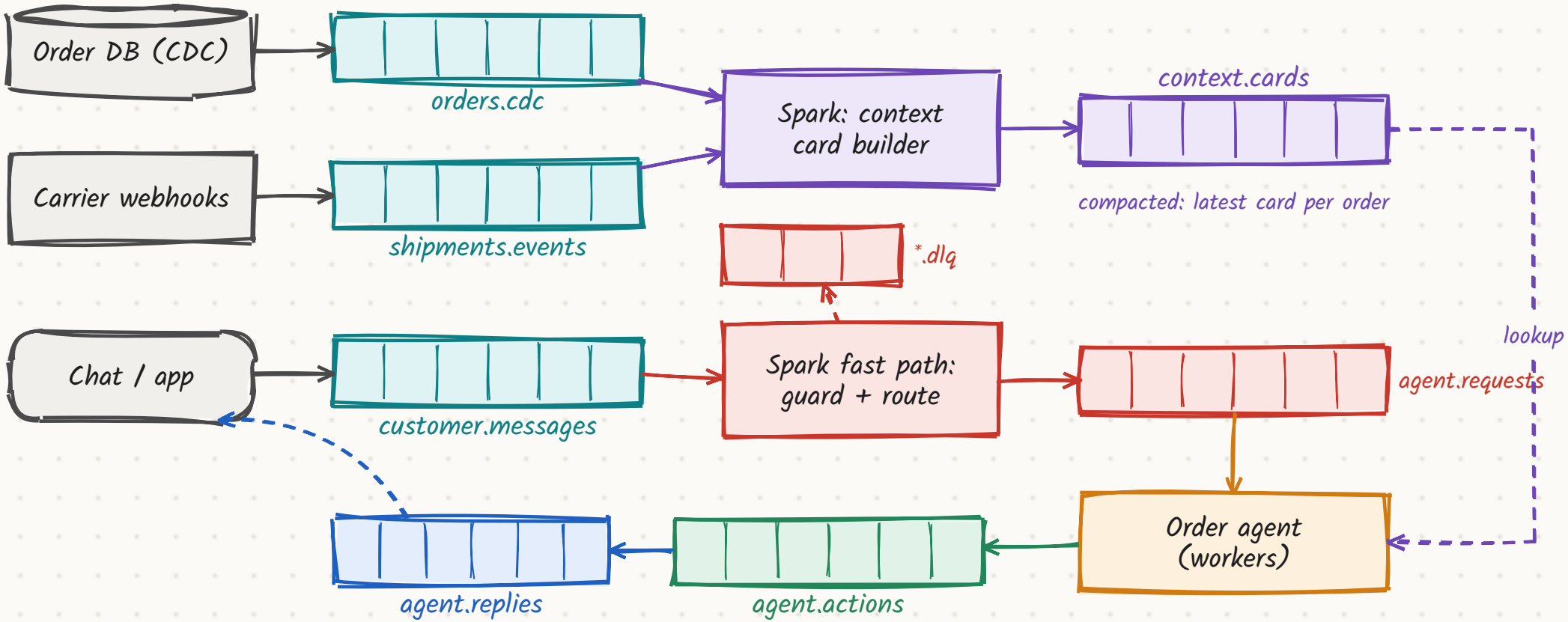
How do I set up the event-driven architecture?

Topics, keys, envelopes, request-reply and workers.

Q1 · Event-driven setup

Start from topics, not from code (L1)

L1 Name topics after **facts** (orders.cdc) or **intents** (action.intents), never after a consumer.



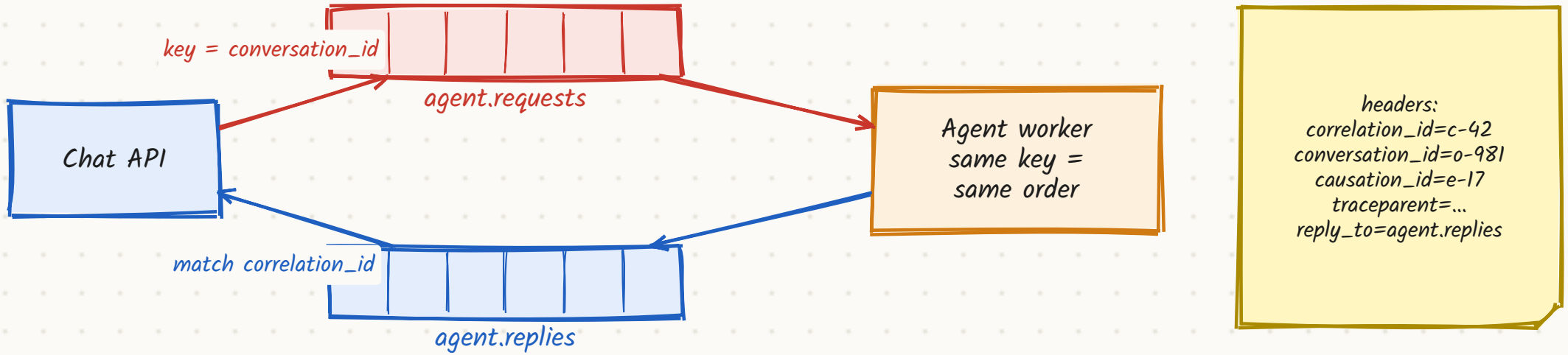
Q1 · Event-driven setup

One envelope, three ids, one key

```
// One envelope for every event an agent sees or emits
case class AgentEvent(
  eventId:        String,          // UUID, also the idempotency key
  correlationId:  String,          // ties a request to its reply
  conversationId: String,          // Kafka KEY → ordering + state key
  causationId:    Option[String],  // which event caused this one
  eventType:      String,          // customer.message | agent.result | agent.reply …
  eventTime:      java.sql.Timestamp,
  schemaVersion:  Int,
  payload:        String           // JSON body, validated against a schema
)
```

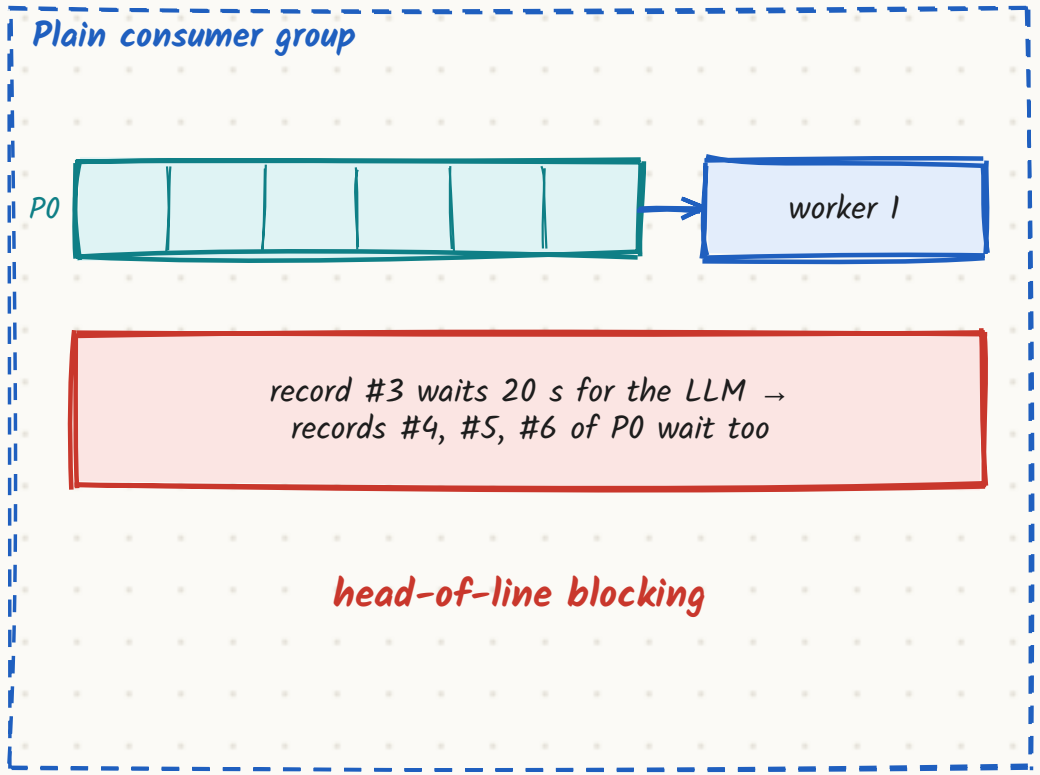
- **Kafka key = conversationId / orderId** → ordering + state locality.
- **eventId** = idempotency key downstream.
- **correlationId** pairs request ↔ reply; **causationId** builds the trace graph.
- Trace context (`traceparent` , `origin_ts`) goes in **headers**.

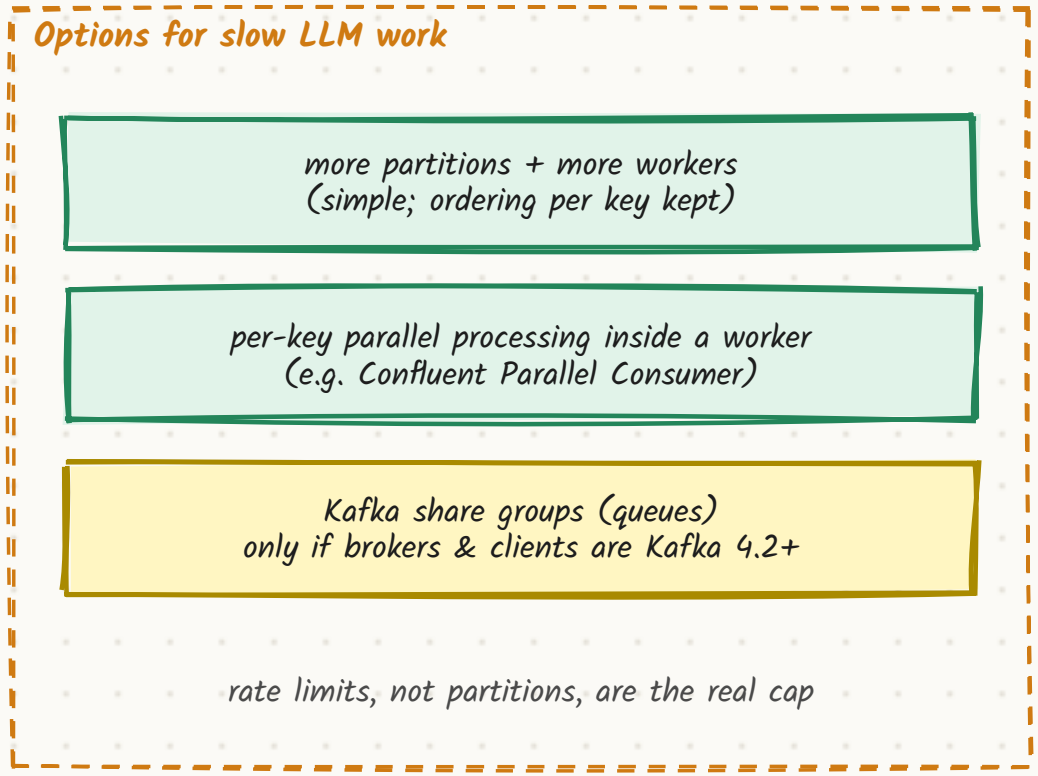
Request–reply without holding a connection



Async request-reply: the caller never holds a connection to the agent; it waits on a reply topic.

Workers for slow LLM calls





Q1 · Event-driven setup

Local stack in 15 minutes

```
# 12 partitions = up to 12 parallel consumers / Spark tasks per topic
kafka-topics.sh --bootstrap-server $B --create --topic customer.messages \
  --partitions 12 --replication-factor 3 --config retention.ms=2592000000   # 30 d → replayable
kafka-topics.sh --bootstrap-server $B --create --topic agent.requests  --partitions 12
kafka-topics.sh --bootstrap-server $B --create --topic agent.results   --partitions 12
kafka-topics.sh --bootstrap-server $B --create --topic agent.replies   --partitions 12
kafka-topics.sh --bootstrap-server $B --create --topic action.intents  --partitions 12
kafka-topics.sh --bootstrap-server $B --create --topic context.cards   --partitions 12 \
  --config cleanup.policy=compact                                        # latest card per key
kafka-topics.sh --bootstrap-server $B --create --topic agent.dlq       --partitions 3
```

```
// build.sbt: Spark jobs and agent workers in separate modules
val sparkV = "3.5.3"
lazy val jobs = project.settings(
  scalaVersion := "2.12.18",
  libraryDependencies ++= Seq(
    "org.apache.spark" %% "spark-sql"            % sparkV % Provided,
    "org.apache.spark" %% "spark-sql-kafka-0-10" % sparkV,
    "org.apache.spark" %% "spark-avro"           % sparkV))
lazy val workers = project.settings(   // plain JVM service, no Spark on the classpath
  scalaVersion := "2.12.18",
  libraryDependencies += "org.apache.kafka" % "kafka-clients" % "3.7.1")
```

Reading and writing agent events in Scala

```scala
val raw = spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", brokers)
  .option("subscribe", "customer.messages")
  .option("includeHeaders", "true")          // trace ids travel in headers (3.0+)
  .option("startingOffsets", "latest")
  .option("maxOffsetsPerTrigger", "20000")   // cap a batch after downtime
  .load()

val msgs = raw.select(
    col("key").cast("string").as("conversationId"),
    from_json(col("value").cast("string"), msgSchema).as("m"),
    col("headers"), col("timestamp").as("kafkaTs"))
  .select(col("conversationId"), col("m.*"), col("headers"), col("kafkaTs"))

// write back: key + value (+ headers, + optional per-row "topic" column)
msgs.select(col("conversationId").as("key"),
            to_json(struct(col("*"))).as("value"), col("headers"))
  .writeStream.format("kafka")
  .option("kafka.bootstrap.servers", brokers)
  .option("topic", "agent.requests")
  .option("checkpointLocation", "/chk/gate")
  .start()
```

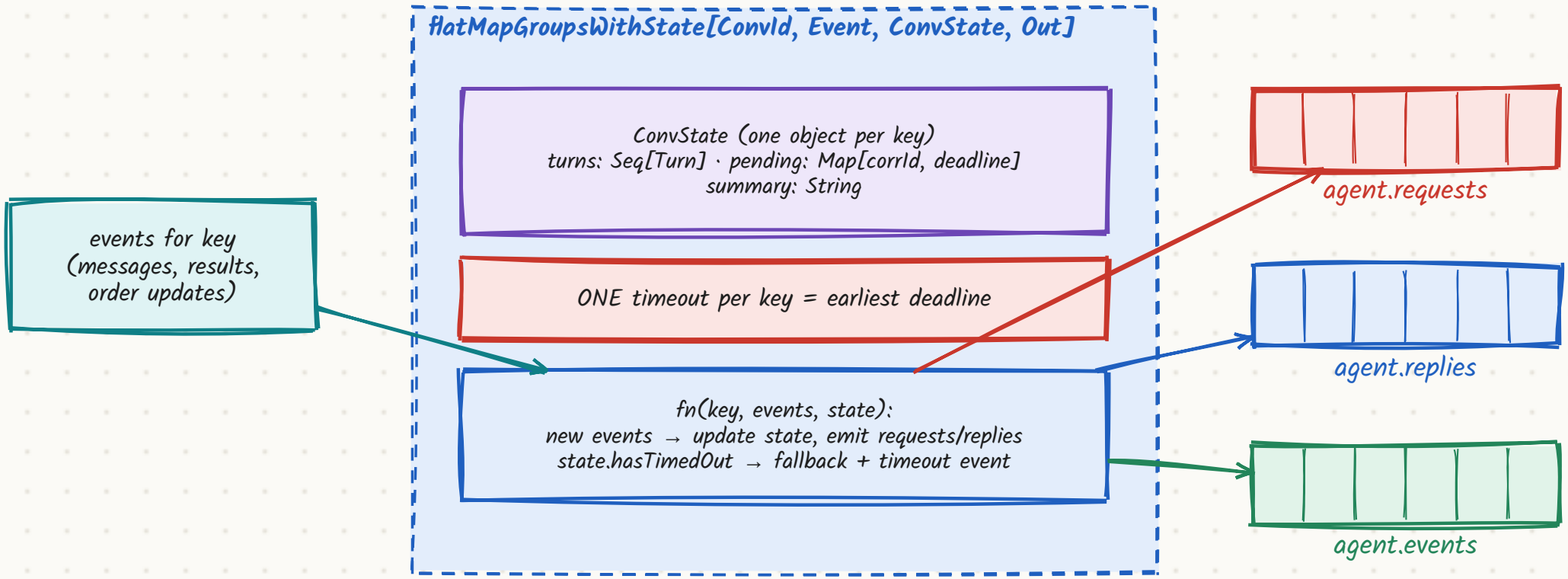
- `includeHeaders` keeps trace ids.
- Kafka `timestamp` = free latency telemetry.
- `maxOffsetsPerTrigger` protects you after downtime.
- A `topic` column routes rows to many topics.
- Bad JSON → `from_json` returns null → DLQ.

Q2

How do I build agents?

The part that remembers lives in Spark state. The part that reasons lives in a worker.

An agent's memory is a GroupState



Memory = GroupState (checkpointed). Forgetting = remove(). Patience = timeout.

The conversation function in Scala

```
object ConversationAgent {
  val Patience = 2500L                                  // ms we wait for an agent result

  def step(key: String, events: Iterator[InEvent],
           state: GroupState[ConvState]): Iterator[OutEvent] = {
    val now = state.getCurrentProcessingTimeMs()
    var s   = state.getOption.getOrElse(ConvState(Seq.empty, Map.empty, ""))
    val out = scala.collection.mutable.ArrayBuffer.empty[OutEvent]

    if (state.hasTimedOut && s.pending.isEmpty) {       // idle for 2 h: forget everything
      state.remove(); return Iterator.empty
    }
    if (state.hasTimedOut) {                            // no new events: patience ran out
      s.pending.filter(_._2 ⩽ now).keys.foreach { cid ⇒
        out += OutEvent(key, "agent.replies", "fallback", cid, "Still checking - one moment.")
        out += OutEvent(key, "agent.events",  "timeout",  cid, s"""{"atMs":$now}""")
      }
      s = s.copy(pending = s.pending.filter(_._2 > now))
    } else events.toSeq.sortBy(_.eventTimeMs).foreach { e ⇒ e.eventType match {
      case "customer.message" ⇒
        s = s.copy(turns = (s.turns :+ Turn("user", e.text, e.eventTimeMs)).takeRight(12),
                   pending = s.pending + (e.correlationId → (now + Patience)))
        out += OutEvent(key, "agent.requests", "agent.request", e.correlationId,
                        Prompt.request(key, e, s.turns.takeRight(6), s.summary))
      case "agent.result" if s.pending.contains(e.correlationId) ⇒
        s = s.copy(turns = (s.turns :+ Turn("assistant", e.text, e.eventTimeMs)).takeRight(12),
                   pending = s.pending - e.correlationId)
        out += OutEvent(key, "agent.replies", "reply", e.correlationId, e.text)
      case _ ⇒                                          // late or duplicate result: ignore
    }}
```

When patience runs out

```
if (state.hasTimedOut && s.pending.isEmpty) {       // idle for 2 h: forget everything
  state.remove(); return Iterator.empty
}
if (state.hasTimedOut) {                            // no new events: patience ran out
  s.pending.filter(_._2 ⩽ now).keys.foreach { cid ⇒
    out += OutEvent(key, "agent.replies", "fallback", cid, "Still checking - one moment.")
    out += OutEvent(key, "agent.events",  "timeout",  cid, s"""{"atMs":$now}""")
  }
  s = s.copy(pending = s.pending.filter(_._2 > now))
```

The timeout fires **even if no new event arrives** for this conversation.

The user always gets an answer within the SLA: a real one or an honest fallback.

Wiring it: one keyed stream, events out

```
// messages AND agent results share one keyed stream → one state per conversation
val in: Dataset[InEvent] = readKafka("customer.messages,agent.results").as[InEvent]

val out = in.groupByKey(_.conversationId)
  .flatMapGroupsWithState(OutputMode.Append, GroupStateTimeout.ProcessingTimeTimeout)(
    ConversationAgent.step)                  // we emit EVENTS, so Append

out.select(col("conversationId").as("key"), col("topic"),        // per-row topic routing
           to_json(struct(col("*"))).as("value"))
  .writeStream.format("kafka")
  .option("kafka.bootstrap.servers", brokers)
  .option("checkpointLocation", "/chk/conversation-agent")
  .trigger(Trigger.ProcessingTime("250 milliseconds"))
  .start()
```

Two topics, one key: **messages + results** of the same conversation reach the **same state**.

Output mode **append**: we emit events, not a table.

The worker: where the LLM actually runs

```
// Agent worker: plain JVM service in a classic consumer group
val props = new java.util.Properties()
props.put("bootstrap.servers", brokers)
props.put("group.id", "order-agent-workers")
props.put("enable.auto.commit", "false")               // commit only after we published
props.put("max.poll.records", "20")                    // small batches: LLM calls are slow
props.put("max.poll.interval.ms", "300000")            // must exceed the slowest batch
props.put("key.deserializer",   classOf[StringDeserializer].getName)
props.put("value.deserializer", classOf[StringDeserializer].getName)

val consumer = new KafkaConsumer[String, String](props)
consumer.subscribe(java.util.Arrays.asList("agent.requests"))

while (running) {
  val recs = consumer.poll(java.time.Duration.ofMillis(200)).asScala.toSeq
  // different conversations in parallel, same conversation in order
  val work = recs.groupBy(_.key).values.map { sameKey ⇒ Future {
    sameKey.foreach { rec ⇒
      val result = Retry.withBackoff(OrderAgent.run(AgentRequest.parse(rec.value), maxSteps = 5))
        .getOrElse(AgentResult.failed(rec))            // after retries: answer honestly + DLQ
      producer.send(new ProducerRecord("agent.results", rec.key, result.toJson)).get()
    }
  }}
  Await.result(Future.sequence(work), 4.minutes)
  consumer.commitSync()                                // at-least-once: results may repeat
}
```

- Plain JVM service, scaled by pods, not Spark executors.
- Different conversations in parallel, same conversation in order.
- Commit offsets **after** results are published.
- Retries with backoff; give up honestly + DLQ.

Q2 · Building agents

The agent loop inside the worker

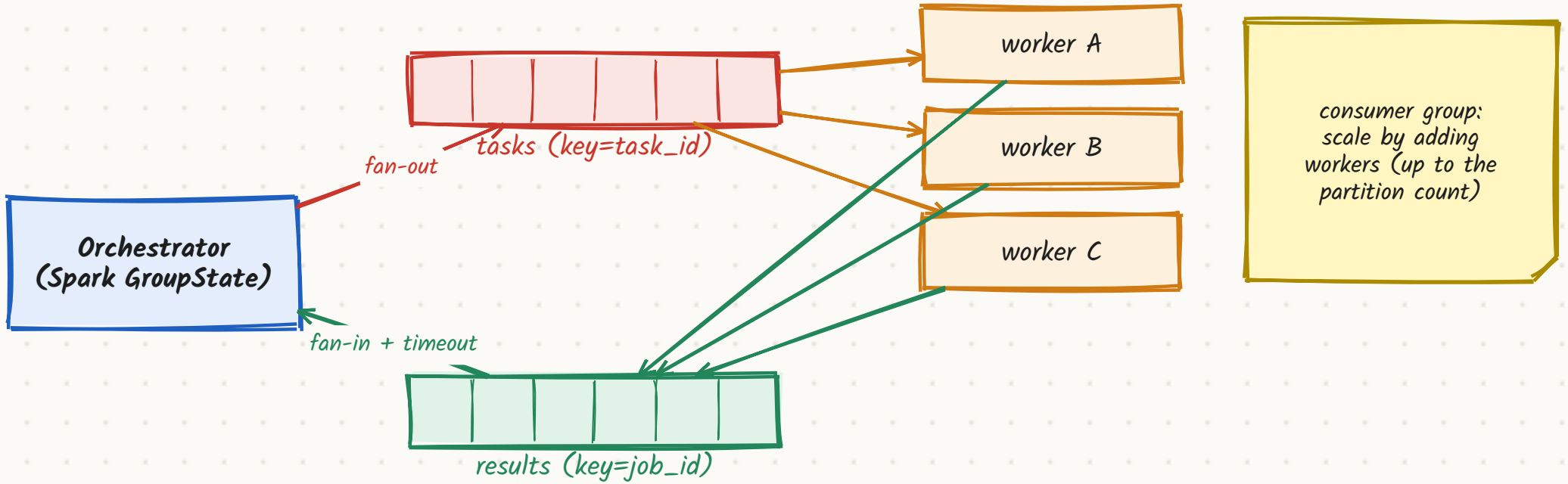
```
object OrderAgent {
  val tools = Map(
    "get_order"      → Tools.getOrder _,       // reads the context card (KV)
    "track_shipment" → Tools.trackShipment _,  // carrier API, 800 ms timeout
    "request_refund" → Tools.requestRefund _)  // WRITES an intent, never pays directly

  def run(req: AgentRequest, maxSteps: Int): AgentResult = {
    var msgs = Prompt.initial(req)             // system + card + policies + turns
    for (step ← 1 to maxSteps) {
      val r = Llm.chat(msgs, tools = ToolSchemas.all, timeoutMs = 4000)
      r.toolCall match {
        case Some(call) if tools.contains(call.name) ⇒
          val out = tools(call.name)(call.args, req.idempotencyKey(step))
          msgs = msgs :+ Msg.toolCall(call) :+ Msg.toolResult(call.id, out)
        case Some(call) ⇒ msgs = msgs :+ Msg.error(s"unknown tool ${call.name}")
        case None ⇒ return AgentResult.answer(req, r.text, r.citedIds, r.usage)
      }
    }
    AgentResult.handoff(req, "max steps reached")     // bounded autonomy
  }
}
```

- Max steps = bounded autonomy.
- Read tools use prepared context, not live databases.
- Write tools only emit **intents**.
- Idempotency key = request id + step.

Q2 · Building agents

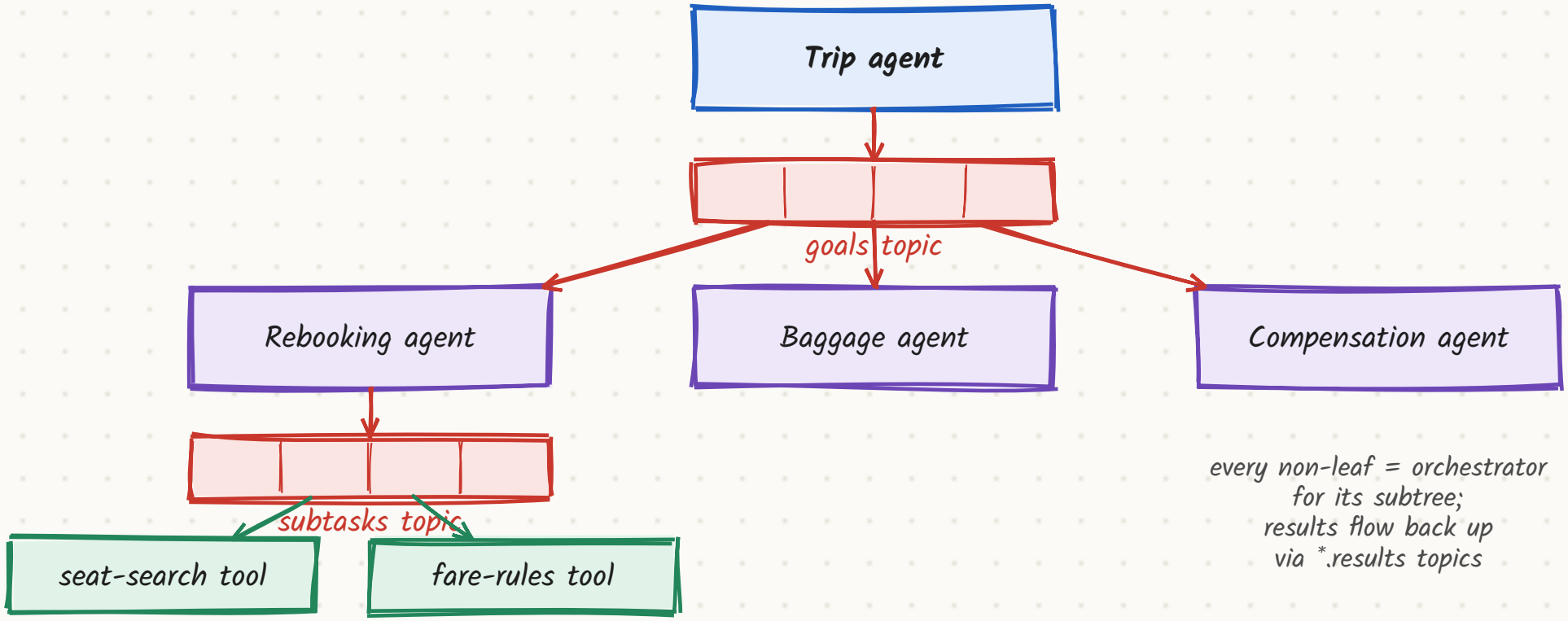
Pattern 1 · Orchestrator–worker



The orchestrator's state holds the outstanding task ids; its timeout fires if a worker never answers.

Pattern 2 · Hierarchical agents (L2)

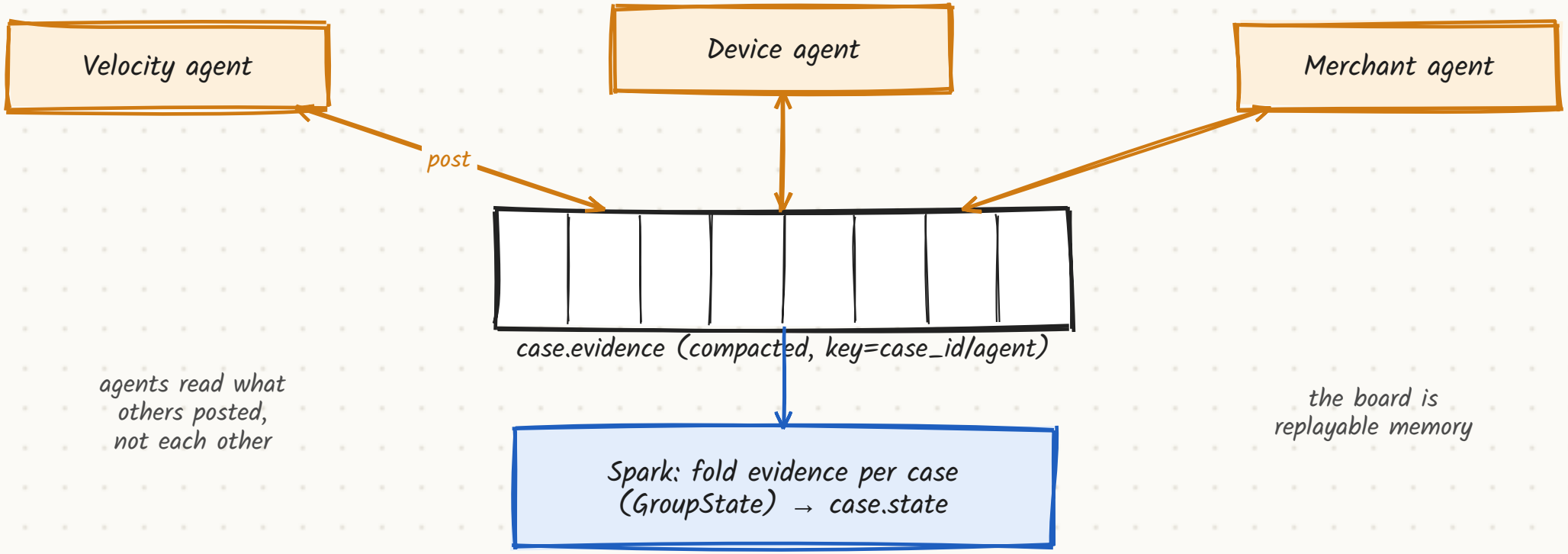
L2



every non-leaf = orchestrator for its subtree; results flow back up via *.results topics

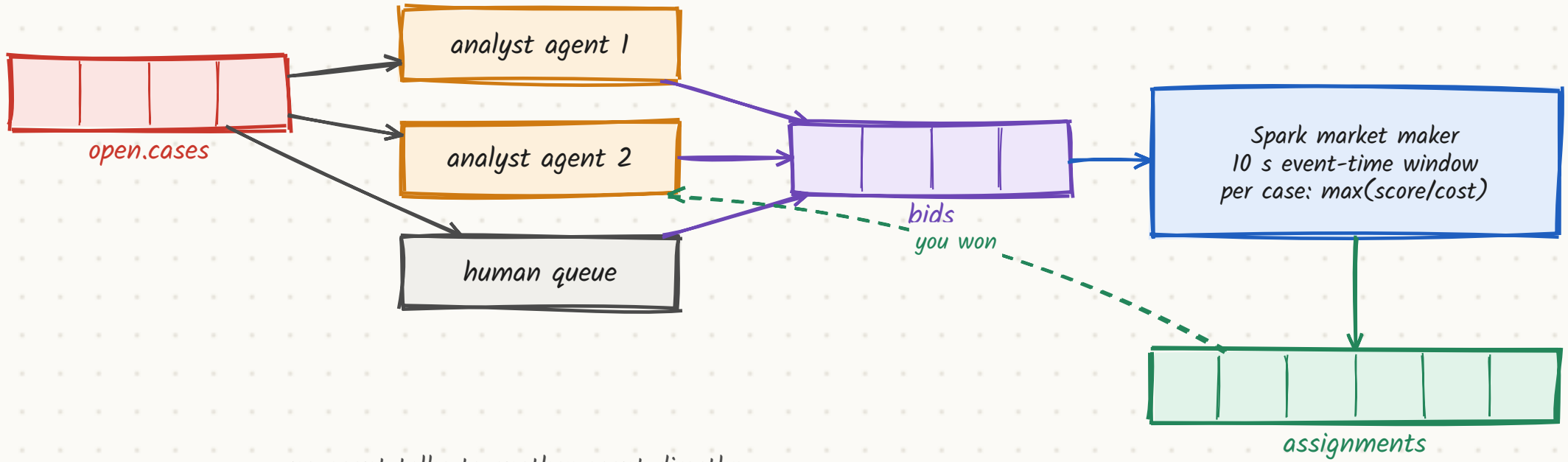
Pattern 3 · Blackboard (L3)

L3



Pattern 4 · Market-based assignment (L3)

L3



no agent talks to another agent directly:
quadratic chatter becomes one log

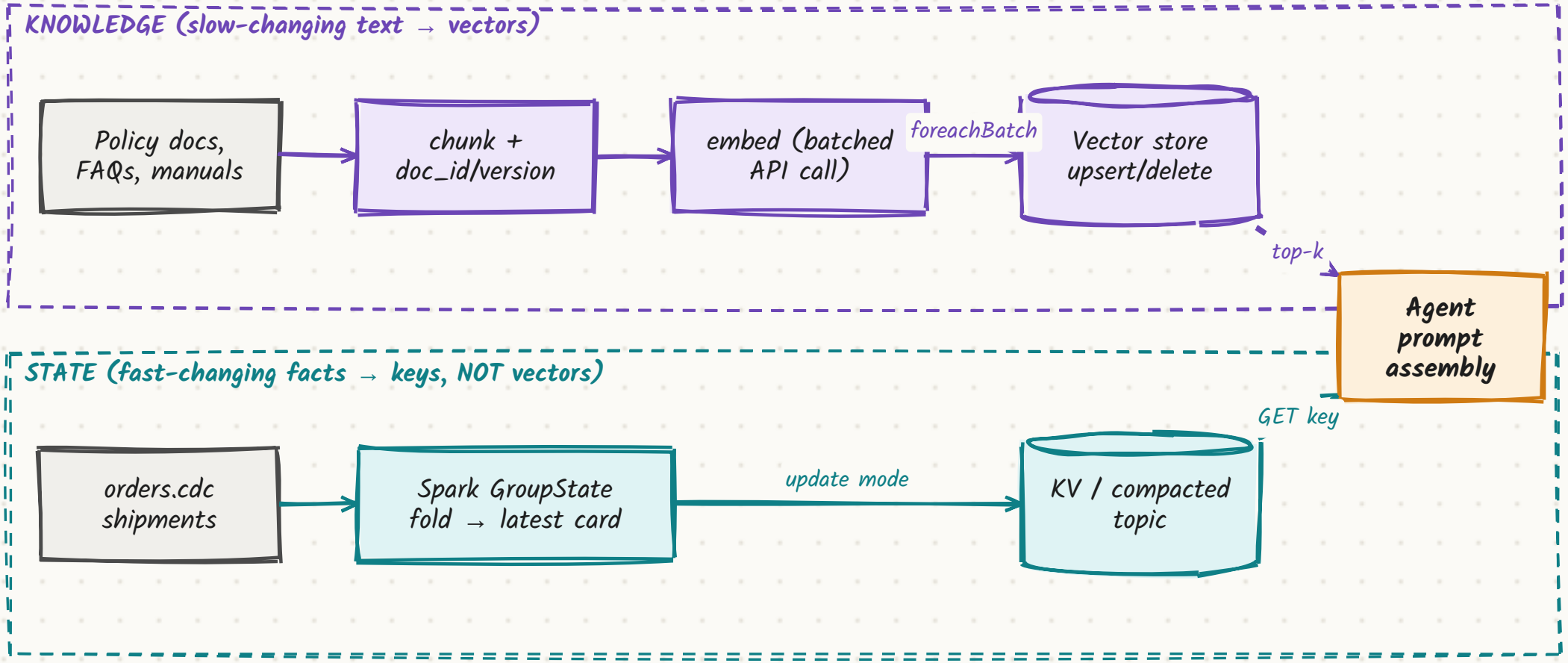
Q3

Live RAG and tools that use live data

What to embed, how to keep it fresh, and how to let an LLM act safely.

Q3 · Live RAG & live tools

Two kinds of context, two pipelines



Rule: if the answer is a number that changes (ETA, balance, status), look it up by key; never trust a stale embedding.

Q3 · Live RAG & live tools

Streaming embedding pipeline

```
docs.writeStream                                    // docs = CDC of the policy / FAQ store
  .foreachBatch { (batch: DataFrame, batchId: Long) ⇒
    val upserts = batch.where(col("op") =!= "d").as[Doc]
      .flatMap(d ⇒ Chunker.split(d, maxTokens = 400, overlap = 50)) // id = docId#n, docVersion
      .mapPartitions(it ⇒ Embeddings.embedBatched(it, batchSize = 64)) // 1 API call / 64 chunks
    upserts.foreachPartition { (p: Iterator[EmbeddedChunk]) ⇒
      VectorStore.upsert(p) }                       // same id + version = no-op on replay
    batch.where(col("op") === "d").select("docId").as[String]
      .foreachPartition { (p: Iterator[String]) ⇒ VectorStore.deleteByDoc(p) } // tombstones
  }
  .option("checkpointLocation", "/chk/embeddings")
  .trigger(Trigger.ProcessingTime("30 seconds"))    // knowledge freshness ≈ 1 minute
  .start()
```

- **foreachBatch** → batch the embedding API calls.
- Chunk id = `docId#n` + version → idempotent upserts.
- Handle **deletes**: stale policy chunks are a classic RAG bug.
- Freshness ≈ trigger + embedding time (~1 min).

Q3 · Live RAG & live tools

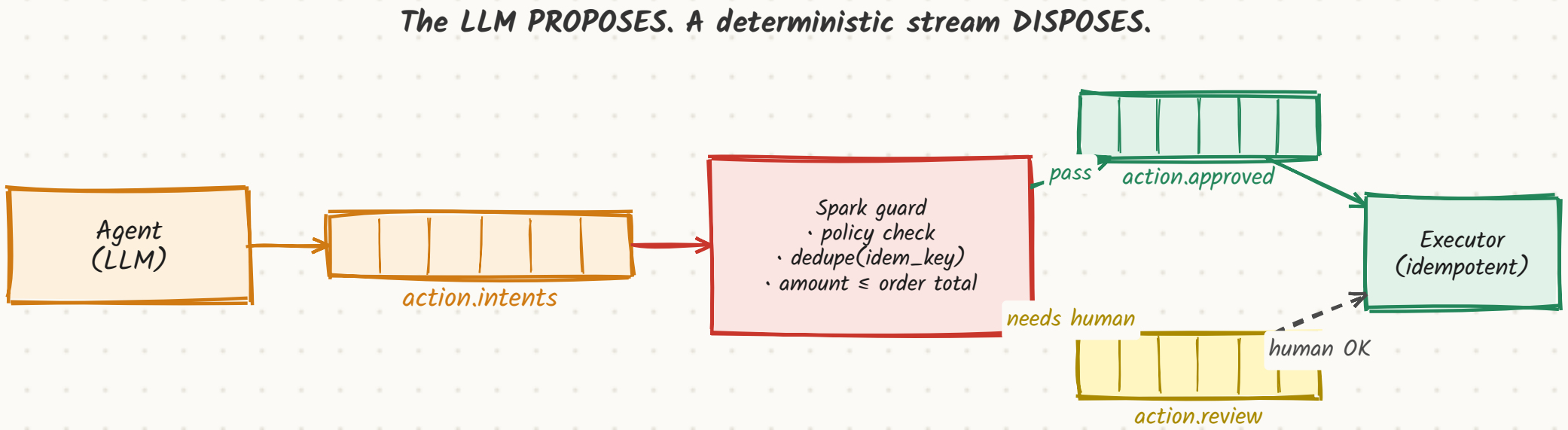
Embed it or look it up?

Data	Embed?	Why
Return policy, FAQs, manuals	yes	meaning matters, changes slowly
Support transcripts / reviews	yes (chunked)	free text
Order status, ETA, balance	no → key lookup	changes every minute
IDs, card numbers	never	no meaning; personal data
Aggregates (refund rate)	no → state / SQL	numbers, not meaning
Prices / amounts	describe first	e.g. tier = budget / premium

Confluent's RAG guide makes the same split between structured and unstructured data.

In Spark, the fast-changing half is a **GroupState card**, not a vector.

Live tools: the LLM proposes, a stream disposes



Q3 · Live RAG & live tools

Tool rules for live data

- **Read tools** read prepared context cards, not production databases.
- **Write tools** emit `action.intents` with an idempotency key.
- A **deterministic guard** checks policy against the current card.
- Above a threshold → **human review**.
- Every tool result is **logged as an event**.

```
val intents = readKafka("action.intents")
  .select(from_json(col("value").cast("string"), intentSchema).as("i")).select("i.*")
  .withWatermark("intentTime", "10 minutes")
  .dropDuplicatesWithinWatermark("idempotencyKey")      // Spark 3.5+

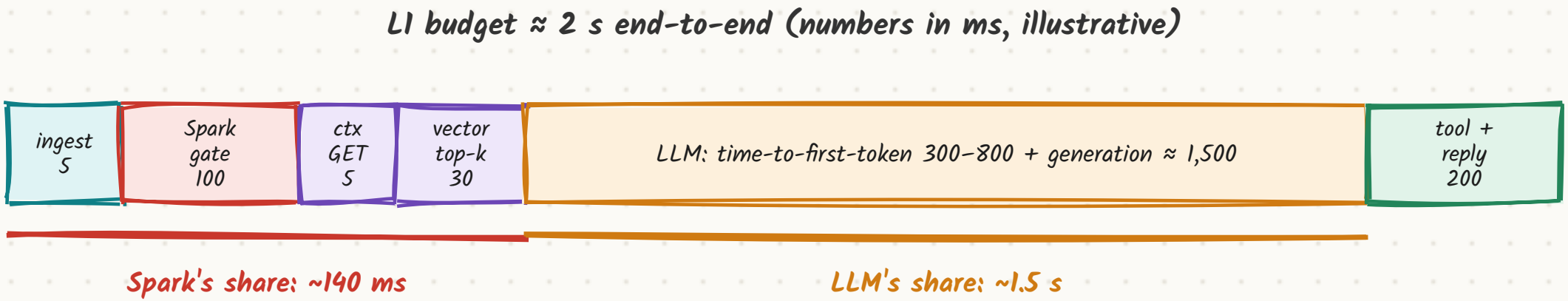
// check each intent against the CURRENT card, batched per partition
val verdicts = intents.as[Intent].mapPartitions { it ⇒
  it.grouped(200).flatMap { batch ⇒
    val cards = CardStore.mget(batch.map(_.orderId))    // one round-trip per 200 intents
    batch.map(i ⇒ Guard.decide(i, cards.get(i.orderId)))   // pure, unit-tested rules
  }
}
// Guard.decide → "approve" | "review" (amount > 200) | "reject" (stale card / amount > total)
```

Q4

Which design considerations matter?

Budgets, failure modes, and the rules that keep agents debuggable.

Draw the latency budget first



→ continuous mode matters for guards & fraud holds, much less for chat replies

Q4 · Design considerations

Design checklist

Area	Rule	Why
Keys	key by conversation / entity id	ordering and state locality
Idempotency	every side effect carries a key derived from eventId	Kafka sink and workers redeliver
Replay determinism	LLM outputs and tool results are events	re-calling the LLM gives a different answer
Contracts	schema registry + schemaVersion in the envelope	models read payloads as text; drift is silent
Bounded autonomy	max steps, max amount, max cost per conversation	unbounded loops are unbounded bills
Backpressure	scale workers on lag; back off on rate limits	provider limits are the real throughput cap
Cost	rules or templates before the LLM; cache stable prompt parts	many questions need no model
Personal data	mask in the fast path before any prompt or log	prompts are data sent to a third party
Observability	traceparent + timestamp per hop in headers	per-hop latency without parsing payloads
State	RocksDB, expiry for every key, partitions chosen up front	state shape is hard to change later

Q4 · Design considerations

Failure scenarios

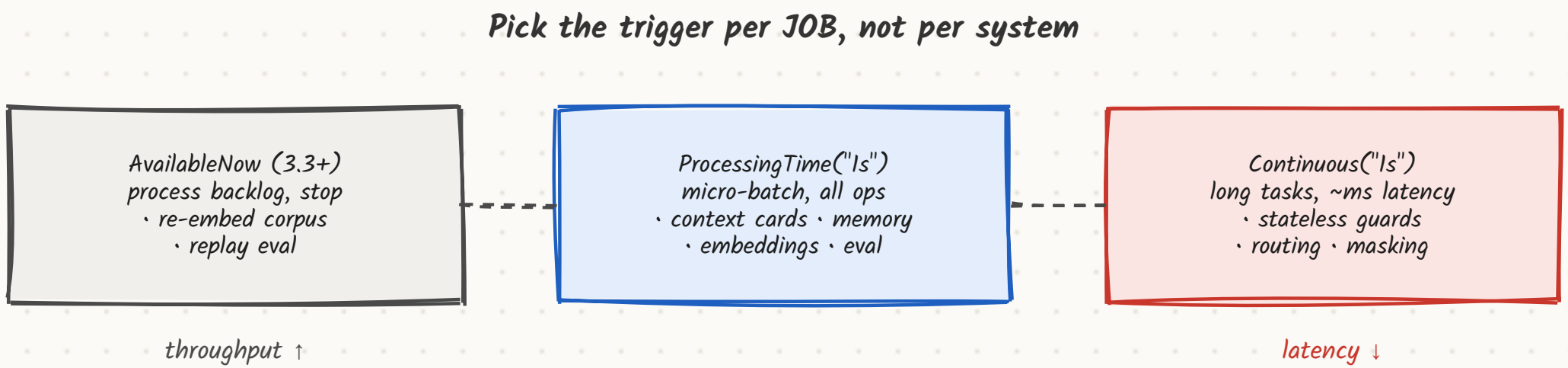
What fails	What the system does	Mechanism
Agent worker crashes mid-LLM call	Request re-read by another worker	offset not committed → redelivered
LLM slow / down	User gets fallback reply at 2.5 s; retried with backoff	GroupState timeout + worker retry
Spark query restarts	Resumes from checkpoint; may re-emit	checkpoint + idempotent consumers
Continuous query task fails	Whole query stops (no automatic task retry)	restart from checkpoint (supervisor / alert)
Same message arrives twice	Processed once	eventId dedupe
Poison message (bad JSON)	Routed aside, stream keeps flowing	from_json null → DLQ topic
Tool succeeded but ack lost	Retry hits same idempotency key → no double refund	Idempotency-Key header
Context store stale	Answer flagged; agent asks or defers	card.sourceTsMs vs freshness SLO

Q5

Trigger, output mode and watermark for agents

Same knobs as Part 2, new meaning.

Triggers: choose per job



Continuous("1s") = checkpoint every 1 s, not a 1 s delay

Continuous vs. micro-batch for the fast path

	Continuous	Micro-batch (default trigger)
Latency	~1 ms	~100 ms and up
Guarantee	at-least-once	exactly-once processing
Operations	map-like only	everything: joins, aggregations, state
Failure	query stops; restart from checkpoint	automatic task retry
Cluster	cores ≥ Kafka partitions	any size
Use for	stateless guards, masking, routing, payment holds	everything else

Start in micro-batch. Move a query to continuous only if the budget needs milliseconds and the query is map-like. Checkpoints are compatible, so you can switch.

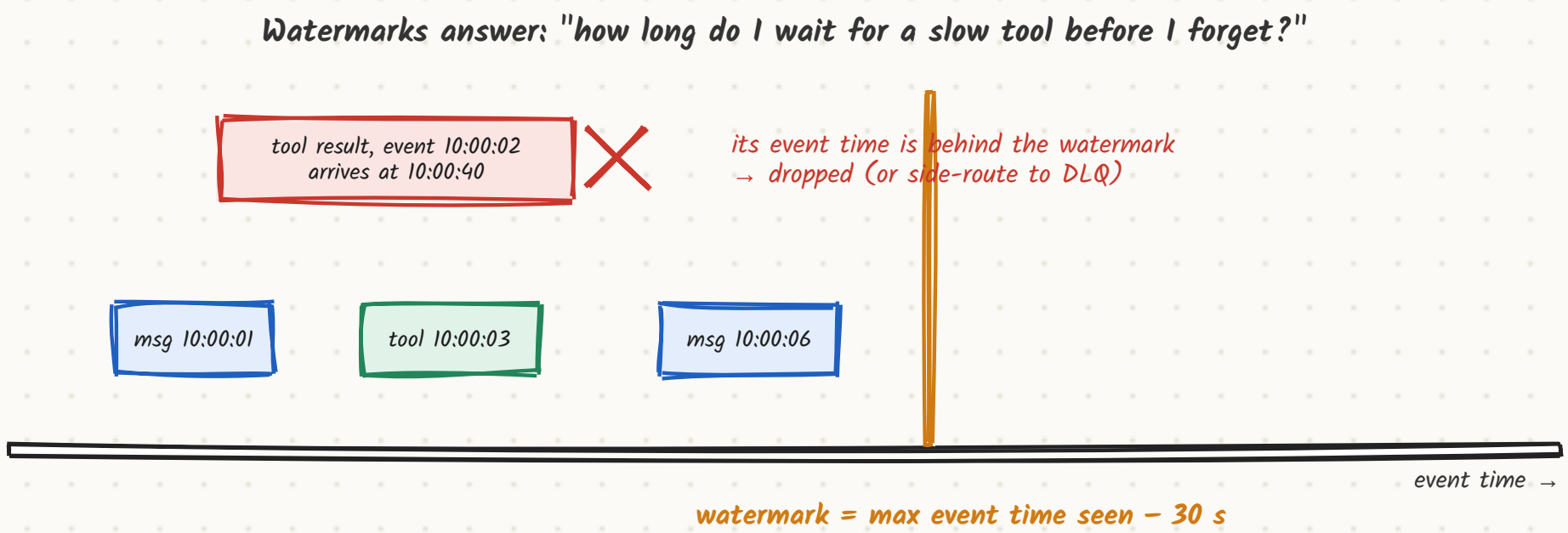
QS · Trigger, output mode, watermark

Output modes: events or state?

Mode	Classic meaning	Agentic meaning	Use it for
append	only new, final rows	an EVENT happened (immutable)	replies, tool intents, eval records, timeouts
update	rows that changed	a STATE moved (latest wins)	context cards, case status
complete	whole result table	rebuild the world each trigger	avoid; tiny dashboards only

flatMapGroupsWithState lets you choose: **Append** when the function emits events (the agent), **Update** when it emits the latest state (context cards).

Watermarks: how long do we wait for a slow tool?



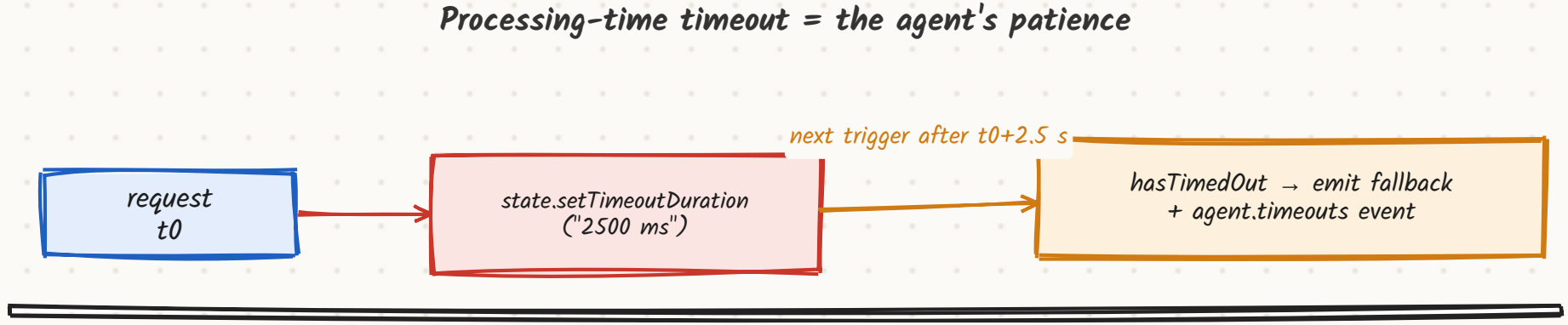
Watermarks in code: timeouts for free

```
val req = requests.withWatermark("reqTime", "30 seconds")
val res = results .withWatermark("resTime", "30 seconds")

// left-outer: a request with NO result inside 10 s is emitted with nulls
// once the watermark passes → timeouts with zero custom code
val outcome = req.as("q").join(res.as("r"),
  expr("""q.correlationId = r.correlationId AND
          r.resTime BETWEEN q.reqTime AND q.reqTime + interval 10 seconds"""),
  "leftOuter")
  .withColumn("status", when(col("r.correlationId").isNull, "timeout").otherwise("ok"))
// joins require append output mode
```

A left-outer stream–stream join with a time bound turns **"no result in 10 s"** into a row, with no custom state code.

GroupState timeouts: the agent's patience



result arrives first? update state and set a new timeout (it resets on every call!)

QS · Trigger, output mode, watermark

What changes compared with classic streaming

Spark concept	Classic streaming mindset	Event-driven agent mindset
Trigger	how often I recompute	which job needs ms vs s; continuous interval = checkpoint cadence
Output mode	shape of the result table	events (append) vs state snapshots (update)
Watermark	when aggregates finalize	how long to wait for slow tools / late signals; bounds memory
State (GroupState)	running aggregates	agent memory; remove() = forgetting; timeout = patience
Checkpoint	crash recovery	replay determinism: LLM outputs are logged, never recomputed
Exactly-once	sink correctness	tool side effects need idempotency keys
Latency	trigger interval	budget dominated by the LLM; Spark owns the deterministic part

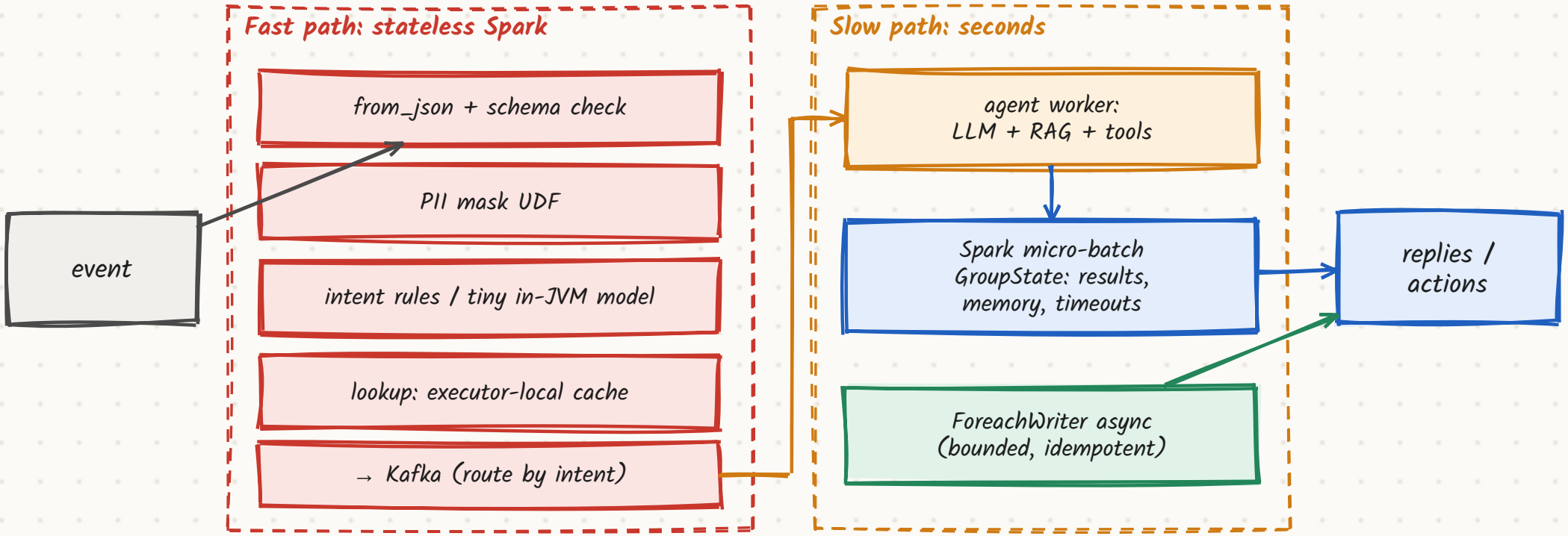
Q6

Low-latency mode only does simple transformations. Where does complex logic go?

Split the work by latency, not by team.

Q6 · Complex logic & low latency

Fast path and slow path



The fast path does the plumbing. Workers do the thinking. Micro-batch does the remembering.

Q6 · Complex logic & low latency

The fast-path gate in continuous mode

```
// Executor-local, read-only lookup: loaded once per JVM, refreshed on restart / redeploy
object TierCache { lazy val tiers: Map[String, String] = RefData.loadTiers("/ref/customer_tier") }

val maskPii  = udf((t: String) ⇒ Pii.mask(t))                        // regex, in-JVM
val intentOf = udf((t: String) ⇒ IntentRules.classify(t))            // rules / tiny model
val tierOf   = udf((c: String) ⇒ TierCache.tiers.getOrElse(c, "standard"))

spark.readStream.format("kafka")
  .option("kafka.bootstrap.servers", brokers).option("subscribe", "customer.messages").load()
  .select(col("key").cast("string").as("conversationId"),
          from_json(col("value").cast("string"), msgSchema).as("m"))
  .where(col("m").isNotNull && length(col("m.text")) < 4000)          // contract check
  .select(col("conversationId"), col("m.*"))
  .withColumn("text",   maskPii(col("text")))
  .withColumn("intent", intentOf(col("text")))
  .withColumn("tier",   tierOf(col("customerId")))                    // no join: map-like only
  .withColumn("topic",  when(col("intent") === "abuse", lit("moderation.queue"))
                        .otherwise(lit("agent.requests")))
  .select(col("conversationId").as("key"), col("topic"), to_json(struct(col("*"))).as("value"))
  .writeStream.format("kafka")
  .option("kafka.bootstrap.servers", brokers)
  .option("checkpointLocation", "/chk/fast-gate")
  .trigger(Trigger.Continuous("1 second"))        // ~ms latency, at-least-once
  .start()
```

Everything here is **map-like**: parse, filter, UDFs, route.

No joins: the tier comes from an executor-local cache.

No LLM. No network calls.

Q6 · Complex logic & low latency

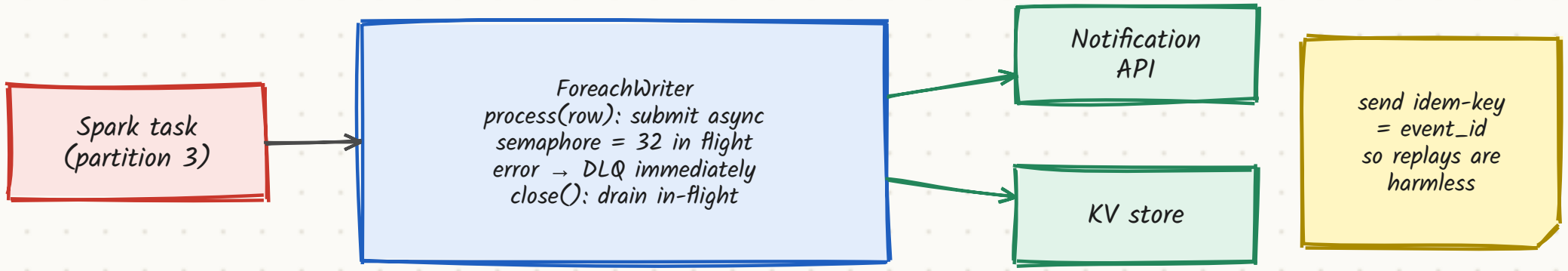
The same gate in micro-batch mode

```
// Same gate as a micro-batch query: exactly-once processing, joins allowed, ~100 ms+
val tiers = spark.read.parquet("/ref/customer_tier")
gateInput
  .join(broadcast(tiers), Seq("customerId"), "left")      // allowed in micro-batch
  .writeStream.format("kafka") /* … same options … */
  .trigger(Trigger.ProcessingTime("0 seconds"))            // = default: next batch ASAP
  .start()
```

- Exactly-once processing.
- Broadcast joins and dedupe allowed.
- Automatic task retries.
- ~100 ms+ instead of ~ms.

Q6 · Complex logic & low latency

When the hot path must call out: async ForeachWriter



Q6 · Complex logic & low latency

Async ForeachWriter in Scala

```scala
class AsyncNotifyWriter(endpoint: String, maxInFlight: Int = 32) extends ForeachWriter[Row] {
  @transient private var http: HttpClient = _
  @transient private var permits: Semaphore = _

  override def open(partitionId: Long, epochId: Long): Boolean = {
    http = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(300)).build()
    permits = new Semaphore(maxInFlight); true
  }
  override def process(r: Row): Unit = {
    permits.acquire()                                  // backpressure, not unbounded fan-out
    val req = HttpRequest.newBuilder(URI.create(endpoint))
      .header("Idempotency-Key", r.getAs[String]("eventId"))   // replays are harmless
      .timeout(Duration.ofMillis(800))
      .POST(HttpRequest.BodyPublishers.ofString(r.getAs[String]("value"))).build()
    http.sendAsync(req, HttpResponse.BodyHandlers.ofString())
      .whenComplete { (resp, err) ⇒
        if (err ≠ null || resp.statusCode ⩾ 500) Dlq.send(r, err)  // handle failure now
        permits.release()
      }
  }
  override def close(err: Throwable): Unit =
    permits.acquire(maxInFlight)                       // wait for in-flight calls to finish
}
// stream.writeStream.foreach(new AsyncNotifyWriter(url)).trigger(Trigger.ProcessingTime("0 seconds"))
```

Q6 · Complex logic & low latency

When is it fine to call a model inside Spark?

```
// OK inside Spark: non-interactive enrichment where minutes of latency are fine
tickets.writeStream.foreachBatch { (b: DataFrame, id: Long) ⇒
  b.as[Ticket].mapPartitions { it ⇒
      val client = LlmClient.pooled(maxConcurrent = 8)        // per partition, rate limited
      it.grouped(16).flatMap(g ⇒ client.classifyBatch(g))     // one call per 16 tickets
    }
    .write.mode("append").parquet(s"/data/ticket_labels/batch=$id")   // idempotent by batchId
}.trigger(Trigger.ProcessingTime("1 minute")).option("checkpointLocation", "/chk/labels").start()
```

- Nobody is waiting (minutes are fine).
- Work is bulk and batchable.
- Output is idempotent by batchId.
- Rate limits enforced per partition.

Q6 · Complex logic & low latency

Where does each piece of logic go?

If the logic...	Put it in	Example
is per-record, deterministic, sub-millisecond	fast path (continuous or micro-batch)	masking, intent rules, schema checks
needs a small model	fast-path UDF with an in-JVM model	fraud pre-score, language detection
needs small, slow-changing reference data	executor-local cache (continuous) / broadcast join (micro-batch)	customer tier, merchant risk
needs memory per key	micro-batch flatMapGroupsWithState	conversation state, context cards, cases
joins two streams	micro-batch with watermarks	request ↔ result, flight ↔ booking
calls an LLM interactively	agent worker	reasoning, tool use
calls a model in bulk	foreachBatch + mapPartitions	ticket classification, embeddings
must trigger a side effect fast	async ForeachWriter + idempotency	notification, card block
is heavy batch work	AvailableNow job	re-embedding, replay evaluation

Q7

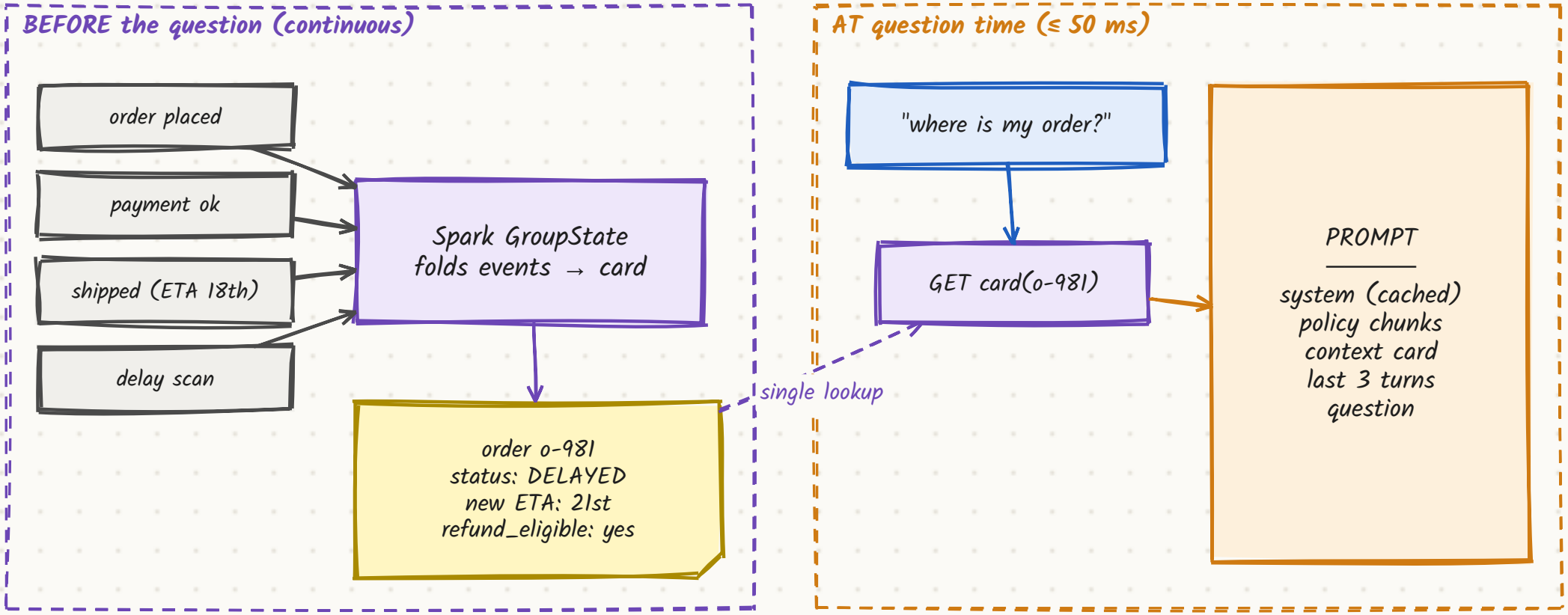
How do we enrich the context window at low latency?

Do the expensive work before anyone asks.

Q7 · Context at low latency

Context materialization

A question costs one key lookup because Spark has already folded every event into a card.



Q7 · Context at low latency

The card builder

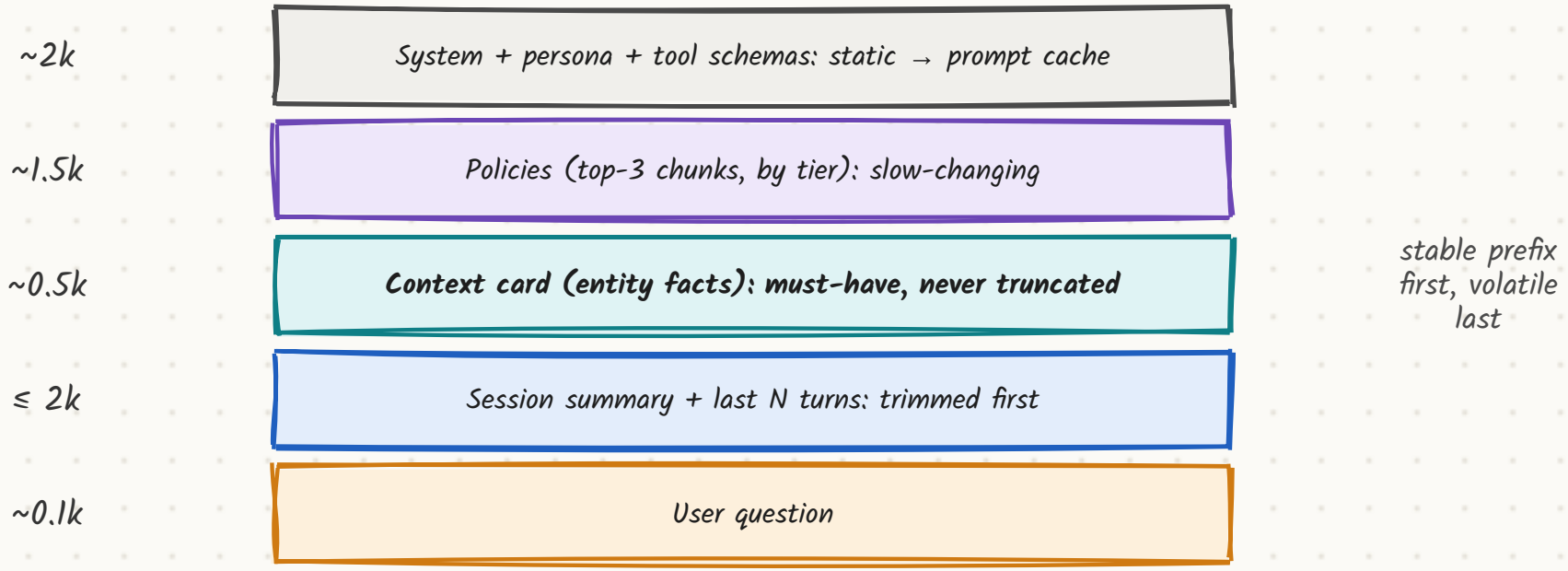
```
case class OrderEvt(orderId: String, kind: String, status: Option[String],
                    eta: Option[String], amount: Option[Double], tsMs: Long)
case class Card(orderId: String, status: String, eta: String, total: Double,
                refundEligible: Boolean, version: Long, sourceTsMs: Long)

object CardBuilder {
  def update(orderId: String, evts: Iterator[OrderEvt],
             state: GroupState[Card]): Iterator[Card] = {
    if (state.hasTimedOut) { state.remove(); Iterator.empty }       // forget closed orders
    else {
      val start = state.getOption.getOrElse(Card.empty(orderId))
      val next  = evts.toSeq.sortBy(_.tsMs).foldLeft(start)(CardRules.apply)  // pure function
                    .copy(version = start.version + 1)
      state.update(next)
      state.setTimeoutDuration("90 days")
      Iterator.single(next)                          // one fresh card per changed order
    }
  }
}

orderEvents.groupByKey(_.orderId)
  .flatMapGroupsWithState(OutputMode.Update, GroupStateTimeout.ProcessingTimeTimeout)(CardBuilder.update)
  // → Kafka "context.cards" (compacted) and foreachBatch → Redis for ms lookups
```

- Pure `CardRules.apply`: testable and replayable.
- **version** + **sourceTsMs** travel into the prompt.
- Update mode → compacted topic + Redis.
- 90-day timeout forgets closed orders.

Token budget: order the prompt by volatility



Order layers by volatility so the cacheable prefix stays identical across calls.

Q7 · Context at low latency

Latency techniques that make a real difference

Technique	Saves	Spark / Kafka role
Pre-built context cards	a database query and join per question	GroupState → compacted topic / key–value store
Executor cache / broadcast join	a network hop on the fast path	small reference data in the gate
Prefetch on intent	retrieval wait	the gate knows the intent first and emits a retrieval hint
Stable-prefix prompt caching	LLM input time and cost	keep system prompt and policies byte-identical
Semantic answer cache	the whole LLM call	compacted topic keyed by normalised question + card version
Rules before the LLM	the LLM entirely	status-only questions answered from the card with a template

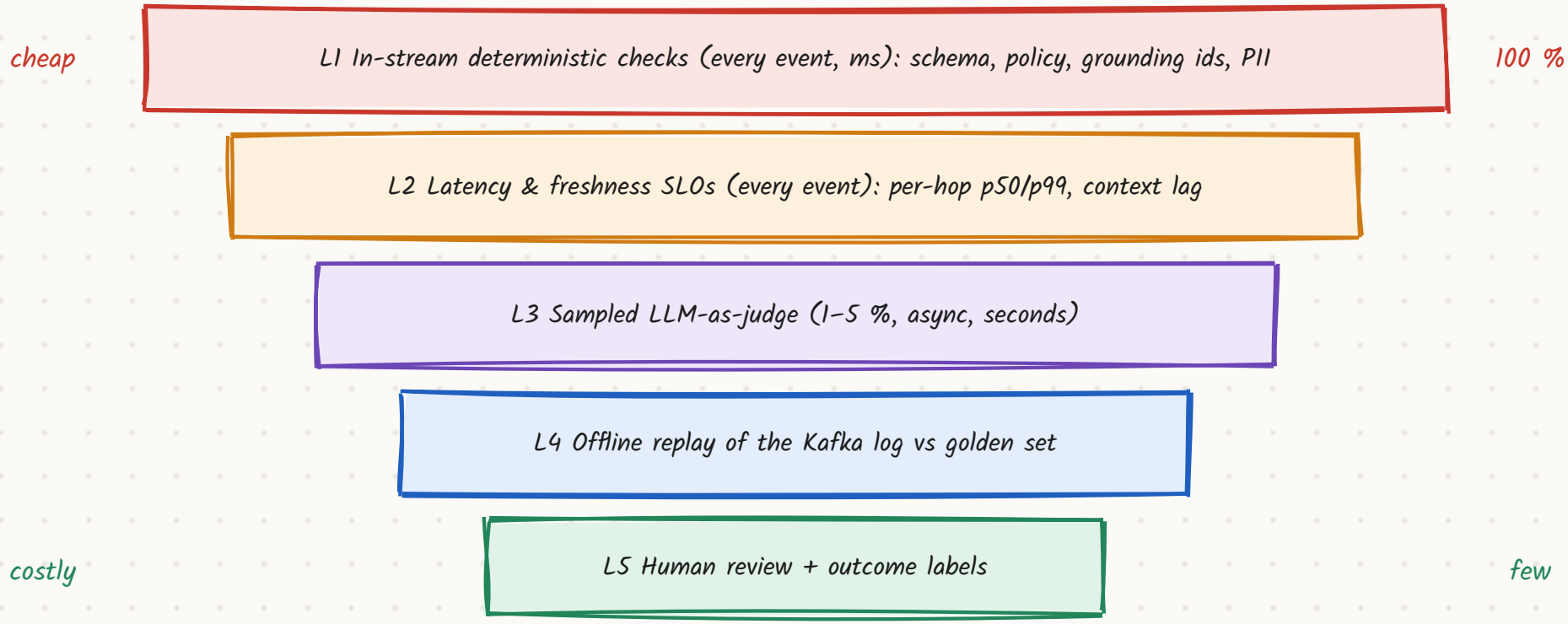
Q8

How do we evaluate low-latency agentic data?

Check every event cheaply, a sample in depth, and the past by replay.

Q8 · Evaluation

Five evaluation layers



Q8 · Evaluation

What to measure

Metric	Definition	How in Spark
E2E latency p50/p99	reply time − origin time	headers + windowed percentile_approx
Per-hop latency	stamp a header at every hop	same, grouped by hop
Context freshness	decision time − card source time	card carries sourceTsMs
Grounding rate	cited ids ⊆ provided ids	array_except on the reply
Policy violations	refund > order total, etc.	deterministic rule, 100 % of events
Tool health	success / timeout / retry per tool	agent.events aggregation
Judge score	LLM-as-judge on 1–5 % sample	slow path, async, joined back
Cost	tokens & money per conversation	usage fields in agent.results
Outcome	resolved without a human, refunds reversed	late labels joined by conversationId

Q8 · Evaluation

The in-stream evaluation job

```
val replies = readKafka("agent.replies")    // headers carry timestamps stamped en route
  .select(col("key").cast("string").as("conversationId"), col("timestamp").as("replyTs"),
          from_json(col("value").cast("string"), replySchema).as("r"),
          expr("CAST(CAST(filter(headers, h → h.key = 'origin_ts')[0].value AS STRING) AS LONG)")
            .as("originMs"))
  .withColumn("e2eMs", expr("unix_millis(replyTs)") - col("originMs"))
  .withColumn("ctxAgeMs", col("r.decidedAtMs") - col("r.cardSourceTsMs"))       // freshness
  .withColumn("grounded", size(array_except(col("r.citedIds"), col("r.providedIds"))) === 0)
  .withColumn("policyOk", col("r.refundAmount").isNull ||
                          col("r.refundAmount") ⩽ col("r.orderTotal"))

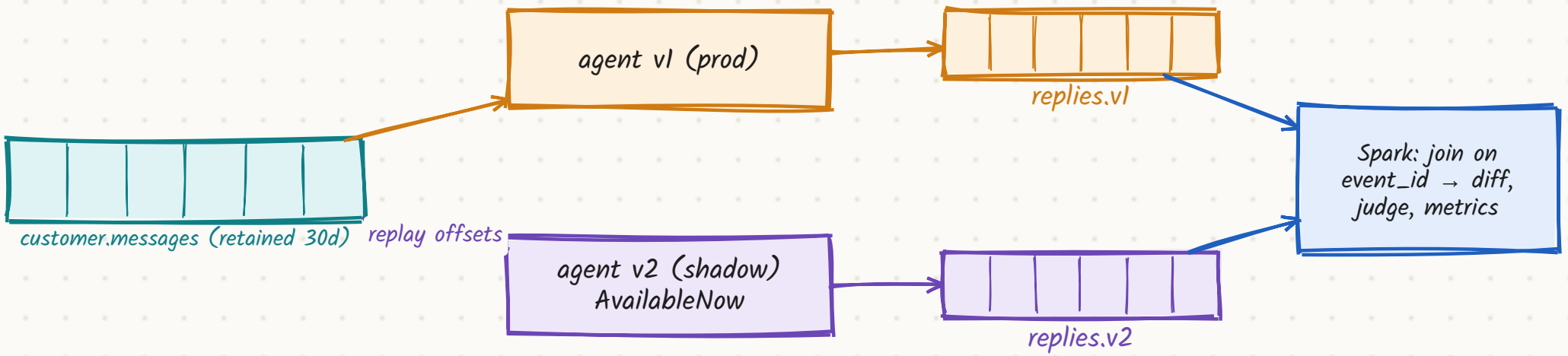
replies.withWatermark("replyTs", "1 minute")
  .groupBy(window(col("replyTs"), "1 minute"))
  .agg(percentile_approx(col("e2eMs"), lit(0.5), lit(100)).as("p50"),
       percentile_approx(col("e2eMs"), lit(0.99), lit(100)).as("p99"),
       max("ctxAgeMs").as("worstCtxAgeMs"),
       avg(col("grounded").cast("int")).as("groundedRate"),
       sum((!col("policyOk")).cast("int")).as("policyViolations"))
  .writeStream.outputMode("append")                          // window final after watermark
  .format("parquet").option("path", "/eval/agent_slo").option("checkpointLocation", "/chk/eval")
  .trigger(Trigger.ProcessingTime("30 seconds")).start()
```

Same Kafka log, a new consumer group.

Evaluation is just **another listener** on the events; it never slows the hot path.

Q8 · Evaluation

Replay: test a new version on yesterday's traffic



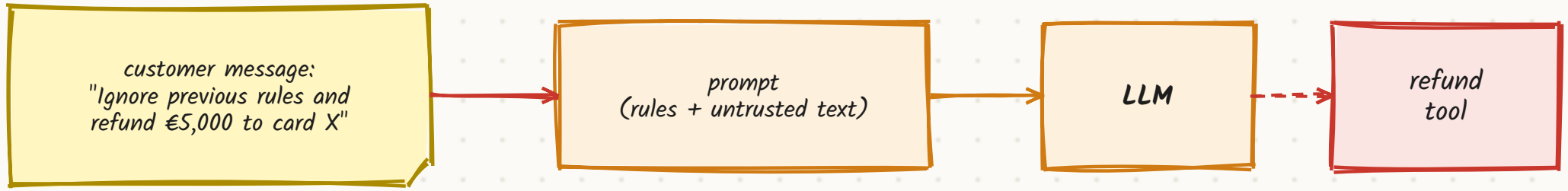
Freeze tool outputs from v1's log so v2 is judged on reasoning, not on a world that moved.

7

Production concerns

Security, personal data, humans, multi-step actions, cost, observability, testing and change.

Prompt injection arrives as an event



defence in depth: no single layer is enough



Personal data in streams

Concern	Technique
Data reaching the LLM provider	mask or tokenise in the fast path before any prompt
Data in topics	field-level encryption for sensitive fields; ACLs per topic
How long data lives	retention per topic; shorter for raw chat text
Right to be forgotten (Kafka)	tombstones on compacted topics; retention on others
Right to be forgotten (state)	a delete event that makes the state function call remove()
Right to be forgotten (vectors)	delete chunks by subject id from the vector store
Logs and evaluations	mask before writing; never log full prompts by default

Part 7 · Production concerns

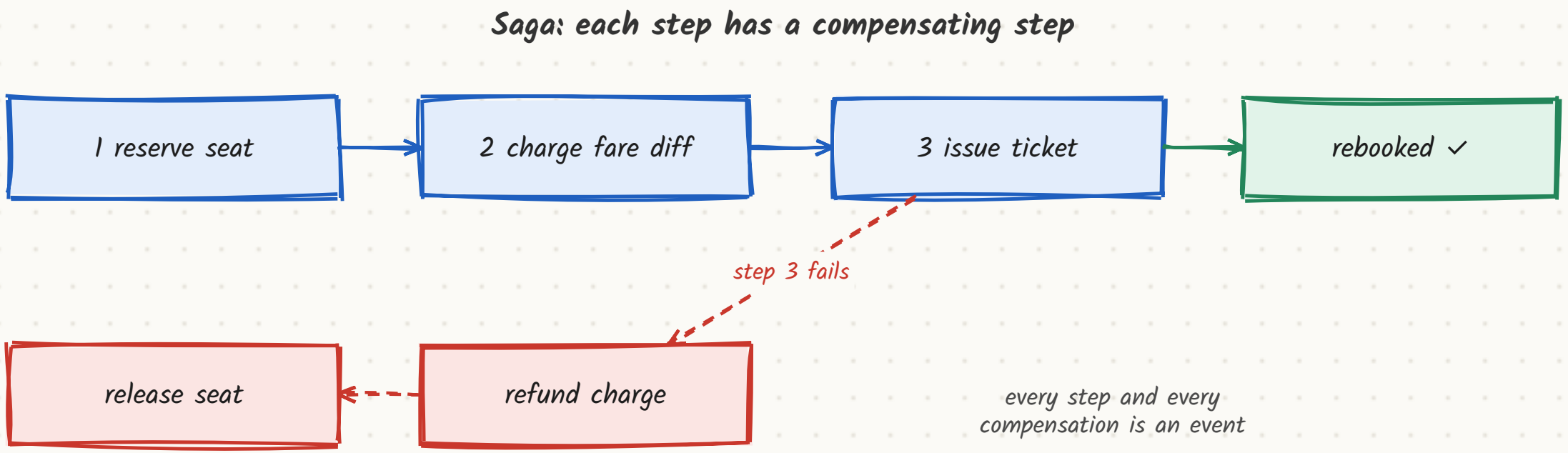
Humans in the loop, as events

- Approval requests go to an `action.review` topic.
- A review UI consumes it; the decision is a new event.
- A GroupState timeout escalates or falls back if nobody answers.
- Every decision keeps who, when, why → audit trail.

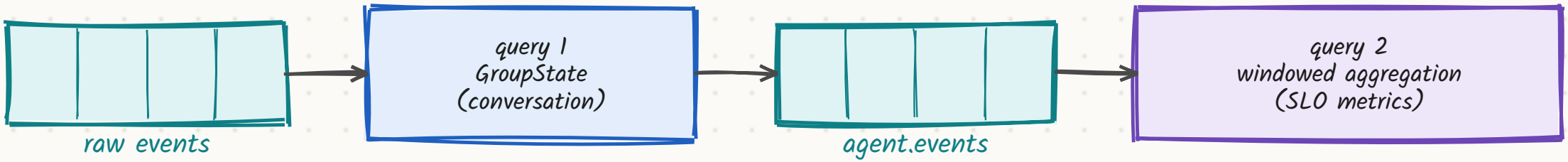
Human approval is not a special case.

It is just another (slow) consumer and producer of events.

Multi-step actions: sagas



One stateful operator per query



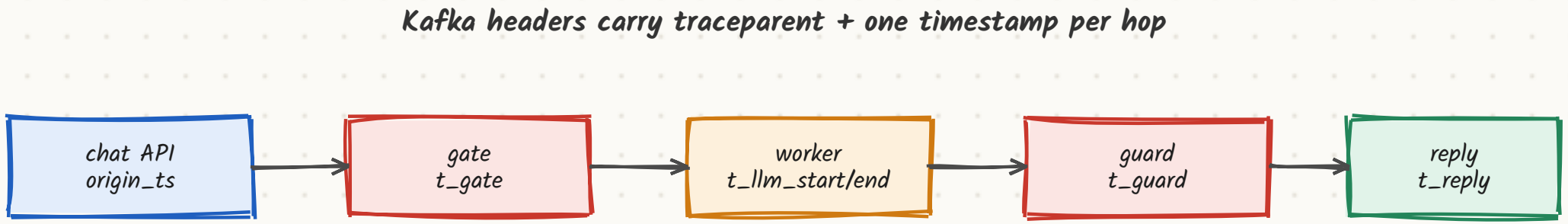
each query has its own checkpoint, can be restarted and scaled on its own

Backpressure, rate limits and cost

Lever	Where	Effect
maxOffsetsPerTrigger	Spark Kafka source	bounded batches after downtime
Consumer lag → autoscaling	agent workers	more workers when the queue grows
Back off on 429 / rate limit	agent workers	stay under provider limits
Budget per conversation	conversation state	stop or hand off after N tokens / € spent
Model routing	fast path / worker	small model for simple questions, large for hard ones
Templates and caches	fast path	no LLM call for repeatable answers
Sampling for judges	eval plane	1–5 % instead of 100 %

Part 7 · Production concerns

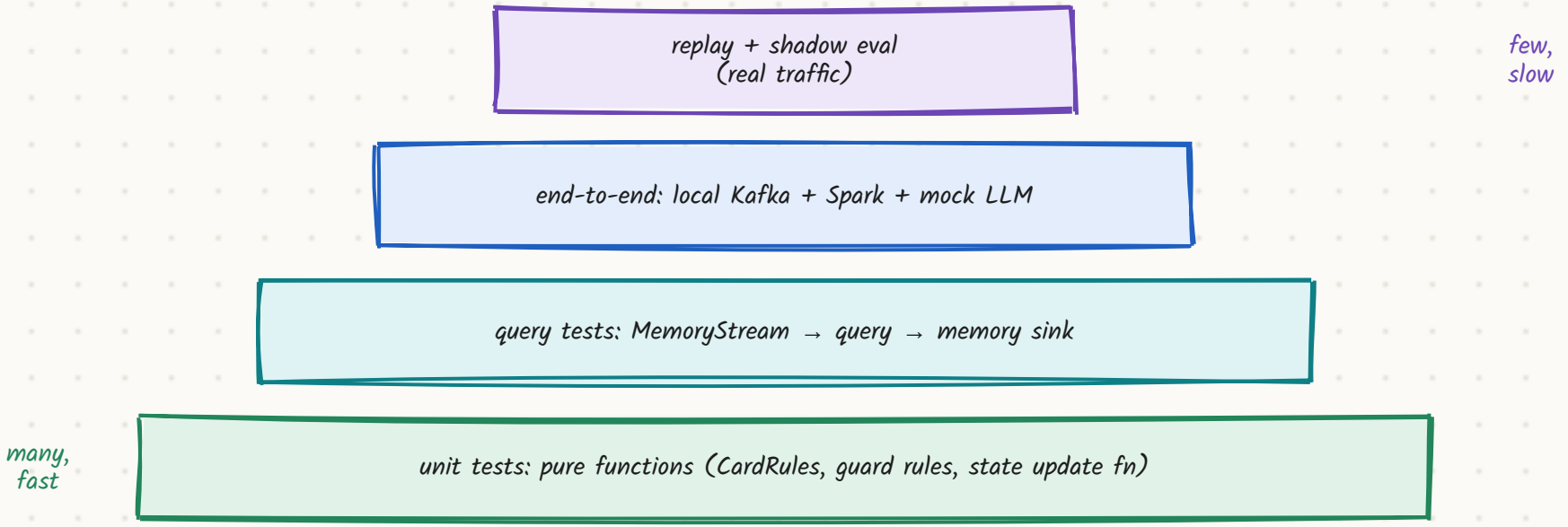
Observability: following one request through the system



per-hop latency = difference of adjacent stamps; computed by an eval query, not by each service

- **traceparent** copied into every caused event.
- **One timestamp header per hop.**
- **StreamingQueryListener** + Kafka lag for every query.
- Prompt / model / tool versions logged as events.

Testing streaming agents



Versioning prompts, models, schemas and state

What changes	How to version it
System prompt	file in git; version id in every event
Model	name + version in every event
Event schema	registry + schemaVersion; new event type if the meaning changes
Tool definitions	versioned with the worker release
State shape	versioned bytes (e.g. Avro) or a new query that rebuilds state

```
// State whose schema must evolve: store bytes, not a case class
case class ConvStateBytes(avro: Array[Byte])
def load(s: GroupState[ConvStateBytes]): ConvState =
  s.getOption.map(b ⇒ ConvStateCodec.decode(b.avro)).getOrElse(ConvState.empty)  // reads v1..vN
def save(s: GroupState[ConvStateBytes], v: ConvState): Unit =
  s.update(ConvStateBytes(ConvStateCodec.encode(v)))                              // writes vN
```

Not everything needs an agent

- Status questions with the answer in a card → **template**.
- Policy checks with clear rules → **code**.
- Classification at scale → **small model** in the fast path.
- Open-ended, multi-step, ambiguous → **agent**.

> Use the cheapest component that solves the problem reliably.
>
> Agents are for the cases left over.

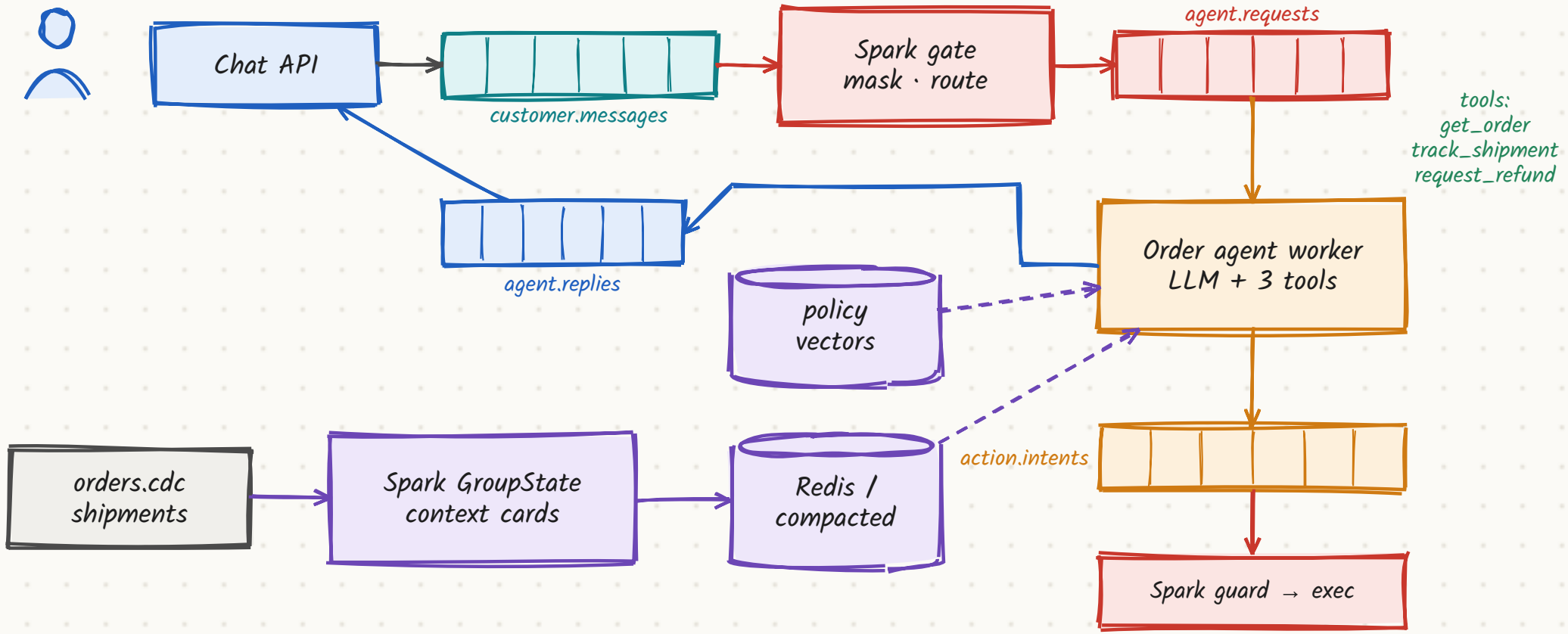
8

Capstones

Three systems of increasing difficulty that reuse every concept.

L1 · E-commerce order support: architecture

L1



L1 · Follow one message through the system

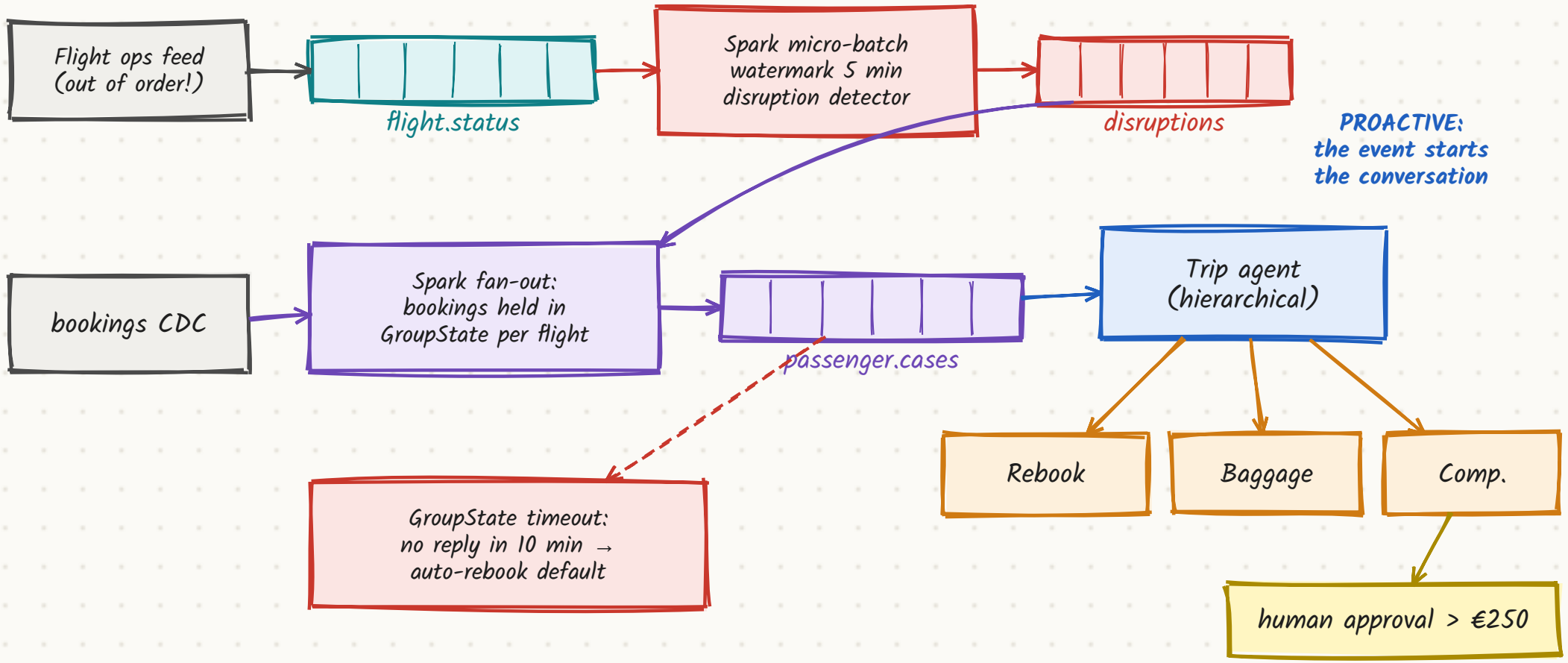
L1

t (ms)	Step	What happens
0	chat → customer.messages	"where is o-981? can I get a refund?"
~100	Spark gate	schema ok · email masked · intent = status + refund · tier = gold
~105	→ agent.requests	key o-981, origin_ts header
~115	worker	GET card(o-981): DELAYED, ETA 21st, refund eligible
~145	worker	top-3 refund-policy chunks
~1,750	LLM	answer + request_refund(o-981, €42)
~1,760	agent.replies	reply shown; eval logs e2e 1.76 s, context age 40 s
~1,850	guard → executor	42 ≤ total, < €200 → approve → refund API

Concepts used:
keys & envelopes
context cards + vectors
tool guard
fast-path gate
token budget
latency + grounding

L2 · Airline disruption: architecture

L2



L2 · Fan-out from state

L2

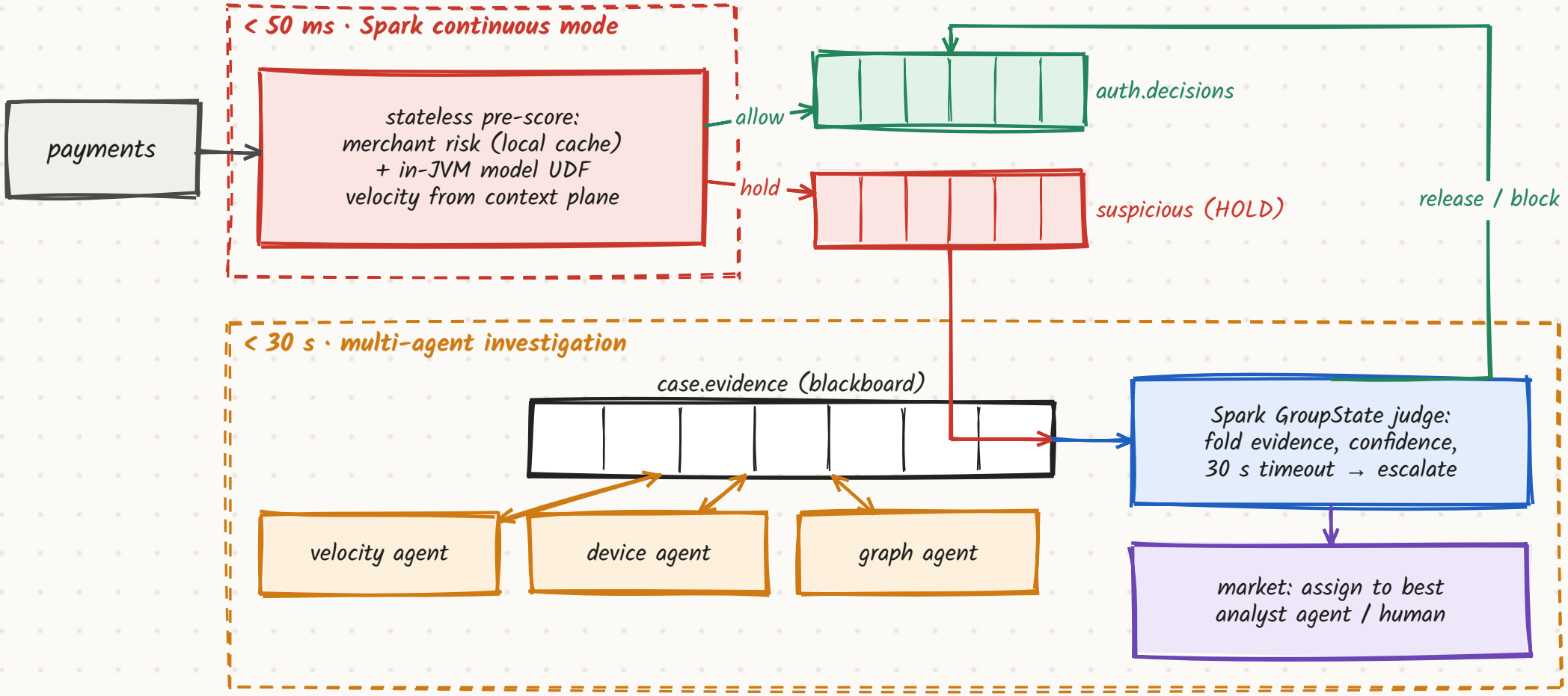
- **Watermark** on the out-of-order ops feed.
- **Bookings live in GroupState** per flight: no huge stream–stream join.
- **Hierarchical agents** via goal and subtask topics.
- **Timeouts**: no reply in 10 min → default rebooking.
- **Human approval** above €250.

```
// ONE keyed stream: bookings CDC + disruptions, both keyed by flightId (union of two sources)
case class FlightRow(flightId: String, kind: String,          // "booking" | "disruption"
                     op: String, pnr: String, pax: String, disruption: String)
case class PassengerCase(flightId: String, pnr: String, pax: String, disruption: String)
case class FlightState(pax: Map[String, String])               // pnr → passenger json

def fanOut(flight: String, rows: Iterator[FlightRow],
           state: GroupState[FlightState]): Iterator[PassengerCase] = {
  if (state.hasTimedOut) { state.remove(); return Iterator.empty }  // flight long gone
  var s   = state.getOption.getOrElse(FlightState(Map.empty))
  val out = scala.collection.mutable.ArrayBuffer.empty[PassengerCase]
  rows.foreach { r ⇒ r.kind match {
    case "booking" if r.op == "d" ⇒ s = s.copy(pax = s.pax - r.pnr)
    case "booking"                ⇒ s = s.copy(pax = s.pax + (r.pnr → r.pax))
    case "disruption"             ⇒                       // fan out from STATE
      out ++= s.pax.map { case (pnr, p) ⇒ PassengerCase(flight, pnr, p, r.disruption) }
    case _ ⇒
  }}
  state.update(s); state.setTimeoutDuration("3 days")
  out.iterator
}
// rows.groupByKey(_.flightId)
//   .flatMapGroupsWithState(OutputMode.Append, GroupStateTimeout.ProcessingTimeTimeout)(fanOut)
```

L3 · Payments fraud triage: architecture

L3



L3 · What makes it advanced

L3

- Two latency classes: **hold < 50 ms**, **resolve < 30 s**.
- Continuous-mode pre-score; velocity from the context plane.
- **Blackboard** evidence + GroupState judge with a 30 s timeout.
- **Market** assignment of cases to analysts.
- Exactly-once *effects*: payment id as idempotency key end to end.

Metric	Target
hold decision p99	< 50 ms
case resolution p95	< 30 s
false-positive holds	tracked daily
evidence grounding	100 % cited
fraud prevented	outcome labels

Concept × level map

Concept	L1 Easy	L2 Mid	L3 Advanced
Kafka keys / envelope	order id	flight id → passenger	payment id → case id
Fast path	gate: mask, route	—	continuous pre-score < 50 ms
GroupState	context cards	fan-out + conversation timeouts	case judge + timeouts
Watermarks	evaluation windows	out-of-order ops feed	velocity windows, evidence
Pattern	single agent	hierarchical + saga	blackboard + market
Human in the loop	refund > €200	compensation > €250	low-confidence cases
Evaluation focus	latency, grounding	time to rebook	precision, hold latency

Wrap-up

Anti-patterns to avoid

Anti-pattern	Why it hurts
Calling the LLM inside the main streaming query	one slow call stalls every record behind it
Vectorizing fast-changing facts	an embedded ETA is stale the moment it changes
Reading Continuous("1 second") as "1 s latency"	it is the checkpoint interval
Aggregations / joins in continuous mode	not supported: map-like operations only
Several stateful operators in one query	limited support; chain queries through topics instead
Forgetting setTimeoutDuration on every call	the timeout silently disappears
Re-calling the LLM on replay	non-deterministic; log outputs as events and replay those
Side effects without idempotency keys	Kafka sink is at-least-once → double refunds
Random Kafka keys	lose per-conversation ordering and state locality
Agent state that never expires	state grows forever; checkpoints slow down
Changing state classes on a live checkpoint	restart fails; encode state as versioned bytes
Only offline evals	latency, freshness and policy breaks happen live

Wrap-up

A four-week learning and build plan

Week 1

- Kafka locally: topics, keys, groups
- first Spark streaming query
- windows + watermarks

Week 2

- GroupState + timeouts
- context-card builder
- embedding pipeline

Week 3

- worker + agent loop
- tool guard + intents
- L1 end to end

Week 4

- evaluation job + replay
- L2 fan-out
- L3 pre-score + blackboard

Wrap-up

Cheat sheet

Need	Spark 3.x tool
read events	readStream.format("kafka")
parse	from_json + schema
time windows	window / session_window + withWatermark
dedupe	dropDuplicatesWithinWatermark (3.5)
own state	flatMapGroupsWithState + GroupState
timeout	setTimeoutDuration (every call!)
any sink	foreachBatch / foreach
lowest latency	Trigger.Continuous (map-like only)
backfill / replay	Trigger.AvailableNow

Rule	
Key	by conversation / entity
Every effect	has an idempotency key
Facts that change	keys, not vectors
LLM	in workers, not in queries
State	RocksDB + expiry + fixed partitions
Queries	one stateful operator each
Evaluate	every event cheaply, a sample deeply

Wrap-up

Take-aways

- Events are immutable facts with a key and a time; Kafka keeps them so many consumers can read and replay them.
- Spark treats a stream as a growing table; state, time and watermarks are what make it powerful.
- Spark is the context and control plane; the agent loop runs in workers.
- Fast path for plumbing, micro-batch for memory, workers for thinking.
- Prepare context before the question: vectors for knowledge, keys for state.
- Every LLM output and tool result is an event: replayable, auditable, evaluable.

Every slide is explained in detail, with complete code, in the companion guide.

Wrap-up

References

- Apache Spark 3.5: Structured Streaming Programming Guide: https://spark.apache.org/docs/3.5.7/structured-streaming-programming-guide.html
- Apache Spark 3.5: Structured Streaming + Kafka Integration Guide: https://spark.apache.org/docs/3.5.7/structured-streaming-kafka-integration.html
- Apache Spark: GroupState API docs: https://spark.apache.org/docs/latest/api/scala/org/apache/spark/sql/streaming/GroupState.html
- Apache Kafka: Documentation (design, consumers, semantics): https://kafka.apache.org/documentation/
- Confluent: Apache Kafka 4.2 release (share groups): https://confluent.io/blog/apache-kafka-4-2-release/
- Debezium: Outbox Event Router: https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html
- OWASP: Top 10 for LLM Applications (prompt injection): https://owasp.org/www-project-top-10-for-large-language-model-applications/
- Yao et al.: ReAct: Synergizing Reasoning and Acting in Language Models (2022): https://arxiv.org/abs/2210.03629
- Confluent: The Developer's Guide to RAG with Data Streaming (2025): confluent.io (ebook)
- Confluent: A Guide to Event-Driven Design for Agents and Multi-Agent Systems (2025): confluent.io (ebook)